

Methods S1: FLIM Playground Manual

(related to STAR Methods)

Table of contents

1	Quick Start	7
1.1	Installation	8
1.2	Introduction	8
1.3	Challenges	9
1.4	Method	10
1.4.1	Feature Classes	10
1.4.2	Design	11
1.4.3	Summary	13
1.4.4	Citation	14
I	Data Extraction	15
2	Overview	16
2.1	Stages	16
2.2	Input File Types	17
2.2.1	Decay	17
2.2.2	Mask	18
2.2.3	IRF	18
2.2.4	SPCImage t1	18
2.2.5	Fluorescence Lifetime Standard	18
2.2.6	Intensity (2D)	19
2.3	Limitations	19
3	Configuration	20
3.1	Extraction Profiles	21
3.2	Cross-Channel Configurations	22
3.2.1	Categorical Features	22
3.2.2	Decay Types	22
3.2.3	Identifiers	23
3.2.4	Decay Info	24
3.2.5	Calibration Method	24
3.2.6	Number of Channels	24
3.3	Per-Channel Configurations	25
3.3.1	Channel Name	25

3.3.2	Imaging Modality	25
3.3.3	Feature Extractor	26
3.3.4	Number of Components	27
3.3.5	Fixed Lifetimes	27
3.3.6	File Suffix	27
3.4	Derived Features	28
3.5	Save	28
4	Field of View Metadata	29
4.1	Metadata	29
4.1.1	Decay Type	30
4.1.2	Channel Names	30
4.1.3	Feature Extractors	30
4.1.4	Decay Info	30
4.1.5	File Suffix	31
4.1.6	Fluorescence Lifetime Standard	31
4.1.7	Folder Path	31
4.2	Metadata Extraction	32
4.2.1	FOV File Paths	32
4.2.2	Decay Info	33
4.2.3	Result Preview and Export	34
5	Numerical Feature Extraction	36
5.1	Input	36
5.2	Calibration	37
5.2.1	Fit Calibration	37
5.2.2	Fit Free Calibration	43
5.2.3	Confirm Calibration	44
5.3	Feature Extraction	44
5.4	Save Results	45
6	Lifetime Features (Fit)	46
6.1	Fitting	46
6.1.1	Preprocessing	47
6.1.2	Fraction of components	47
6.1.3	Mean lifetime	48
6.1.4	Intensity-weighted mean lifetime	48
6.1.5	Goodness of fit	48
6.2	Pixel pre-fit	49
7	Lifetime Features (Fit-Free)	50
7.1	Preprocessing	50

7.2	Phasor features	51
7.2.1	Step 1: raw phasor coordinates	51
7.2.2	Step 2: calibrated phasor coordinates	52
7.2.3	Step 3: phasor-derived lifetime	52
8	Morphological Features	53
9	Textural Features	54
9.1	Granularity	54
9.2	Radial Distribution	54
9.3	Mass Displacement	54
9.4	Intensity Sum	55
10	Derived Features	56
10.1	Build Derived Features	56
10.1.1	Templates	57
10.1.2	Custom Formulas	57
10.2	Compute Derived Features	58
10.3	Derived Feature In Data Analysis	58
11	Categorical Feature Extraction	59
11.1	Merging Datasets	59
11.2	Assigning Categorical Features	61
II	Data Analysis	63
12	Overview	64
12.1	General Workflow	64
12.2	Methods	64
12.3	Input	66
12.3.1	Requirements	66
12.3.2	Warning Messages	67
12.3.3	Error Messages	67
12.4	Shared Interactive Widgets	67
12.4.1	Numerical Selection Widgets	68
12.4.2	Filter Widgets	69
12.4.3	Visual Channels Widgets	70
12.4.4	Unique ID Hover	71
12.4.5	Plotting Configuration Widgets	72
12.4.6	Feature Labels	72
12.4.7	Export Workflow as Python Script	73

13 Configuration	74
13.1 Analysis Profiles	74
13.2 Identifiers Config	75
13.3 Categorical Feature Config	75
13.4 Numerical Feature Config	75
13.4.1 Example	76
13.5 Save	76
13.6 Reset	76
 III Univariate Analysis	 77
14 Feature Comparison	78
14.1 Shared Interface Components	78
14.2 Plot Options	79
14.3 Separate by	79
14.4 Comparison Widget	80
14.5 Effect Size	81
14.5.1 Glass's Delta	81
14.5.2 Absolute Cohen's d	82
14.6 Statistical Test	83
14.7 Comparison Annotation	83
 15 Feature Histogram	 85
15.1 Shared Interface Components	86
15.2 Histogram Mode	86
15.3 Gaussian Mixture Model Mode	88
15.3.1 Fit Gaussian Mixture Models	88
15.3.2 Heterogeneity index	89
15.3.3 Classification	89
15.3.4 Example	90
 16 Field of View Comparison	 93
16.1 Shared Interface Components	94
16.2 Example	94
 IV Bivariate Analysis	 96
17 Feature Distribution	97
17.1 Shared Interface Components	98
17.2 Log-scale Axes	98
17.3 Marginal Distribution	98
17.4 2D Gaussian Mixture Model	99

17.5 Regression line	100
18 Phasor Analysis	102
18.1 Shared Interface Components	103
18.2 Select Phasor Coordinate	103
18.3 K-means Clustering	103
18.3.1 Export Clustered Dataset	104
 V Multivariate Analysis	 105
19 Dimension Reduction	106
19.1 Shared Interface Components	107
19.2 Hyperparameter Widget	107
19.2.1 UMAP	108
19.2.2 t-SNE	108
20 Classification	110
20.1 Shared Interface Components	111
20.2 Method-specific Components	111
20.2.1 Classifier Hyperparameters	113
20.2.2 Class Imbalance Handling	118
20.2.3 Confusion Matrix	123
20.2.4 Performance Metrics	123
20.2.5 ROC Curve	124
20.2.6 Feature Importance	125
 21 Common Q & A	 126

1 Quick Start

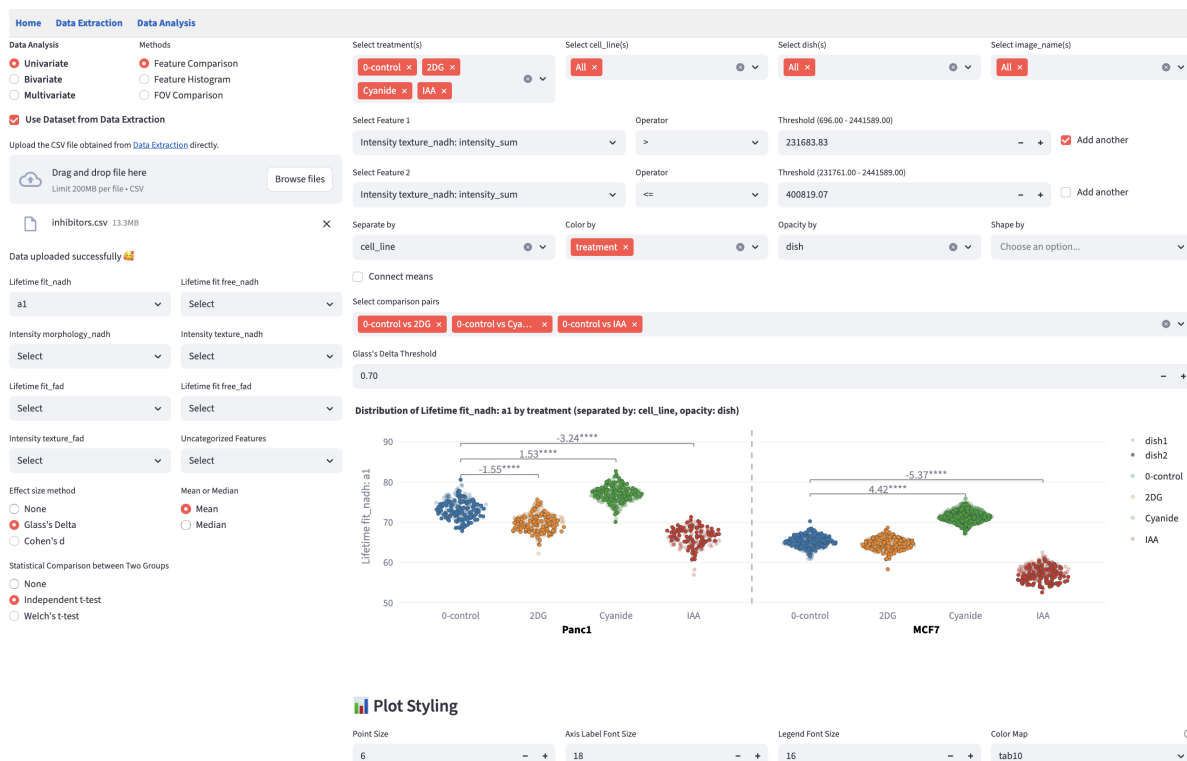
Welcome to the FLIM Playground 🥳🎉! This is an interactive graphical user interface (GUI) that allows you to extract single-cell¹ features from fluorescence lifetime imaging microscopy (FLIM) raw data ([Data Extraction](#)) and analyze extracted features or your own datasets using a built-in repertoire of methods ([Data Analysis](#)).

To quickly try out different analysis methods, download this [sample dataset](#) and try the [online demo](#). If you prefer to use your own datasets, read [this chapter on data analysis configuration](#) to learn how to configure the system.

Several talks have been given about FLIM Playground:

- [FLIM Playground @ SPIE 2026: 13 mins](#)
- [FLIM Playground @ the Fluorescence Microscopy Summit by Photonics Media: 31 mins](#)

¹in general, any region of interest works, including the whole field of view



Due to the limitation in local file system access, extracting features from raw data is not available in the online demo. Read on to discover how FLIM Playground transforms raw fluorescence decay data into biological insight through an integrated, interactive workflow.

The latest version of this FLIM Playground manual is available [online](#).

1.1 Installation

FLIM Playground is built entirely in Python and is open-source.

For the complete, always-current instructions — per-platform download links, install steps, **upgrading**, and building from source — see the project README, kept as the single source of truth:

👉 [Installation & upgrade guide](#)

1.2 Introduction

Fluorescence lifetime imaging microscopy (FLIM) measures the time it takes for a fluorescent molecule to emit light (return to the ground state) after being excited by a pulse of light (enter the excited

state). It is sensitive to changes in fluorophore microenvironment including, pH, temperature, and conformational changes due to protein-binding and the presence of quenchers¹. Coupled with modern automated cell-segmentation methods², FLIM enables single-cell analyses that can reveal biological heterogeneity.

i Instrumentation

To acquire FLIM data, a light source—typically a pulsed laser for time-domain methods or a modulated continuous-wave source for frequency-domain methods—is used to excite the fluorophore of interest. The emission is detected using instrumentation capable of resolving fluorescence decay, such as time-correlated single-photon counting (TCSPC), time-gated, or phase/modulation-based detection. In time-domain FLIM, the delay between excitation and photon arrival is measured, and often a histogram is built, with the x-axis representing the delay time and the y-axis representing the number of photons falling into each time bin. Compared to intensity images, FLIM has an additional dimension of time (e.g., 256 time bins per 12.5 nanoseconds).

In frequency-domain FLIM, the phase shift and modulation depth of the emission relative to the excitation are determined.

1.3 Challenges

A diverse set of **tools** — both open-source and commercial, ranging from libraries to code-free graphical user interfaces (GUIs) — is available to extract and analyze FLIM data, providing alternative methods and therefore flexibility to FLIM researchers. Examples include **PhasorPy**³, an open-source library for analyzing fluorescence lifetime using the phasor approach; **FLUTE**⁴, an open-source GUI for interactive phasor analysis; **FLIMPA**⁵, an open-source phasor analysis GUI enabling batch processing, ROI-based quantification, and experiment-level comparison through manual assignment; **FLIMLib**, an open-source generic curve fitting library that can be used to fit fluorescence lifetime decay data; **SPCImage**⁶, a commercial software for fitting and phasor features.

However, while some tools offer code-free interfaces, users still need to write custom code—either to prepare data in the proper format as input, or to further process their outputs for downstream analysis. The fragmentation between tools arises because each focuses on only a subset of **data levels**: pixel, cell ROI, channel, field of view, and experiment.

i Data Levels

- **Pixel:**
 - A single decay curve encoded in vendor-specific file formats (e.g., Becker & Hickl, PicoQuant, etc.)
- **Region of Interest (ROI)**

- Mask with cell labels
- **Channel**
 - Different fluorophores
 - Fluorophore-specific calibration files
 - Masks focusing on different parts of the cell (e.g., whole cell, cytoplasm, nucleus, stain, etc.)
 - Different feature extraction methods (fitting, phasor, morphology, texture)
- **Field of View (FOV)**
 - Input decay types (e.g., 2D, 3/4D, pixel-level pre-fit lifetime features)
- **Experiment**
 - Different treatments, time points, cell lines, etc., and combinations thereof

An integrated framework should take into account all data levels ■■■■ (•) while maintaining the flexibility to handle various input types (°). **It should provide a level of abstraction to address fragmentation from data levels and input types.**

Additionally, the use of FLIM is rapidly evolving, and new methods are being developed all the time. An integrated framework should allow users to choose among alternative methods seamlessly, be the backbone of iterative explorations integral to research, and be ready to incorporate new methods: **The provided level of abstraction should address fragmentation from extraction and analysis methods.**

Finally, many of the existing tools, including GUI-based software, are not cross-platform, which limits their accessibility. The [Installation](#) addresses this last challenge.

A closer parallel to FLIM Playground’s integrated approach is the combination of [CellProfiler](#)⁷, which extracts per-object morphological and texture features, and [CellProfiler Analyst](#)⁸, which ingests these outputs or other feature tables for visualization and statistical analysis. However, FLIM Playground stands apart in that it can extract lifetime features from time-resolved data, alongside morphological and texture features from intensity images. Its general Data Analysis section incorporates FLIM-specific methods (e.g., [phasor analysis](#)) and provides statistical models that address additional analysis aspects such as data heterogeneity, in addition to training machine learning classifiers.

1.4 Method

1.4.1 Feature Classes

Tabular data columns can be categorized into three feature classes:

- **Identifiers**: unique row identifier and (optional) field of view identifier that allow us to investigate the biological heterogeneity at the single-cell level
- **Categorical features**: conceptually help us group the rows (e.g., treatment will group the rows into different treatment groups)
- **Numerical features**: quantify the differences/similarities between data groups

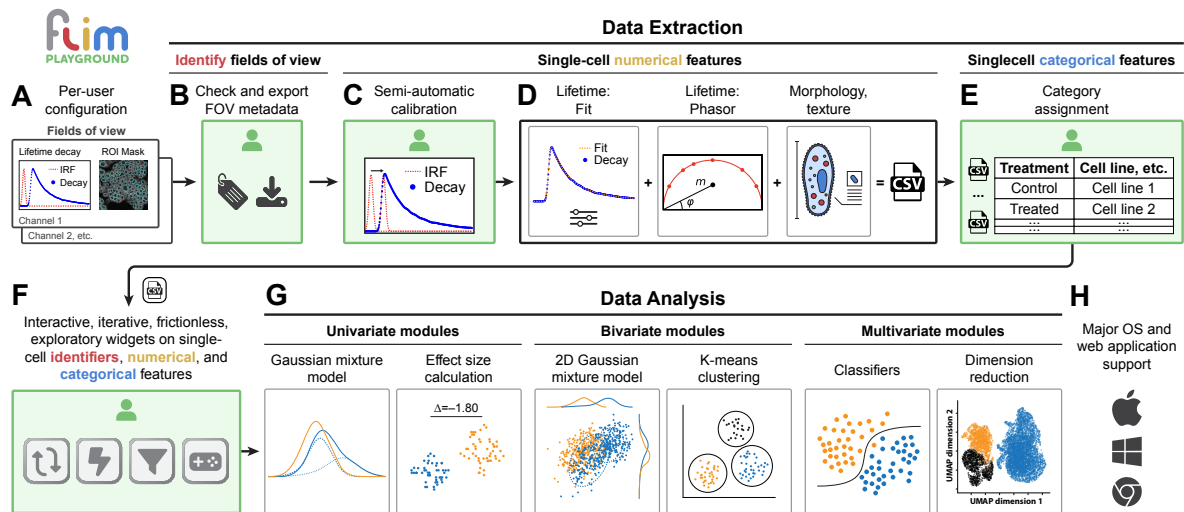
Science, from the data perspective, is about closing the conceptual categorical gaps with quantitative measurements.

At data levels, the data are processed to extract the feature classes in [Data Extraction](#), and the classes are used in the [Data Analysis](#) section.

	Data Levels	Data Processing	Feature Classes
Increasing complexity ↑	Experiment	Assign category labels (e.g., treatment, cell line, etc.) to each cell; perform data analysis on labeled cell groups.	Categorical features
	Field of View	Assign FOV identifier to each cell, obtain unique cell identifiers	Identifiers
	Acquisition Channel	Apply per-channel ROI mask, calibration and extraction methods to cell-level signals	Numerical features
	Cell ROI	Aggregate pixel-level decays to obtain cell-level signals	N/A
	Pixel	Read pixel-level decays encoded in different formats	N/A

1.4.2 Design

FLIM Playground has two independent sections:



1.4.2.1 Data Extraction

Data Extraction extracts single-cell features from the raw data. It adopts a **framework** that offers channel-level flexibility in input types and extraction methods without incurring too much overhead for users. Following the above **categorization**, it is divided into the following steps:

- **Data Extraction Configuration (A)**: allows users to choose among alternative input types and extraction methods (extractors).
- **Metadata organization (B)**: extracts the field of view **identifiers** and their configurations
- **Numerical Feature Extraction**: extracts single-cell **numerical features** based on user-selected extractors. More extractors can be integrated in the future.
 - **Calibration (C)**: calibrate for IRF shift or use fluorescence lifetime standard
 - Lifetime extractors (D):
 - * **Fitting**
 - * **Phasor**
 - Intensity-based extractors (D):
 - * **Morphology**
 - * **Texture**
 - **Derived features**
- **Categorical feature extraction (E)**: extracts single-cell **categorical features** and combines experiment-level datasets.

1.4.2.2 Data Analysis

Data Analysis analyzes features—whether extracted through Data Extraction or by other methods—using visualizations and statistical modeling. It deploys a **shared framework (F)** built to handle the **feature classes** across all analysis methods, enabling the same interactive and frictionless exploration experience and allowing new methods to be integrated in the future easily.

- **Data Analysis (G)** goes in-depth into how FLIM Playground handles the feature classes and **Data Analysis Config** goes through how users can configure FLIM Playground to analyze datasets that are not extracted by **Data Extraction**.

Here is the list of analysis methods incorporated into FLIM Playground, grouped by the number of numerical features they take as inputs:

- **Univariate analysis**
 - Feature Comparison
 - Feature Histogram
- **Bivariate analysis**
 - Feature Distribution
 - Phasor Analysis
- **Multivariate analysis**
 - Dimension Reduction
 - Classification

Both sections are built in Python and as self-contained apps ready to run on major operating systems and in browsers (**H**).

1.4.3 Summary

FLIM Playground resolves these **challenges** with an interactive code-free graphical user interface (GUI) that spans the full pipeline. It integrates validation checks that guide users at every step, and has a built-in repertoire of analytical methods with interactive widgets that encourage hypothesis-driven, iterative exploration of large datasets. It is built on a modular architecture that enables incorporation of new algorithms in the future.

1.4.4 Citation

FLIM Playground is published in [Cell Reports Methods](#). If it contributed to your research—whether through Data Extraction for single-cell feature generation or through Data Analysis for data exploration, visualization, selection of analysis methods, or hyperparameter tuning (UMAP, clustering, classification, etc.)—please cite this work in your publication. Your citation directly supports us in maintaining and improving it ✨🎈🍾.

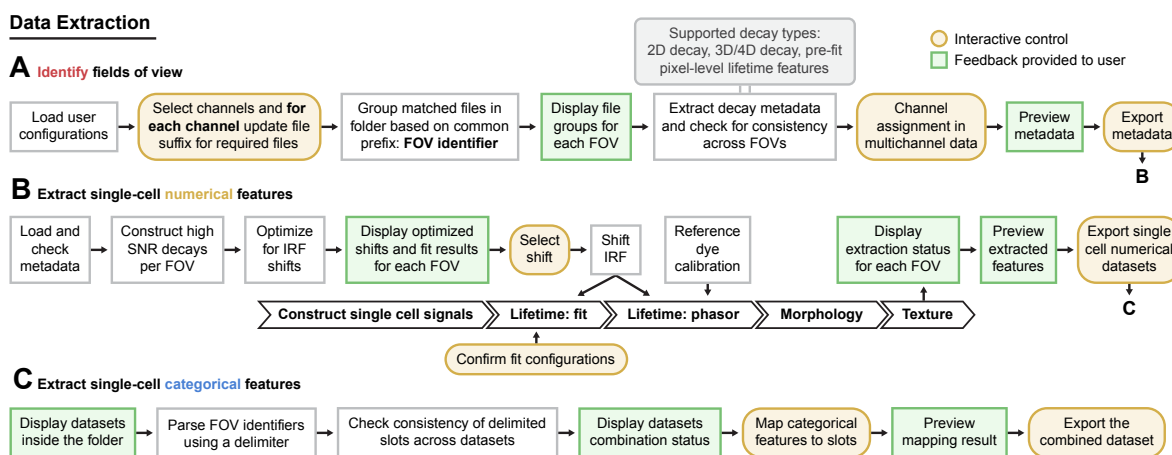
Part I

Data Extraction

2 Overview

FLIM Playground transforms raw FLIM data into single-cell numerical and categorical features, ensuring smooth integration with [downstream analysis](#). All steps are seamlessly connected through interactive widgets, with built-in error checking and reporting to ensure correctness.

A high-level overview of the data extraction process is shown below.



2.1 Stages

Users can [configure](#) the system once and apply it to future data. After configuration, users can extract features from raw data using the following steps, inspired by the [categorization](#):

Select a step to perform ?

- ☒ FOV Metadata Extraction
- ☐ Numeric Feature Extraction
- ☐ Categorical Feature Extraction

1. **FOV Metadata Extraction (A)**: each experiment session consists of multiple fields of view (FOVs). This step helps users organize the metadata of the FOVs.

2. **Numeric Feature Extraction (B)**: it calibrates and then extracts single-cell numerical features.
 - Calibration
 - Fit Calibration
 - Fit Free Calibration
 - * IRF Shift
 - * Fluorescence lifetime standard
 - Per-channel feature extraction includes:
 - **Lifetime fitting features**: fit an exponential decay curve to the measured decay and extract lifetime features per cell ROI.
 - **Phasor features**: calculate the phasor features such as the phasor coordinates. It also provides **phasor analysis**.
 - **Morphology features**: calculate morphology features based on the ROI mask.
 - **Texture features**: calculate texture features based on the intensity image and the ROI mask.
3. **Categorical Feature Extraction (C)**: extract the categorical features such as the treatment, time point, etc. based on the FOV name.
 - Users can then upload the extracted datasets to perform **Data Analysis**.

2.2 Input File Types

2.2.1 Decay

See [decay types](#) for more details.

- 2D decay:
 - .csv: a tabular data sheet with each row representing a cell and each column representing a time bin.
- 3D/4D decay: besides the spatial dimensions and the time dimension, the additional dimension in the 4D array represents acquisition channels.
 - .sdt: Becker & Hickl
 - .ptu: PicoQuant

2.2.2 Mask

Because of the single-cell focus of FLIM Playground, cell-level masks are needed for each channel. Channels can share the same mask, or use different masks (when they use different masks, the mask ID for each cell should match across masks).

The cell-level mask can focus on different parts of the cell region, such as the whole cell, the cytoplasm, the nucleus, or the stain part.

For an example, see [here](#).

- `.tiff/.tif`: a 2D array with background labeled as 0 and each cell ROI region labeled as a unique positive integer. That integer will be part of the [unique cell identifier](#).

2.2.3 IRF

IRF must be a 1D array. Currently, the supported formats are (extendable to more formats in the future):

- `.txt`: it uses `np.loadtxt` to parse a txt file. It returns a 1D array if the IRF numbers are in one row or in one column.

2.2.4 SPCImage t1

If the decay type is [3D/4D pixel pre-fit](#), users can provide pixel pre-fit lifetime feature files.

- `.asc`: a 2D array in spatial dimensions outputted by [SPCImage](#), with each (row, column) containing the value of the lifetime feature (e.g. `t1`: the first component's lifetime) of that pixel.
- Depending on the chosen number of lifetime components, the other pixel pre-fit feature files are included and their names are inferred from the `t1` file, by replacing `t1` in the file name with `t2`, `t3`, `a1 [%]`, etc.

2.2.5 Fluorescence Lifetime Standard

If the fluorescence lifetime standard-based calibration is chosen, users are expected to provide the fluorescence lifetime standard file.

- `.tiff/.tif`: a 3D array, with one dimension representing the time axis.

2.2.6 Intensity (2D)

If the channel's [imaging modality](#) is Intensity-only, users are expected to provide a 2D intensity image in:

- `.tiff/.tif`: a 2D array.

2.3 Limitations

- To simplify the workflow and due to the framework chosen, FLIM Playground does not support pixel-level fitting and fit-free analysis, and there are many other open-source tools that can do so (for a comprehensive list, see [here](#)). Instead, it does cell-level fitting and fit-free (phasor) feature extraction by summing up all the decays belonging to the same cell ROI to get one cell-level decay as a preprocessing step⁹. It also accepts pre-fit pixel-level lifetime features from [SPCImage](#) and [aggregates pixel-level lifetime features to cell-level lifetime features](#).

3 Configuration

Note

Configure FLIM Playground once and it is applied to all future data. You can also keep multiple named **configuration profiles** and switch between them — see [Extraction Profiles](#).

Constructing a framework that balances flexibility and usability is about choosing the right abstraction level. FLIM Playground chooses acquisition *channel* from data levels as the focus and adopts the **channel-centric framework**. Settings are divided into two categories: [cross-channel configurations](#) and [per-channel configurations](#).

Home Data Extraction Data Analysis



Welcome 🎉 to FLM Playground! 🎉 📄
Data Extraction pulls single-object features from raw microscopy data.
Data Analysis transforms tabular datasets into insights through visualization and statistical modeling.

Configuration

Profile: test + 🗑️

New profile: e.g. experiment-B + Create

Number of channels: 6

FLM Decay input type: Decay (3/40)

Unique cell identifier column name: cell_id

FOV column name: image_name

Laser rate (SR/s) for Decay (3/40): 0.08

Fit free calibration method: IRF Fluorescence Lifetime Standard

Channels 1-4

Channels 5-6

Imaging modality: FLIM

Channel 5 name: NAD(P)H

Extract feature types from NAD(P)H: Lifetime fit free Lifetime fit Intensity morph... Intensity feature

Number of components for NAD(P)H: 2

Fix lifetime components (ns) — set 0 to fit freely:

Fix t1 (ns): 0.000 Fix t2 (ns): 0.000

File suffixes: NAD(P)H

Decay: n.sdtf

IRF: NADH_IRF.dat

Mask: _mask.tif

Imaging modality: Intensity-only

Channel 6 name: red

Extract feature types from red: Intensity morph... Intensity feature

File suffixes: red

Intensity (2D): .tif

Mask: _mask.tif

Categorical columns (types to add more)

experiment patient_id day hour cell_type media dish cell_line treatment condition replicate

☒ Derived features

`norm_redox_ratio = Intensity_texture_MdH: Intensity_sum(Intensity_texture_MdH: Intensity_sum(Intensity_texture_FAD: Intensity_sum`

Template: Normalized A / (A + B)

A: Lifetime fit_NADH: t1

B: Lifetime fit_NADH: t1

Formula: $A/(A+B)$

+ Add derived feature

Update Configuration

3.1 Extraction Profiles

🌟 **NEW** 🌟: Users can now create, save, and switch between multiple data extraction profiles — each an independent, ready-to-go setup for a different experiment or dataset. Up to **10 profiles** are supported.

On the **Configuration** page:

- Select an existing profile from the **Profile** dropdown to load it.
- Create a new profile with **Create** (give it a name, e.g. `experiment-B`).
- Delete the active profile with **Delete** (disabled when only one profile remains).

Configuration

Profile ?

default ▼

Delete

New profile

e.g. experiment-B

+ Create

A newly created profile starts blank. Fill in the settings described below and click Update Configuration to save it. Everything described below pertains to the current profile.

3.2 Cross-Channel Configurations

3.2.1 Categorical Features

Categorical columns (type to add more)

cell_line × treatment × condition × patient_id × day × experiment

Add: experiment

Categorical features are useful to organize the data into groups of interest, and will be used extensively in the [Data Analysis](#) section. Users can specify the potential categorical feature names to be extracted in the [Categorical Feature Extraction](#) step.

3.2.2 Decay Types

FLIM Playground supports different decay types, including 2D decay, 3D/4D decay, and 3D/4D pixel pre-fit decay, through user specification.

3.2.2.1 2D Decay

In the system developed by Samimi et al.¹⁰, single cells flow through and a decay curve is acquired for each cell by aggregating all the photon arrival time delays deemed to be from the same cell. In exchange for spatial distribution, the acquisition speed is increased enormously. The output of the system is a tabular data sheet with each row representing a cell and each column representing a time bin. **FLIM Playground currently supports 2D decay in the CSV format.**

Note

Every column in a 2D decay CSV must be a numeric time bin. If any non-numeric column is present (for example, a cell-label or annotation column), FLIM Playground rejects the file with an error indicating that all columns are expected to be numeric time bins. Remove such columns before extraction.

3.2.2.2 3D/4D Decay

In TCSPC-based FLIM, the data is stored in a 3D/4D array and in special format (e.g. `.sdt`, `.ptu`). In addition to the spatial dimensions and the time dimension (XYT), the channel dimension may also be present, making it 4-dimensional (CXYT). **FLIM Playground currently supports both `.sdt` and `.ptu` files up to those 4 dimensions.** It reads the `duration` and the number of `time bins` from the decay file metadata. Some systems can also record a time stack, which is not supported yet.

3.2.2.3 3D/4D pixel pre-fit

Since FLIM Playground currently does **not** support pixel-fitting, users have the option to provide pixel pre-fit outputs from [SPCImage](#) in the format of `.asc`. See [here](#) for how FLIM Playground processes SPCImage outputs to obtain cell-level lifetime fitting features.

3.2.3 Identifiers

Unique cell identifier column name

cell_id

 FOV column name

image_name

3.2.3.1 Cell Identifier

Users can specify the column name for the unique identifier of the cell to be stored in the output dataset. The cell identifier is constructed as `{FOV Identifier}_{cell_label}`, where the `cell_label` is the unique integer label in the mask or the row number if the decay type is [2D decay](#).

3.2.3.2 FOV Identifier

Users can specify the column name for the field of view identifier. If the decay type is [2D decay](#), users may want to specify the column name as `exp_name`, because the FOV is not an image.

3.2.4 Decay Info

3.2.4.1 Laser frequency

Users can specify the laser frequency in GHz. For example, 0.08 GHz equals 80 MHz. It will be used in the [phasor analysis](#). If you do not intend to use the phasor analysis, you can fill in any number.

3.2.4.2 2D Decay-specific

Since the [2D decay](#) file does not provide the duration and the number of time bins per laser pulse interval, users need to specify them.



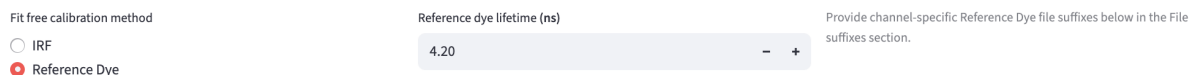
Decay (2D) duration (s)	Decay (2D) time bins
20.00	1024

3.2.5 Calibration Method

It supports two calibration methods for phasor analysis:

- IRF shift-based calibration
- [fluorescence lifetime standard-based calibration](#)

Since the instrumental response function (IRF) or the fluorescence lifetime standard is measured for each channel, they are set in the [per-channel configurations](#). The fluorescence lifetime standard's lifetime is channel-agnostic.



Fit free calibration method	Reference dye lifetime (ns)	Provide channel-specific Reference Dye file suffixes below in the File suffixes section.
☐ IRF	4.20	
☒ Reference Dye		

3.2.6 Number of Channels

Each FOV is assumed to have the same number of channels. The configuration will dynamically adjust the [per-channel configurations](#) based on the specified number of channels. Currently, the maximum number of channels is 8. When more than four channels are configured, the first four fold into a collapsed Channels 1-4 group while the newer channels (5 and up) stay expanded, so the ones you just added remain in view.

3.3 Per-Channel Configurations

3.3.1 Channel Name

Users can customize the channel name based on, for example, the name of the fluorophore.

Channel 1 name	Channel 2 name	Channel 3 name
nadh	fad	cd8

3.3.2 Imaging Modality

Users can specify the imaging modality for each channel. Currently, two modalities are supported:

- FLIM: The signal is time-resolved, i.e., has a time dimension, and is expected to be from one of the [decay types](#).
- Intensity-only: The signal is a spatial map of intensity values (2 spatial dimensions), i.e., has no time dimension. **Currently, the only supported signal type is a 2D intensity image in the format of .tiff/.tif.**

Imaging modality

Intensity-only|

FLIM

Intensity-only

Tip

This setup allows users to image some channels in FLIM mode, and others in non-FLIM mode. It also allows users to extract and analyze their data if they image in non-FLIM microscopic systems such as epifluorescence, confocal, brightfield, etc. by selecting *Intensity-only* for all channels.

Note

If the decay type is [2D decay](#), the [imaging modality](#) for all channels is restricted to FLIM.

3.3.3 Feature Extractor

Four types of feature extractors are available if the [imaging modality](#) is FLIM:

- [Lifetime fit](#)
- [Lifetime fit free](#)
- [Intensity Morphology](#)
- [Intensity Texture](#)

Extract feature types from nadh

Lifetime fit free ×

Intensity morph... ×

Lifetime fit ×

×

▼

Intensity texture

i Note

If the decay type is [2D decay](#), the [Intensity Morphology](#) feature extractor is **not** applicable, because there is no ROI mask to measure. The [Intensity Texture](#) extractor *is* available, but a 2D decay CSV has no intensity image, so it produces only the single `intensity_sum` feature (the sum of each cell's decay curve).

The [Intensity Morphology](#) and [Intensity Texture](#) feature extractors are available if the [imaging modality](#) is Intensity-only.

Imaging modality

Intensity-only

Channel 3 name

cd8

Extract feature types from cd8

Intensity morph... ×

Intensity texture ×

No results

Each channel can be assigned its own set of feature extractors. For example, if users do not intend to do lifetime extraction, they can de-select the `Lifetime fit` and `Lifetime fit free` options.

3.3.4 Number of Components

If users select the `Lifetime fit` under this channel, they can specify the number of components to fit.

3.3.5 Fixed Lifetimes

Users can optionally assign a fixed value (in ns) for specific lifetime components (when $n > 1$, e.g., t_1 , t_2). If a value is provided, it serves as the default fixed lifetime during the [interactive fitting phase](#). If left blank (or set to 0), the lifetime acts as a free parameter to be optimized. This is particularly useful when analyzing fluorophores with prior knowledge of their lifetimes.

Note

Fixed lifetimes apply only to channels that FLIM Playground fits. For [pre-fit](#) inputs (SPCImage .asc), whose lifetimes are read rather than fit, the fixed-lifetime controls are not shown.

3.3.6 File Suffix

The [FOV Metadata Extraction](#) step looks for associated input files for all channels of a given FOV using file suffixes. Based on the [decay type](#), [calibration method](#), and the selected [feature extractors](#), the system automatically generates the file suffixes for required input files.

File suffixes: nadh	File suffixes: fad	File suffixes: cd8
Decay n.sdt	Decay f.sdt	Decay s.sdt
IRF NADH_IRF.txt	IRF FAD_IRF.txt	Mask stain_mask.tiff
Mask n_photons_mask_cyto.tiff	Mask n_photons_mask_cyto.tiff	

In the example above, channels nadh and fad were asked to provide an IRF file, while channel cd8 did not, because `Lifetime fit` was selected for the former two channels and not for the latter. The file suffixes of the IRF files were different for each of the two channels, but they shared the same ROI mask file suffix because they used the same ROI mask. The cd8 channel used a different ROI mask than the other two channels, indicated by the mask file suffix.

For detailed information on what file formats are supported for each file type (Mask, IRF, etc.), see [input file types](#).

For detailed information on how the file suffixes are used, and what each file type (e.g. IRF, Mask, etc.) is, see the [FOV Metadata Extraction](#) page.

3.4 Derived Features

The Configuration page also hosts the **Derived features** builder, which composes new single-cell features by arithmetic over the features you already extract — optionally combining operands from different channels (for example, an optical redox ratio). Definitions are saved with the [current profile](#) and computed later, during [Numerical Feature Extraction](#). See [Derived Features](#) for the full walkthrough.

Categorical columns (type to add more)

experiment × patient_id × day × hour × cell_type × media × dish × cell_line × treatment × condition × replicate ×

☒ Derived features ⓘ

norm_redox_ratio = Intensity texture_NADH: intensity_sum / (Intensity texture_NADH: intensity_sum + Intensity texture_FAD: intensity_sum) 

Template ⓘ Name ⓘ

Custom... ▼ custom_formula

Operands (each becomes A, B, C... in the order picked)

Intensity texture... × Lifetime fit_NAD... × Intensity texture... ×

A = Intensity texture_NADH: intensity_sum, B = Lifetime fit_NADH: t1, C = Intensity texture_FAD: intensity_sum

+

-

*

/

(

)

A B C

Formula ⓘ

(A+B*2) / C - 20

Press Enter to apply

✓ Derived: custom_formula = Intensity texture_NADH: intensity_sum + Lifetime fit_NADH: t1

3.5 Save

Finally, users can save the configuration into the **current profile** for future use by clicking the **Update Configuration** button.

4 Field of View Metadata

It is the first step in the [Data Extraction](#) workflow. It organizes the metadata of each field of view (FOV)—including file paths for all required inputs and decay information—into a single metadata file. This file is then used for [numerical feature extraction](#), as well as for users' own bookkeeping and troubleshooting.

Data Extraction

Select a step to perform

- ☒ FOV Metadata Extraction
- ☐ Numeric Feature Extraction
- ☐ Categorical Feature Extraction

Decay input type: Decay (3/40)

☒ has nadh ☒ has fad

Feature extractors for nadh:

Feature extractors for fad:

```
[
  0 :
    "Intensity morphology"
  1 : "Intensity texture"
  2 : "Lifetime fit"
]
```

Laser rate (GHz)

0.80

File suffixes: nadh

Mask Decay IRF

n_photons_mask_cyto n.sdt NADH_IRF.txt

File suffixes: fad

Mask Decay IRF

n_photons_mask_cyto f.sdt FAD_IRF.txt

Channels ["nadh", "fad"] share the same mask suffix: n_photons_mask_cyto.tif

Copy the folder path here

/Users/allan/Downloads/40

Field of views:

H9_DAY_40_5
 All files found.

H9_DAY_40_1
 All files found.

H9_DAY_40_2
 All files found.

H9_DAY_40_6
 All files found.

H9_DAY_40_10
 All files found.

H9_DAY_40_9
 All files found.

H9_DAY_40_4
 All files found.

H9_DAY_40_8
 All files found.

H9_DAY_40_11
 Missing or duplicate files:
• Missing nadh_Decay: H9_DAY_40_11n.sdt
• Missing fad_Decay: H9_DAY_40_11f.sdt

H9_DAY_40_7
 All files found.

H9_DAY_40_3
 All files found.

Field of Views with are loaded successfully . FOVs with (if any) will not be recorded. Here is the preview of the FOVs and metadata recorded:

	image_name	nadh_Mask	nadh_Decay	nadh_IRF	fad_Mask
0	H9_DAY_40_5	/Users/allan/Downloads/40/edited/H9_DAY_40_5n_photons_mask_cyto.tif	/Users/allan/Downloads/40/H9_DAY_40_5n.sdt	/Users/allan/Downloads/40/NADH_IRF.txt	/Users/allan/Downloads/40/edited/H9_DAY_40_5f.sdt
1	H9_DAY_40_1	/Users/allan/Downloads/40/edited/H9_DAY_40_1n_photons_mask_cyto.tif	/Users/allan/Downloads/40/H9_DAY_40_1n.sdt	/Users/allan/Downloads/40/NADH_IRF.txt	/Users/allan/Downloads/40/edited/H9_DAY_40_1f.sdt
2	H9_DAY_40_2	/Users/allan/Downloads/40/edited/H9_DAY_40_2n_photons_mask_cyto.tif	/Users/allan/Downloads/40/H9_DAY_40_2n.sdt	/Users/allan/Downloads/40/NADH_IRF.txt	/Users/allan/Downloads/40/edited/H9_DAY_40_2f.sdt

Export FOV Metadata as CSV

It is divided into two sections:

- The left 1/3 of the screen is the metadata configuration panel, inherited from the [Data Extraction Configuration](#) step.
- The right 2/3 shows the extraction status for all FOVs.

4.1 Metadata

Metadata are configured earlier in the [Data Extraction Configuration](#) step and copied here. Some fields remain editable for flexibility, while others are fixed to ensure usability. Non-editable fields can be

modified in the Data Extraction Configuration step. After saving your modifications, return to this page and refresh it to see the changes.

4.1.1 Decay Type

See [decay types](#) for more details. The decay type is not editable here.

Decay input type: Decay (3/4D)

4.1.2 Channel Names

See [channel name config](#) for more details. The names are not editable here, though users can choose which channel to include.

☒ has nadh

☒ has fad

4.1.3 Feature Extractors

See [feature extractor config](#) for more details. The feature extractors for each channel are shown but not editable here.

Feature extractors for nadh

Feature extractors for fad

▼ [

0 :
"Intensity morphology"

1 : "Intensity texture"

2 : "Lifetime fit"

]

4.1.4 Decay Info

See [decay info config](#) for more details. The decay information is editable here.

Laser rate (GHz)

0.80

- +

4.1.4.1 2D Decay-specific

Since the [2D decay](#) file does not provide the duration and the number of time bins per laser pulse interval, users need to specify them.

Duration (s)	Time bins	Laser rate (GHz)
20.00 - +	1024 - +	0.50 - +

4.1.5 File Suffix

Copied from the [file suffix config](#). Users can edit the file suffixes here.

File suffixes: nadh

Mask	Decay	IRF
n_photons_mask_cyto	n.sdt	NADH_IRF.txt

File suffixes: fad

Mask	Decay	IRF
n_photons_mask_cyto	f.sdt	FAD_IRF.txt

Channels ['nadh', 'fad'] share the same mask suffix: n_photons_mask_cyto.tiff

4.1.6 Fluorescence Lifetime Standard

If the selected fit free calibration method is [fluorescence lifetime standard](#), users are asked to specify the fluorescence lifetime standard file path for each channel and the fluorescence lifetime standard's lifetime, which are copied from the data extraction configuration.

4.1.7 Folder Path

Finally, users can specify the folder path that contains all the required input files.

Copy the folder path here

/Users/allan/Downloads/40

Press Enter to apply

4.2 Metadata Extraction

4.2.1 FOV File Paths

The first step is to find the fields of view:

- It finds all the files recursively in the folder path that ends with the 1st file suffix of the 1st channel.
- The prefix of all the matched files is considered to be the field of view name.
 - FOV name = file name - 1st file suffix

Then, it uses the found FOV names to find all other files:

- The file name to search = FOV name + other file suffixes for each file suffix

The only exceptions are the calibration files, because they are not FOV-specific. They will be searched for based on their file suffixes only.

4.2.1.1 Success

H9_DAY_40_5

✓ All files found.

4.2.1.2 Missing

When it cannot find the file based on the file name (FOV name + file suffix for that file type) inside the folder path, it will complain.

H9_DAY_40_11

✗ Missing or duplicate files:

- Missing nadh_Decay: H9_DAY_40_11n.sdt

4.2.1.3 Duplicate

Because the calibration file is not FOV-specific, it will be searched based on the file suffixes only. If there are multiple files that match the same file suffix, FLIM Playground will be confused about which one to use.

H9_DAY_40_5

✗ Missing or duplicate files:

- Duplicate nadh_IRF with suffix: _IRF.txt

Fields of view with  are loaded successfully . FOVs with  (if any) will not be recorded.

Tip

If the issue persists after renaming the files to match the required prefix, hit the Rescan Folder

button to force a re-scan of the folder.

Rescan folder

4.2.2 Decay Info

If the decay type is **3D/4D decay**, FLIM Playground will try to infer the duration and the number of time bins per laser pulse interval from the decay file metadata. It will also check for inconsistencies across all decay files.

4.2.2.1 Channel Assignment

If the decay file associated with the channel found by the previous step is a 4D array, then FLIM Playground will try to infer the channel number for that channel. For each channel name, it will list all non-empty channels as potential channels to be assigned. If there is only one, the assignment is automatic. If there are multiple, users need to select the channel intended. It will also check for inconsistencies across all decay files about their dimensions and complain if there are any.

4.2.2.2 Fluorescence Lifetime Standard

If the file path to the [fluorescence lifetime standard](#) is specified, FLIM Playground will try to look for and read the file. It expects a `tiff` or `tif` file that contains a 3D array, with one of them as the time dimension. It will try to match the time dimension with the time dimension of the decay file. If it cannot find the matching time dimension, it will complain.

Error: Reference dye file 'test.tif' not found in folder. Please check the file name.

Error: Cannot find the time axis (256 time bins) in the reference dye file dimensions: (56, 512, 512)

Once the time axis is matched, the file path to the fluorescence lifetime standard, the fluorescence lifetime standard's lifetime, and the time axis will be recorded.

4.2.3 Result Preview and Export

If there is a non-zero number of FOVs with  and channel assignment and fluorescence lifetime standard checks (if applicable) are passed, a preview of the metadata data sheet will be shown for users to check.

	image_name	nadh_Mask	nadh_Decay	nadh_IRF	fad_Mask
0	H9_DAY_40_5	/Users/allan/Downloads/40/edited/H9_DAY_40_5n_photons_mask_cyto.tiff	/Users/allan/Downloads/40/H9_DAY_40_5n.sdt	/Users/allan/Downloads/40/NADH_IRF.txt	/Users/allan/Downloads/40/edited/H
1	H9_DAY_40_1	/Users/allan/Downloads/40/edited/H9_DAY_40_1n_photons_mask_cyto.tiff	/Users/allan/Downloads/40/H9_DAY_40_1n.sdt	/Users/allan/Downloads/40/NADH_IRF.txt	/Users/allan/Downloads/40/edited/H
2	H9_DAY_40_2	/Users/allan/Downloads/40/edited/H9_DAY_40_2n_photons_mask_cyto.tiff	/Users/allan/Downloads/40/H9_DAY_40_2n.sdt	/Users/allan/Downloads/40/NADH_IRF.txt	/Users/allan/Downloads/40/edited/H

Export FOV Metadata as CSV

As an example, the following metadata are extracted:

- **image_name**: The `image_name` column is the name of each FOV, which is specified in the [fov identifier config](#).
- **nadh_Mask**, **nadh_Decay**, **nadh_IRF**, **fad_Mask**, **fad_Decay**, **fad_IRF**: the [file paths](#) of the mask, decay, and IRF files for the nadh and fad channels.
- **nadh_input_type**, **nadh_imaging_modality**, **fad_input_type**, **fad_imaging_modality**: the [imaging modality](#) and the corresponding input type for each channel. These are two distinct values. For a channel whose modality is FLIM, the input type is one of the [decay types](#); for a channel whose modality is Intensity-only, the input type is Intensity (2D). Channels that share the same imaging modality share the same input type.

- nadh_Lifetime fit free, nadh_Intensity morphology, nadh_Lifetime fit, fad_Intensity morphology, fad_Intensity texture, fad_Lifetime fit: [feature extractors](#) for each channel.
- nadh_channel, duration, fad_channel, time_bins, laser_rate: the [decay info](#) for each channel and info shared by all channels.

The user can click the Export FOV Metadata as CSV button to export the metadata as a CSV file.

5 Numerical Feature Extraction

In this step, FLIM Playground extracts *single-cell* numerical features from the raw data in a folder. For each channel, the features to be extracted are specified in the [user-selected feature extractors](#). The available feature extractors are:

- [Lifetime fit](#): fit exponential models to decay curves to estimate lifetimes and their fractional contributions by minimizing an objective function.
- [Lifetime fit free](#): transform decay curves to phasor space using Fourier transform to obtain phasor coordinates.
- [Intensity Morphology](#): morphological features of single-cell ROI masks
- [Intensity Texture](#): texture features of single-cell ROI intensity images (or, for 2D decay input, just the per-cell decay-curve sum, `intensity_sum`)

5.1 Input

A CSV file extracted in the [fov metadata extraction](#) step that contains the metadata of the FOVs including:

- [required file paths](#)
- [decay type](#)
- [decay info](#)
- [selected feature extractors](#) for each channel
- [fit free calibration method](#) (if fit free calibration is required)

Users can either upload a previously extracted metadata file (left), or use the cached metadata file just extracted in the [fov metadata extraction](#) step (right).

Upload the field of view metadata csv ?



Drag and drop file here
Limit 200MB per file • CSV

Browse files

Using the latest extracted metadata file:
/Users/allan/Downloads/40/fov_metadata_20250824_140117.csv. Refresh the page to use a different file.

For `Lifetime fit` and `Lifetime fit free` feature extractors, the first step is always [calibration](#).

5.2 Calibration

Calibration in FLIM is essential because raw decays are convolved with the instrument response function (IRF)—the timing profile of the detection system’s response to an ultrashort light pulse that broadens and shifts the measured decay—so without correcting for these, fitted lifetimes and phasor positions are biased and not comparable across days, samples, or instruments.

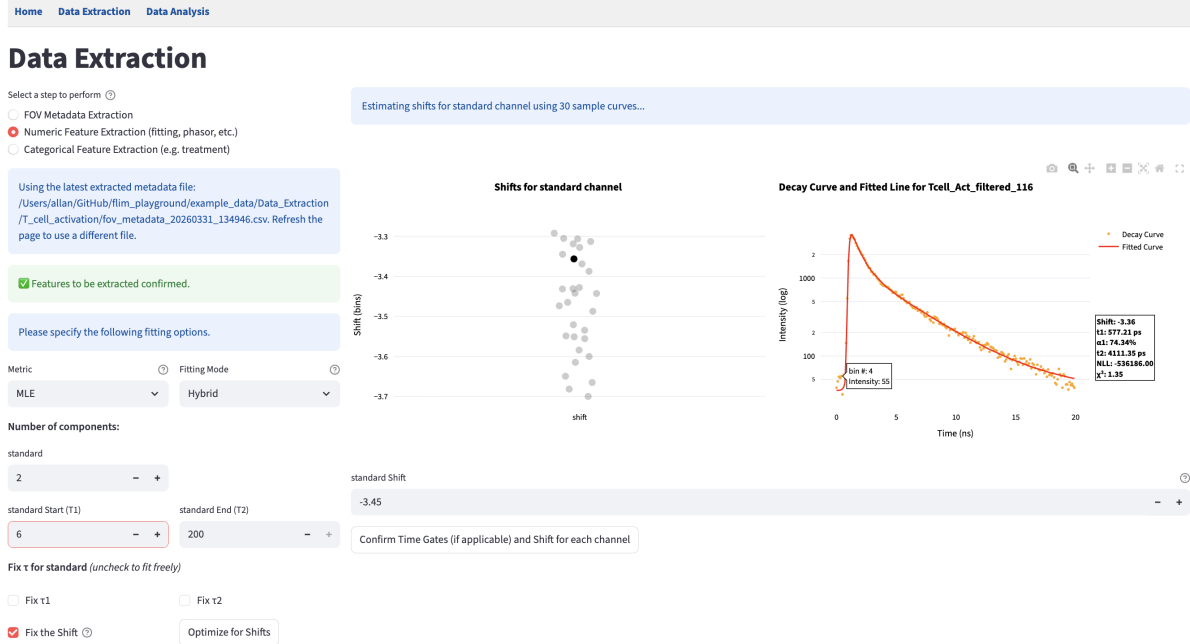
5.2.1 Fit Calibration

During different experiments, the IRF may shift differently with respect to the measured decay. Before applying reconvolution fitting, the IRF shift needs to be estimated for each channel if the `Lifetime fit` feature extractor is selected for the channel.

It is performed in two steps:

1. Gather the high signal-to-noise ratio (SNR) decay curves
2. For each curve, perform reconvolution fitting and set the shift value as the free parameter to be optimized/fitted. The shift search range is centered on the initial shift guess and extends $\pm 2 \times \text{FWHM}$ of the IRF in both directions, where FWHM is the full width at half maximum of the IRF measured in time bins. This focused search window prevents the optimizer from getting stuck in suboptimal basins when [time gates](#) are restrictive.

The distribution of the shift values is displayed as an interactive scatter plot. When clicking on a point, the corresponding decay curve with the fitted curve and the key statistics are displayed for diagnostics. Based on the distribution or using certain prior knowledge, users can specify the shift value applied to all fields of view (if `Fix the Shift` is selected, see fitting options), or use the fov-specific shift value found by the fitting (if `Fix the Shift` is deselected).



5.2.1.1 Gather High SNR Decay Curves

The high SNR decay curves are constructed automatically based on the [decay type](#). If it is in [2D format](#), a total of the brightest 30 curves that are below 100000 photons are selected and evenly distributed across all fields of view (one CSV file is considered as one FOV). If it is in [3D/4D format](#), the fields of view are images, and one curve is constructed for each image that includes all the non-zero pixels within the [ROI mask](#).

5.2.1.2 Reconvolution Fitting

Fitting is essentially an optimization problem: it minimizes the difference between the fitted curve and the measured curve. The fitted curve is modeled by an n^{th} component exponential function convolved with the shifted IRF, and the difference is modeled as an objective metric. Therefore, users are provided with controls over two parts of the fitting process through the fitting options panel:

1. How to construct the objective metric
2. How to perform the optimization

Implementation-wise, FLIM Playground uses the `lmfit` package that takes generic objectives to perform the optimization process that is flexible enough to handle the reconvolution fitting.

Each decay curve is fitted independently. **Fit calibration** (one optimization per high-SNR curve in the IRF shift step) and **Lifetime fit** ROI-level extraction (one optimization per ROI per channel) both

schedule those independent fits **in parallel across CPU cores**, so on a multi-core machine many fits run at once.

If at least one channel has `Lifetime fit` feature extractor selected, the fitting options panel is displayed.

Metric ? Fitting Mode ? ☒ Fix the Shift ?

MLE ▼ Hybrid ▼

Number of components:

nadh

2 - +

nadh Start (T1) - + nadh End (T2) - +

41 256

Start Finding Shifts

Let's break down the fitting options one by one.

5.2.1.2.1 Number of Components

The number n in the n^{th} component exponential function:

$$I(\mathbf{t}) = \sum_{i=1}^n A_i e^{-\mathbf{t}/\tau_i}, \quad \tau_i > 0$$

Therefore, n determines the parameters to be fitted: the amplitudes A_i and the lifetimes τ_i . $\mathbf{t}$ is the time axis of the decay curve calculated by the duration and `time` bins from the [decay info](#):

$$\mathbf{t} = [0, \Delta t, 2\Delta t, \dots, (N-1)\Delta t], \quad \text{where } \Delta t = \frac{T}{N}.$$

It supports $n = 1, 2, 3$ components.

Additionally, the constant offset parameter (Z) that represents a time-independent background (i.e., room light) is also optimized¹¹.

$$\hat{y}(t) = [I(t) + Z] \circledast \text{IRF}(t)$$

where $\circledast$ denotes circular (periodic) convolution, implemented via FFT multiplication (`np.fft.ifft(np.fft.fft(decay) * np.fft.fft(irf))`). Circular convolution correctly accounts for the wrap-around effects inherent in periodic TCSPC decays, where photons arriving after the end of the measurement window appear at the beginning of the next period.

i Previous Versions (< 1.8.0)

In versions prior to 1.8.0, the fitting process utilized linear convolution with a cut off:

$$\hat{y}(t) = (I(t) * \text{IRF}(t))_{0:N} + Z$$

where $*$ denotes linear convolution and the subscript $(0 : N)$ denotes truncating the result to the length of the original decay curve, effectively ignoring the part of the convolution that extends beyond the measurement window. The difference between this approach and circular convolution is minimal for shorter lifetimes. However, the correct way to fit incomplete decays is formulated above using circular convolution, which is now the standard used in the latest version.

5.2.1.2.2 Fixed Lifetimes

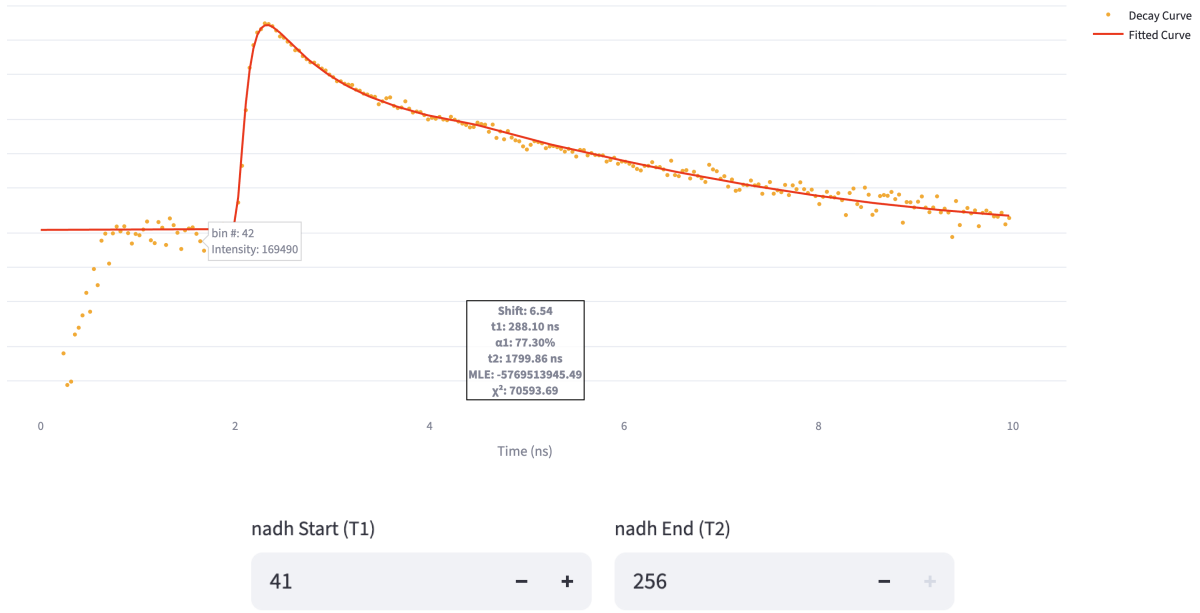
While finding the optimal amplitudes A_i and lifetimes τ_i (when [number of components](#) is greater than 1), users can optionally assign fixed values to specific τ_i components during the interactive fitting phase. These explicit constraints remain constant (are not treated as free parameters) while minimizing the cost metric, making the optimization more robust when the true lifetime state is known. The default fixed lifetime values can be initialized in the [Data Extraction Configuration](#).

i Note

Fixed lifetimes apply only to channels that FLIM Playground fits. For [pre-fit](#) inputs (SPCImage .asc), whose lifetimes are read rather than fit, the fixed-lifetime controls are not shown.

5.2.1.2.3 Time Gates

Due to the dead time of the system certain time bins from the head and/or tail of the decay curve are not reliable. The T1 (head) and T2 (tail) gates are used to select the time range of the decay curve to be fitted. Users can inspect the decay curve by clicking the full-screen mode button of the plot on the right of the shift result. Hover-based interaction is implemented so users can see the time bin numbers to have a better sense of the time range.



Only the time bins within the T1 and T2 gates are used to calculate the cost metric.

5.2.1.2.4 Metric

Metric ?

MLE ▼

MLE

WLS

Maximum Likelihood Estimation (MLE) estimates the parameters by maximizing the likelihood function, which is the probability of the measured data given the model parameters. A mathematically convenient way to do this is to minimize the negative log-likelihood function:

$$\text{NLL}(\theta; t_s, t_e) = - \sum_{n=0}^{N-1} \mathbf{1}_{[t_s, t_e]}(t_n) [y_n \log \hat{y}(t_n) - \hat{y}(t_n)].$$

$\mathbf{1}_{[t_s, t_e]}$ is the indicator function that is 1 if t_n is within the [time gates](#) $[t_s, t_e]$, and 0 otherwise. y_n is the observed count at time t_n , and $\hat{y}(t_n)$ is the model prediction at t_n .

Weighted Least Squares (WLS) minimizes the squared residuals between the measured curve and the model prediction, with each time bin weighted by the inverse of its expected variance. Because photon

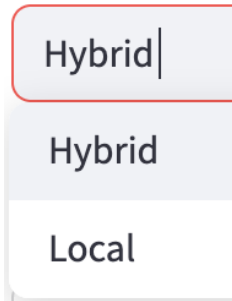
counts follow Poisson statistics (variance $\approx$ mean), the per-bin variance is estimated by the model prediction $\hat{y}(t_n)$, so each residual is scaled by $1/\sqrt{\max(\hat{y}(t_n), 1)}$, and the cost minimized is

$$\text{WLS}(\theta; t_s, t_e) = \sum_{n=0}^{N-1} \mathbf{1}_{[t_s, t_e]}(t_n) \frac{(y_n - \hat{y}(t_n))^2}{\max(\hat{y}(t_n), 1)}.$$

This weighting is Pearson's χ^2 — the per-bin variance is taken from the model $\hat{y}(t_n)$ rather than the noisy measured count — which keeps the high-count bins near the decay peak from dominating the fit; the $\max(\cdot, 1)$ floor avoids division by zero where the model is near zero. The minimized cost therefore uses the same definition as the reported [reduced chi-square](#) (χ_r^2) goodness-of-fit metric.

5.2.1.2.5 Fitting Mode

Fitting Mode



The fitting mode selects the optimization strategy. It is a trade-off between speed and the effort to avoid local minima.

The calibration step and the subsequent cell-level fit share this one setting. Whenever at least one channel requires shift estimation, the dropdown offers **Hybrid** only, so that shift estimation runs at full accuracy.

- **Local**: Each cell is fit with a single local optimizer — `leastsq` (least squares with Levenberg-Marquardt) if the chosen metric is WLS, otherwise `nelder` (Nelder-Mead). To give those per-cell fits a good starting point, a one-time **warm start** is performed first: the differential evolution global optimizer is run once on the *mean* decay of all ROIs in the batch, and the resulting lifetimes (and the constant offset) seed the initial values for every per-cell fit. The mean is used rather than the sum so the seed stays at single-cell scale, keeping a cell's result independent of how many cells share its batch. This warm start runs once, up front, before the per-cell fits are [parallelized across CPU cores](#); because the global search runs once for the whole batch instead of once per cell, **Local** is substantially faster than **Hybrid**.

- Hybrid: The slower but more accurate option. For **each cell**, it runs the differential evolution algorithm to obtain a good initial guess for all parameters, followed by the Local optimizer to refine the solution.

5.2.2 Fit Free Calibration

Users get to choose between the following two methods to calibrate the IRF shift in the [configuration step](#).

The first step of the calibration is to **subtract an offset value and clip each decay curve to 0 if the time bin is negative**. The offset C is estimated as the mean signal over the final 10% of time bins in each decay ('tail').

Then, depending on the chosen [calibration method](#), the following two methods are available:

5.2.2.1 Shift IRF

It is performed similarly to the [fit calibration](#) steps, but in the second step, the shift is not optimized (fitted). It shares the same interface as the [fit calibration](#) step, where a scatter plot of the shift values is displayed for each channel, only that the plot is not interactive to show the fit. Instead, it outputs the shift that maximizes the cross-correlation between the IRF and each decay. If `Fix the Shift` is selected, users can specify the shift value that will be applied to all fields of view. Otherwise, the shift value is chosen to be the one that maximizes the cross-correlation between the IRF and the decay.

Note

If both the `Lifetime fit` and `Lifetime fit free` feature extractors are selected for this channel, the optimized IRF shift derived from [the fit calibration](#) step is reused for the `Lifetime fit free` feature extractor.

The signals of the shifted IRF are deconvolved from the offset-subtracted decay curves using [phasor.phasor_divide](#) from the phasorpy package.

5.2.2.2 Fluorescence lifetime standard

Because the fluorescence lifetime standard is measured in the same system as the decay curves and we know its lifetime, there is no need to account for the IRF shift and the offset. Therefore, after subtracting the offset, phasorpy's [lifetime.phasor_calibrate](#) function is used to calibrate all the decay curves behind the scenes when calculating the phasor coordinates.

5.2.3 Confirm Calibration

Once users are satisfied with the calibration settings (fitting options and shift values), they can click the **Confirm Time Gates (if applicable) and Shift for each channel** button at the bottom of the page.

Confirm Time Gates (if applicable) and Shift for each channel

Once confirmed, the left panel prompts users to either apply the calibration settings to the downstream extractors or calibrate again. Users can also save the updated settings (fitting options and shift values) to the metadata file.

Confirm and Start Analysis

Go back and find shift

Download updated metadata

5.3 Feature Extraction

If the selected feature extractors do not require IRF shift calibration, or users have finished the calibration, they can proceed to the feature extraction step by clicking the **Confirm and Start** button.

Similar to the [fov metadata extraction](#) step, FLIM Playground extracts the features and displays the extraction status for each field of view.

Select a step to perform

☐ FOV Metadata Extraction
 ☒ Numeric Feature Extraction (fitting, phasor, etc.)
 ☐ Categorical Feature Extraction (e.g. treatment)

Upload the field of view metadata csv

Drag and drop file here

Limit 200MB per file • CSV

Browse files

fov_metadata_20250824_162655.csv

10.5KB

X

Features to be extracted confirmed.

Confirm and Start Analysis

Go back and find shift

Download updated metadata

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00009_Ch2_summed

The image_name TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00009_Ch2_summed has 1 cell(s) with '-' or NaN values out of 12 cell(s).

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00001_Ch2_summed

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00005_Ch2_summed

The image_name TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00005_Ch2_summed has 1 cell(s) with '-' or NaN values out of 22 cell(s).

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00024_Ch2_summed

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00013_Ch2_summed

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00017_Ch2_summed

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00020_Ch2_summed

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00016_Ch2_summed

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00021_Ch2_summed

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00025_Ch2_summed

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00012_Ch2_summed

Success!

image_name: TSeries-02042025-1mW_54Pocks_1frames_5x5-003_Cycle00008_Ch2_summed

Running fov_extraction(...) g/fit free) for nadh: for 23 cells...

A progress bar is rendered to show the feature extraction progress for each FOV (bottom right).

Warnings are displayed if cells have NaN values for some features. They are *not* excluded from the final CSV file.

5.4 Save Results

Once the feature extraction is finished, the results are saved to a CSV file. How this happens depends on how the metadata reached this step. If the cached metadata file from the [fov metadata extraction](#) step was used, the CSV is written automatically into the same folder as that metadata file, and no button is shown. If the metadata file was uploaded manually, FLIM Playground cannot know where to write it, so a [Download single cell features as CSV](#) button appears at the bottom of the page instead.

Field of view features with ✔ are extracted successfully ❗ ! FOVs with ✖ (if any) are excluded. The first few rows of the features are shown below.

cell_id	nadh_centroid_x	nadh_centroid_y	Intensity morphology_nadh: area	Intensity morphology_nadh: perimeter	Intensity morphology_nadh: solidity	Intensity morphology_nadh: eccentricity
H9_DAY_40_5_3	214.1314	10.1134	388	116.5685	0.7854	0.7
H9_DAY_40_5_4	242.4038	16.2271	634	118.4914	0.8476	0.6
H9_DAY_40_5_6	155.2827	24.2655	2490	320.8356	0.8156	0.8
H9_DAY_40_5_7	49.0648	25.8541	1172	178.9949	0.8919	0.7
H9_DAY_40_5_8	77.3881	43.406	894	169.3381	0.8976	0

Download single cell features as CSV

45

6 Lifetime Features (Fit)

Biological samples often contain mixtures of fluorescent species, each with its own characteristic lifetime. By modeling the decay as a sum of exponentials, the analysis can separate and quantify these different contributions. For example, NADH in cells exists in both free (short lifetime, ~0.4 ns) and protein-bound (longer lifetime, ~2–3 ns) states; by fitting the fluorescence decay with a bi-exponential model, one can estimate the fraction of each state, which provides direct insight into cellular metabolism and energy production pathways.

FLIM Playground extracts cell-level lifetime fitting features in this feature extractor, including the fraction of each lifetime component (α_i), their lifetimes (τ_i), mean lifetime (τ_{mean}), intensity-weighted mean lifetime ($\tau_{m,iw}$), and the reduced chi-square (χ_r^2) goodness-of-fit metric. (See [here](#) for the definition of the equation below)

$$I(t) = \sum_{i=1}^n A_i e^{-t/\tau_i}$$

6.1 Fitting

If the decays have not been pre-fit, FLIM Playground applies the confirmed fitting options ([number of components](#), [fixed lifetimes](#), [time gates](#), [metric](#), and [fitting mode](#)) to the same [reconvolution fitting process](#) as in the [IRF shift calibration step](#). The IRF shift is no longer a free variable to be optimized. The calibration step and the cell-level fit share a single **Fitting Mode** setting, and whenever any channel requires shift estimation — which is always the case for `Lifetime fit` on decays that have not been pre-fit — that setting is restricted to **Hybrid**. The `Local` mode therefore does not apply to this workflow; see [fitting mode](#) for when it becomes available. As with calibration, these per-cell fits are [parallelized across CPU cores](#). To recap, the reconvolution fitting minimizes the difference between the measured curve and the fitted curve modeled by a multi-exponential model convolved with the IRF, quantified by the cost metric.

Otherwise, FLIM Playground uses the [pre-fit values](#) to calculate the fitting features.

In the final dataset, the primary fitting features are prefixed by the combination of the feature extractor name (i.e. `Lifetime fit`) and the channel name, allowing [Data Analysis](#) to group the features. For example, `Lifetime fit_nadh: a1` means the fraction of the first lifetime component for the NAD(P)H channel.

Not every column produced by this extractor carries that prefix. The lifetimes (τ_i , τ_{mean} , $\tau_{m,iw}$) and the leading fractions do, but the raw amplitudes (A_i), the constant offset, the reduced chi-square, and the final fraction (which is derived as the remainder of the others) are written with only the channel name, e.g. `nadh_reduced_chi_square`. These are treated as bookkeeping values, so Data Analysis places them in the **Uncategorized Features** group rather than under `Lifetime fit_nadh`.

! Units in the exported dataset

The extracted CSV mixes three units, so check the column before using a value in a formula:

- **Lifetimes** (`t1`, `t2`, `t3`, `tm`, `tm_iw`) are exported in **picoseconds**, even though this chapter writes the formulas in nanoseconds.
- **Fractions** (`a1`, `a2`, `a3`) are exported as **percentages** in the range 0–100, not as the 0–1 fraction α_i used in the formulas below. Divide by 100 before substituting them into τ_{mean} or $\tau_{m,iw}$.
- Phasor-derived lifetimes from the [Lifetime fit free](#) extractor (`Tau_phase`, `Tau_mod`) are in **nanoseconds**.

6.1.1 Preprocessing

It sums up all the pixel decays belonging to the same cell ROI labeled by the ROI [mask](#) as one decay curve⁹. The ROI summing reduces variability and bias and allows for short integration times at acquisition in exchange for sub-cellular resolution (i.e., pixel-level). Users do not need to specify the bin factor (each pixel sums up the surrounding pixels' decay curves) to account for insufficient photon counts. ROI-level aggregation also keeps the workload manageable; independent ROI fits are then [run in parallel across CPU cores](#).

It also shifts the IRF using the shift values from the [IRF shift calibration](#) step. To shift an IRF, FLIM Playground upsamples the IRF 10 times using linear interpolation to fill the gaps. Then it shifts the IRF by the shift values $\times 10$ and downsamples the IRF back to the original size.

6.1.2 Fraction of components

In addition to the absolute amplitudes of each component directly from the fitting result, FLIM Playground calculates the fraction of each component, α_i , as the amplitude of the component divided by the sum of all amplitudes. This normalization allows lifetime to be independent of the absolute intensity of the signal.

α_i is a value between 0 and 1, and it is used in that form throughout the formulas in this chapter. In the exported dataset, however, the corresponding `a1/a2/a3` columns are stored as **percentages** ($\alpha_i \times 100$, ranging from 0 to 100). The final fraction is not fitted independently; it is derived as the remainder that makes the fractions sum to 100.

6.1.3 Mean lifetime

The mean lifetime, τ_{mean} , is calculated as the weighted average of the lifetimes of all components, where the weights are the fractions of each component.

$$\tau_{mean} = \sum_{i=1}^n \alpha_i \tau_i$$

6.1.4 Intensity-weighted mean lifetime

The [mean lifetime](#) above is amplitude-weighted: each component is weighted by its fraction α_i . The intensity-weighted mean lifetime, $\tau_{m,iw}$, instead weights each component by its relative intensity contribution ($\alpha_i \tau_i$), so longer-lived components — which emit more photons per molecule — count more.

$$\tau_{m,iw} = \frac{\sum_{i=1}^n \alpha_i \tau_i^2}{\sum_{i=1}^n \alpha_i \tau_i} = \frac{\sum_{i=1}^n \alpha_i \tau_i^2}{\tau_{mean}}$$

For a single component, $\tau_{m,iw} = \tau_1$.

τ_{mean} and $\tau_{m,iw}$ are reported whenever there is more than one component to average over. For the [pixel pre-fit](#) workflow they are reported for every component count, with both set to τ_1 in the single-component case. For fitting, they are reported only for two- and three-component fits: a single-component fit yields τ_1 itself, so no separate mean-lifetime columns are written.

6.1.5 Goodness of fit

For each fitted decay, FLIM Playground also reports the reduced chi-square (χ_r^2) as a goodness-of-fit diagnostic:

$$\chi_r^2 = \frac{1}{N - p} \sum_n \frac{(y_n - \hat{y}_n)^2}{\hat{y}_n}$$

where the sum runs over the time bins within the [time gates](#) that have a positive model value ($\hat{y}_n > 0$), y_n is the measured count, $\hat{y}_n$ is the fitted value, N is the number of those bins, and p is the number of free parameters (amplitudes, lifetimes, and the offset, minus any [fixed lifetimes](#)). Values near 1 indicate a good fit.

6.2 Pixel pre-fit

Currently, FLIM Playground supports pixel pre-fit lifetime features from [SPCImage](#). The pixel-level lifetime fitting features are expected to be stored in [2D arrays](#) in spatial dimensions, with each row and column having the value of a lifetime feature outputted from SPCImage. SPCImage assigns 0 for pixels that are not fitted (e.g. thresholded out). To avoid them biasing the results, FLIM Playground uses `np.ma.masked_array` to create a masked array and disregards them when calculating the averages using `np.ma.average`.

Then it uses the ROI [mask](#) to calculate the cell-level lifetime features by averaging the pixel-level features within each ROI.

7 Lifetime Features (Fit-Free)

Phasor analysis provides a fast, model-free view of lifetimes by transforming fluorescence decays into phasor coordinates through a Fourier transform. This approach is considered “fit-free” because it does not require iterative curve fitting or predefined decay models—each decay is mapped directly to a point on the [phasor plot](#). It has the nice property that mono-exponential decays fall on the universal semicircle while mixtures lie inside as linear combinations, so clusters reveal distinct species and their fractional contributions. FLIM Playground currently extracts cell-level phasor features: `g`, `s`, `Tau_phase`, `Tau_mod`, `g_2nd` (2nd harmonic), and `s_2nd`. (Datasets extracted before this feature was renamed use `Tau_m`, which Data Analysis still recognizes.)

Fit-free features in the final dataset are prefixed by the combination of the feature extractor name (i.e. `Lifetime fit free`) and the channel name, allowing [Data Analysis](#) to group the features. For example, `Lifetime fit free_nadh: G` means the g coordinate for this cell in the NAD(P)H channel.

7.1 Preprocessing

It sums up all the pixel decays belonging to the same cell ROI labeled by the ROI [mask](#) as one decay curve⁹. The phasor features are calculated from the summed decay curve.

Two preprocessing steps are then applied, and they are independent of each other:

- **Offset subtraction** is applied to every decay curve, regardless of the chosen [calibration method](#). For each cell, the offset is estimated as the mean of the last 10% of the time bins of that cell’s own decay curve, subtracted from the curve, and any resulting negative values are clipped to zero. Note that this per-cell baseline is unrelated to the `offset` parameter fitted during [lifetime fitting](#).
- **IRF shifting** is applied only when the chosen calibration method is IRF shift calibration. The IRF is shifted using the shift values as described in the [Shift IRF step](#). Under fluorescence lifetime standard calibration, no IRF shift is applied.

7.2 Phasor features

7.2.1 Step 1: raw phasor coordinates

The reference implementation from [phasorpy](#) assumes the decay curve comes from one period and time bins are evenly spaced (untruncated). So, their calculation is frequency (f) and time (T) independent. However, there are cases when the decay curve is truncated due to the deadtime of the system. For example, when the laser frequency is $f = 0.08$ GHz, the theoretical period is $T = 12.5$ ns, but in practice the time window is truncated to $T = 10$ ns. FLIM Playground applies a custom implementation to the truncated decay curve. (The `sample_phase` option can be used to account for the truncation but it does not handle the second or higher harmonics)

7.2.1.1 Untruncated decay curve

It uses the `phasor_from_signal` function from the [phasorpy](#) package to calculate the uncalibrated phasor coordinates `g_raw` and `s_raw`, `g_raw_2nd` and `s_raw_2nd`, the first and second harmonics' real and imaginary parts, from the cell decay curve.

Either `g_irf`, `s_irf`, `g_irf_2nd`, and `s_irf_2nd` from the IRF, or `ref_mean`, `ref_real`, `ref_imag` from the fluorescence lifetime standard, depending on the chosen [calibration method](#), are calculated using the same function.

Mathematically, the raw phasor coordinates are calculated as (copied from the [phasorpy](#) documentation):

$$F_{DC} = \frac{1}{K} \sum_{k=0}^{K-1} F_k, \quad g_{\text{raw}} = \frac{1}{F_{DC}} \frac{1}{K} \sum_{k=0}^{K-1} F_k \cos\left(2\pi h \frac{k}{K}\right), \quad s_{\text{raw}} = \frac{1}{F_{DC}} \frac{1}{K} \sum_{k=0}^{K-1} F_k \sin\left(2\pi h \frac{k}{K}\right)$$

where K is the number of time bins, F_k is the decay curve, and h is the harmonic number.

7.2.1.2 Truncated decay curve

The phasor coordinates of the IRF and the decay curves of both harmonics are calculated using the time axis $t_k = k(T/K)$ ($k = 0, \dots, K-1$) and the angular frequency $\omega = 2\pi hf$:

$$g_{\text{raw}} = \frac{\sum_{k=0}^{K-1} F_k \cos(\omega t_k)}{\sum_{k=0}^{K-1} F_k}, \quad s_{\text{raw}} = \frac{\sum_{k=0}^{K-1} F_k \sin(\omega t_k)}{\sum_{k=0}^{K-1} F_k}$$

This formula is equivalent to the untruncated formula when $fT = 1$.

For the fluorescence lifetime standard, the phasor coordinates are calculated using phasorpy's `phasor_from_signal` and the `sample_phase` option is set to ωt_k .

7.2.2 Step 2: calibrated phasor coordinates

To get the calibrated phasor coordinates `g`, `s`, `g_2nd`, and `s_2nd`, FLIM Playground uses either

```
from phasorpy import phasor
g, s = phasor.phasor_divide(g_raw, s_raw, g_irf, s_irf)
g_2nd, s_2nd = phasor.phasor_divide(g_raw_2nd, s_raw_2nd, g_irf_2nd, s_irf_2nd)
```

or

```
from phasorpy import lifetime
g, s = lifetime.phasor_calibrate(g_raw, s_raw, ref_mean, ref_real, ref_imag, frequency=laser_rate)
g_2nd, s_2nd = lifetime.phasor_calibrate(g_raw_2nd, s_raw_2nd, ref_mean, ref_real, ref_imag, frequency=laser_rate)
```

The `laser_rate` is specified during the [fov metadata extraction](#) and the [configuration interface](#).

7.2.3 Step 3: phasor-derived lifetime

`Tau_phase` and `Tau_mod` are calculated using the first harmonic's phasor coordinates `g` and `s`.

```
import numpy as np
w = 2 * np.pi * laser_rate
phi = np.arctan2(s, g)
mod = np.sqrt(g**2 + s**2)
tau_phase = 1/w * np.tan(phi)
tau_mod = 1/w * np.sqrt(1/mod**2 - 1)
```

$$\phi = \text{atan2}(s, g), \text{mod} = \sqrt{g^2 + s^2}, \tau_\phi = \frac{\tan \phi}{\omega}, \tau_{\text{mod}} = \frac{1}{\omega} \sqrt{\frac{1}{\text{mod}^2} - 1}$$

Because ω is derived from the laser rate in GHz, `Tau_phase` and `Tau_mod` are expressed in **nanoseconds**. Note that this differs from the [Lifetime fit](#) extractor, whose lifetime columns are exported in picoseconds.

8 Morphological Features

FLIM Playground extracts single-cell morphological features based on the cell region of interest (ROI) mask of each *channel*. By inputting the ROI mask to `skimage.measure.regionprops`, the following features are extracted:

- **area**: the number of pixels in the ROI.
- **perimeter**: length of the ROI boundary in pixels.
- **solidity**: $\text{Solidity} = \frac{\# \text{ROI pixels}}{\# \text{convex hull pixels}}$, where the convex hull is the smallest convex polygon that contains the ROI. A value close to 1 means the shape has few concavities (more solid), while lower values indicate more irregular boundaries.
- **eccentricity**: equals the eccentricity of the ellipse that has the same second moments as the region. It is given by the ratio of the focal distance (the distance between the foci) to the length of the major axis: $e = \frac{c}{a}$, $e \in [0, 1)$, where a is the semi-major axis length, b is the semi-minor axis length, and $c = \sqrt{a^2 - b^2}$ is the focal distance. When $e = 0$, the ellipse reduces to a circle. It quantifies how elongated the ROI is, with 0 being a perfect circle and 1 being a line.
- **major_axis_length**: the length of the major axis of the ellipse that has the same second moments as the region: $2a$. It represents the longest dimension of the ellipse that approximates the region's shape, essentially describing its maximum elongation.
- **minor_axis_length**: the length of the minor axis of the ellipse that has the same second moments as the region: $2b$. It represents the shortest dimension of that ellipse, capturing the region's minimum elongation.
- **circularity**: describes how close the ROI is to a perfect circle. It is defined as $\text{circularity} = \frac{4\pi A_{\text{ROI}}}{P_{\text{ROI}}^2}$, $\text{circularity} \in (0, 1]$, where A_{ROI} is the area of the region and P_{ROI} is its perimeter. A value of 1 corresponds to a perfect circle.

Feature names in the final dataset are prefixed by the combination of the feature extractor name (i.e. Intensity morphology) and the channel name, allowing [Data Analysis](#) to group the features. For example, Intensity morphology_nadh: area means the area of each cell in the NAD(P)H channel.

9 Textural Features

FLIM Playground extracts single-cell texture features based on the intensity image and the ROI mask from each acquisition *channel*. For an example application, refer to this [paper](#) that used CellProfiler to extract the texture features.

The first step is to create a single-cell intensity image for each cell based on the ROI mask and the intensity image. Then a list of texture features (the list may be extended in the future) is calculated:

9.1 Granularity

$Granularity_n$ is the percentage of intensity contributed by bright objects of n pixels in radius. It is computed by performing a morphological opening with a disk of radius n on the cell intensity image and measuring the intensity lost as a percentage of the total intensity of the original cell ROI.

$n = 1, 3, 5, 7$ are computed.

Semantically, higher granularity at smaller scales implies higher fragmentation.

9.2 Radial Distribution

To calculate the radial distribution, each cell intensity image is partitioned into 4 concentric rings, and the intensity fraction over the total intensity for each ring is calculated. For example, a higher fraction in ring 1 implies that signals are more concentrated at the center of the cell, whereas a higher fraction in ring 4 implies that signals are more concentrated at the cell edge.

9.3 Mass Displacement

Mass displacement is the Euclidean distance (in pixels) between the intensity-weighted centroid of the cell ROI and the geometric centroid of the same ROI.

9.4 Intensity Sum

Intensity sum is defined as the sum of pixel intensities within the cell intensity image.

Note

For a [2D decay](#) channel there is no intensity image or ROI mask, so [Granularity](#), [Radial Distribution](#), and [Mass Displacement](#) cannot be computed. Selecting Intensity Texture for such a channel therefore yields only `intensity_sum`, defined here as the sum of the cell's decay curve (its total photon count across all time bins). The output column is `Intensity texture_{channel}: intensity_sum`.

In the final dataset, feature names are prefixed by the combination of the feature extractor name (i.e. `Intensity texture`) and the channel name, allowing [Data Analysis](#) to group the features. For example, `Intensity texture_nadh: granularity_1` means the percentage of intensity removed when bright objects of 1-pixel radius are removed in the NAD(P)H channel.

Visual explanation

[This post](#) provides a visual explanation of and code to calculate some of the texture features above.

10 Derived Features

FLIM Playground lets users compose new **derived features** by combining existing features, potentially from *different channels*. An example would be the normalized optical redox ratio, composed from the `intensity_sum` feature from each of the two channels: $\text{NAD(P)H} / (\text{NAD(P)H} + \text{FAD})$.

10.1 Build Derived Features

The builder is revealed when the `Derived features` checkbox near the bottom of the [Configuration](#) page is ticked.

Categorical columns (type to add more)

experiment × patient_id × day × hour × cell_type × media × dish × cell_line × treatment × condition × replicate ×

☒ Derived features ⓘ

norm_redox_ratio = Intensity texture_NADH: intensity_sum / (Intensity texture_NADH: intensity_sum + Intensity texture_FAD: intensity_sum) 

Template ⓘ Name ⓘ

Custom... custom_formula

Operands (each becomes A, B, C... in the order picked)

Intensity texture... × Lifetime fit_NADH... × Intensity texture... ×

A = Intensity texture_NADH: intensity_sum, B = Lifetime fit_NADH: t1, C = Intensity texture_FAD: intensity_sum

+ - * / ()

A B C

Formula ⓘ

(A+B*2) / C - 20 Press Enter to apply

✓ Derived: custom_formula = Intensity texture_NADH: intensity_sum + Lifetime fit_NADH: t1

Any derived features already defined for the current profile are listed at the top, each showing its name, its expanded formula, and a  button to delete it. Below the list, a new feature is composed from:

- a **Template** (a common formula, or `Custom...` to write your own),
- a **Name** (e.g. `redox_ratio`) — the output column becomes `Derived: <Name>`, and
- the operands that fill the formula.

Not finding the features you expect?

The operands/features available for selection should span all channels. If no channel has a [feature extractor](#) selected yet, the builder shows “Select feature extractors for your channels first — there are no features to derive from yet.” — configure the extractors first, then come back.

Operands come only from the categorized numerical features. Raw bookkeeping columns — such as `amp1`, `offset`, `reduced_chi_square`, the centroid coordinates, and the `a2 / a3` fit remainders — are intentionally excluded from the operand list and cannot be used as derived-feature inputs.

Derived features are stored **per profile**, so each [extraction profile](#) keeps its own set.

10.1.1 Templates

Choosing a Template other than Custom... fills in the formula for you and renders one searchable dropdown per operand slot (A, B, ...). Four ready-made templates are provided:

Template

Normalized A / (A + B) |

Normalized A / (A + B)

Ratio A / B

Difference A – B

Sum A + B

Custom...

Each takes two operands: pick a feature for A and a feature for B, give the result a Name, and the live preview confirms it. The fifth option, Custom..., unlocks the free formula editor described next.

10.1.2 Custom Formulas

Custom... mode lets you compose an arbitrary formula. First pick the operands from the Operands (each becomes A, B, C... in the order picked) multiselect — the features you choose are aliased A, B, C, ... in pick order, and a caption echoes the mapping (e.g. A = Intensity texture_fad:

intensity_sum, B = ...). Then build the formula in the Formula box, typing directly or using the append buttons for the operators (+ - * / ()) and for each alias.

The formula language supports:

- the four arithmetic operators +, -, *, /,
- a leading sign (unary + / -),
- numeric constants, and
- parentheses for grouping.

Division by zero yields NaN rather than raising an error or producing infinity. Anything outside this language — function calls, exponentiation (**), modulo (%), comparisons, or a bare operand with no operator — is rejected.

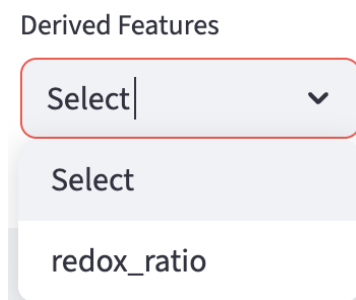
10.2 Compute Derived Features

Derived features are computed during [Numerical Feature Extraction](#), right after each field of view's base features are extracted and concatenated. The definitions are baked into the FOV metadata, so re-running extraction from a saved metadata file reproduces the same derived columns. Each definition adds one `Derived: <name>` column to the output dataset.

If any cell ends up with a NaN in a derived column (for instance from a divide-by-zero or a NaN operand), a single dataset-wide warning is shown for that feature — “Derived feature **name** has **N** NaN value(s) out of *M* cell(s) (e.g. from divide-by-zero or a NaN operand).” As with other features, cells with NaN are **not** dropped from the output.

10.3 Derived Feature In Data Analysis

On the [Data Analysis](#) page, all derived features are collected into one Derived Features group in the [numerical selection widgets](#), alongside the per channel-extractor groups. A derived feature is labeled by its plain name (e.g. `redox_ratio`); unlike extracted lifetime or intensity features, it carries no unit or symbol, since its meaning is entirely user-defined.



11 Categorical Feature Extraction

Based on the datasets extracted in the [numerical feature extraction](#) step, FLIM Playground:

- [merges the datasets](#) into a single dataset
- [assigns categorical features](#) to each cell in the merged dataset

HomeData ExtractionData Analysis

Data Extraction

Select a step to perform

☐ FOV Metadata Extraction

☐ Numeric Feature Extraction (fitting, phasor, etc.)

☒ Categorical Feature Extraction (e.g. treatment)

Copy the folder path here

/Users/allan/Downloads/40

Field of View Name Delimiter

-

Found 1 available csv files ready to be assigned categories

```
{
  0 :
    "/Users/allan/Downloads/40/single_cell_features_20250820_173511"
}
```

Merging datasets...

Finished combining all datasets and got 445 cells!

Now your task is to map the categories to (combination of) slots.

Example fov_name: H9_DAY_40_5 has slots: ['H9', 'DAY', '40', '5']

Choose Categorical features to populate

day

Slots for day

DAY40

Preview of category mapping:

	image_name	day
0	H9_DAY_40_5	DAY_40
33	H9_DAY_40_1	DAY_40
83	H9_DAY_40_2	DAY_40
130	H9_DAY_40_6	DAY_40
180	H9_DAY_40_10	DAY_40

Confirm category mapping & export the combined dataset

11.1 Merging Datasets

It scans the specified folder path and finds all CSV files. It uses the [cell identifier column](#) to check for duplicates both within each dataset and across all datasets.

! Important

The FOV identifier uniquely identifies a field of view and is assumed to contain all information necessary to extract the categorical features (e.g., treatment, time point, etc.), by including segments separated by a certain delimiter (e.g., _). Categorical features are assigned from individual segments or their combinations. Example: The FOV Panc1_Cyanide_dish1

denotes that it comes from the Panc1 cell line, under the Cyanide treatment, in the 1st dish.

Therefore, it checks if all field-of-view names include the same number of segments to determine the eligibility of [categorical feature assignment](#).

Field of View Name Delimiter ?

—

The image_name column in /Users/allan/Downloads/DATA/donor6.csv has different number of parts. For example, check fov_name: HC220113_1n.

The image_name column in /Users/allan/Downloads/DATA/donor5.csv has different number of parts. For example, check fov_name: HC130319_1n.

Found 7 available csv files ready to be assigned categories 😊:

```
[
  0 : "/Users/allan/Downloads/DATA/donor8.csv"
  1 : "/Users/allan/Downloads/DATA/donor9.csv"
  2 : "/Users/allan/Downloads/DATA/donor1.csv"
  3 : "/Users/allan/Downloads/DATA/donor2.csv"
  4 : "/Users/allan/Downloads/DATA/donor3.csv"
  5 : "/Users/allan/Downloads/DATA/donor7.csv"
  6 : "/Users/allan/Downloads/DATA/donor4.csv"
]
```

It merges the datasets that pass the checks by finding common columns and vertically concatenating them.

Merging datasets...

Merging dataset 2 (/Users/allan/Downloads/DATA/donor9.csv) into the combined dataset.

Merging dataset 3 (/Users/allan/Downloads/DATA/donor1.csv) into the combined dataset.

Merging dataset 4 (/Users/allan/Downloads/DATA/donor2.csv) into the combined dataset.

Merging dataset 5 (/Users/allan/Downloads/DATA/donor3.csv) into the combined dataset.

Merging dataset 6 (/Users/allan/Downloads/DATA/donor7.csv) into the combined dataset.

Merging dataset 7 (/Users/allan/Downloads/DATA/donor4.csv) into the combined dataset.

Finished combining all datasets and got 3037 cells!

11.2 Assigning Categorical Features

Available categorical features to be assigned are drawn from the [user-specified configuration](#).

Choose Categorical features to populate

cell_line

treatment

condition

patient_id

day

Based on the assumptions, it breaks down the FOV identifier into individual fields and facilitates the assignment.

Now your task is to map the categories to (combination of) slots.

Example fov_name: H9_DAY_40_5 has slots: ['H9', 'DAY', '40', '5']

A live preview of the assignment is displayed.

Choose Categorical features to populate

cell_line x

day x

Slots for cell_line

H9 x

Slots for day

DAY x

40 x

Preview of category mapping:

	image_name	cell_line	day
0	H9_DAY_40_5	H9	DAY_40
33	H9_DAY_40_1	H9	DAY_40
83	H9_DAY_40_2	H9	DAY_40
130	H9_DAY_40_6	H9	DAY_40
180	H9_DAY_40_10	H9	DAY_40

Confirm category mapping & export the combined dataset

Clicking **Confirm category mapping & export the combined dataset** then writes the merged dataset with the categorical features as a timestamped CSV (`{timestamp}_combined.csv`) into the same folder as the input dataset. Note that this is a direct file write rather than a browser download, so

the file is saved on the machine running FLIM Playground. When using the [hosted demo](#), the file is written to the server and does not reach your computer, which is one reason Data Extraction is intended to be run on a local installation.

Part II

Data Analysis

12 Overview

We are not done yet 🤖! FLIM Playground also provides a suite of methods to analyze and visualize the extracted data (either using the [Data Extraction](#) section or in users' own way). They are designed to be interactive and frictionless ⚡ so that users can perform hassle-free exploration of their data with, hopefully, fun 😊. Inspired by the same data [categorization](#) in [Data Extraction](#), the Data Analysis section designs an [architectural blueprint](#) that all in-house analysis methods build on. It is modularized so that new methods and features can be added in the future easily.

12.1 General Workflow

- Select [numerical feature\(s\)](#) on the left.
- [Subset the data](#) to find data of interest on the top. *All the later interactions are based on this filtered dataset.*
- Use [visual channels widgets](#) to look at the data in different ways.
- Change the plot [style](#).
- [Hover](#) to find data points of interest.
- Interact with method-specific widgets that typically are associated with a modeling method.
- Once users are satisfied with the analysis (visualizations and statistical modeling results), they can [export the entire analysis workflow as a self-contained Python script!](#) 🎉

The visualization and analysis results are updated in real time to reflect users' interactions with any of the above widgets.

12.2 Methods

- Depending on the number of numerical features in the analysis, the methods are categorized into 3 groups:
 - Univariate Analysis
 - * [Feature Comparison](#)
 - * [Feature Histogram](#)
 - Bivariate Analysis
 - * [2D Feature Distribution](#)

- * [Phasor Analysis](#)
- Multivariate Analysis
 - * [Dimension Reduction](#)
 - * [Classification](#)

All the methods [share a set of interactive widgets](#). They also have their own method-specific widgets, the descriptions of which are provided in the corresponding method pages.

Data Analysis

- ☒ **Univariate**
- ☐ **Bivariate**
- ☐ **Multivariate**

Data Analysis

- ☐ **Univariate**
- ☒ **Bivariate**
- ☐ **Multivariate**

Data Analysis

- ☐ **Univariate**
- ☐ **Bivariate**
- ☒ **Multivariate**

Methods

- ☒ **Feature Comparison**
- ☐ **Feature Histogram**
- ☐ **Image Comparison**

Methods

- ☒ **2D Feature Distribution**
- ☐ **Phasor Plot**

Methods

- ☒ **Dimension Reduction**
- ☐ **Classification**

12.3 Input

Users can upload the dataset output from the [Data Extraction](#) directly or *their own datasets* in **CSV** (Comma Separated Values) format after finishing the [interactive configuration](#) setup.

☒ Use Dataset from Data Extraction

Upload the CSV file obtained from [Data Extraction](#) directly.

☐ Use Dataset from Data Extraction

Use the right panel to configure before loading your data ==>

12.3.1 Requirements

! Important

In either case, following the [categorization](#), the dataset should have:

- a column that uniquely identifies each row
- an (optional) field of view identifier column
- a set of numerical features
- zero or more categorical features (e.g. treatment, day, patient id, etc.)

The unique row id and field of view id are used to help users identify the row (e.g. a single cell) and field of view (e.g. a single image) of data of interest through the built-in [hover-based interaction](#). Numerical and categorical features are used to render [shared widgets](#). Internally, FLIM Playground will check whether the dataset fulfills the requirements and output meaningful warning or error messages.

💡 Tip

If you cannot find a certain **numerical feature** under any numerical widgets including Uncategorized Features, please inspect the dataset (e.g., using Excel filters) and look for non-numeric values in that column. A small number of stray non-numeric values (e.g., “-”) is tolerated: if they account for **1% or less** of the column’s non-empty values, FLIM Playground

converts the column to numeric anyway and replaces just those cells with NaN, reporting how many were converted. Above that threshold the column is left as text and will not appear as a numerical feature, so cleaning those cells is only necessary for columns that are mostly non-numeric. FLIM Playground searches for **categorical features** in the uploaded dataset based on the [user-specified configuration](#) if the dataset is not extracted by [Data Extraction](#). Otherwise, it searches for categorical features specified in the [Data Extraction configuration](#).

12.3.2 Warning Messages

- Empty columns: will be dropped.
- Duplicated columns: Only the first occurrence of the duplicated column will be kept. Other occurrences will be dropped.
- Duplicated rows based on the unique row id: Only the first occurrence of the duplicated row will be kept. Other occurrences will be dropped.
- Columns with NaN values: won't be dropped, just a warning message.
 - The analysis will be performed on the rows that are not NaN in the [selected numerical features](#).
- Non-numeric values converted to NaN: reports how many values in a column were non-numeric when that column was otherwise numeric enough to recognize (see above). Those cells become NaN and the column is usable as a numerical feature.

12.3.3 Error Messages

- Missing a column that uniquely identifies each row
- Cannot identify any numerical feature column
 - It uses `pd.api.types.is_numeric_dtype` to check if a column is numerical.

Note

Warning messages will not prevent the analysis but error messages will.

12.4 Shared Interactive Widgets

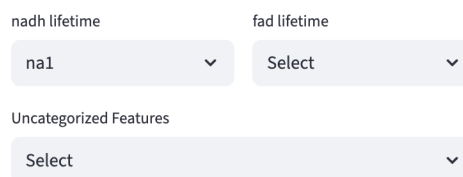
A list of shared widgets is provided to support the [general workflow](#) in analysis.

12.4.1 Numerical Selection Widgets

The Data Extraction recognizes numerical features using `pd.api.types.is_numeric_dtype` internally. Then it groups numerical features based on the [feature extractor type](#) and the channel it belongs to. For user-provided datasets, the numerical features are grouped based on the [user-specified configuration](#). Ungrouped numerical features are put into the Uncategorized Features group. One selection widget is rendered for each numerical feature group.

12.4.1.1 Univariate Analysis

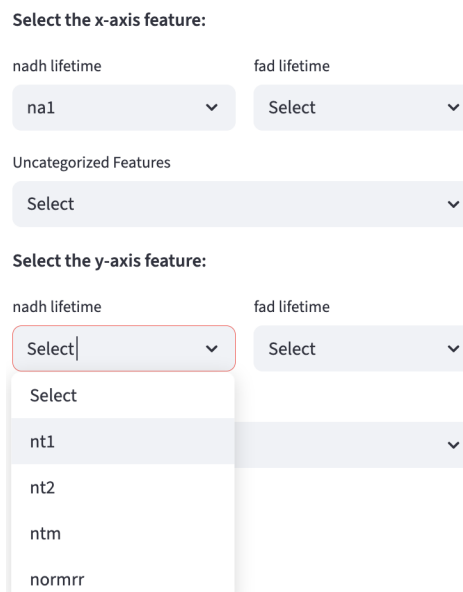
One single-select widget is rendered per group; choosing a feature in any group clears the selections and resets the others to Select.



The image shows three selection widgets for univariate analysis. The first widget is labeled 'nadh lifetime' and has a dropdown menu with 'na1' selected. The second widget is labeled 'fad lifetime' and has a dropdown menu with 'Select' selected. The third widget is labeled 'Uncategorized Features' and has a dropdown menu with 'Select' selected.

12.4.1.2 Bivariate Analysis

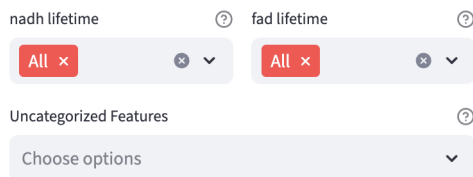
Two sets of selection widgets are rendered to allow maximum flexibility (the two features can be from the same or different feature groups). Each set of selection widgets behaves like the selection widgets in the univariate analysis. The first selected feature will be hidden in the second set of selection widgets.



The image shows two sets of selection widgets for bivariate analysis. The first set is titled 'Select the x-axis feature:' and contains three widgets: 'nadh lifetime' (dropdown with 'na1'), 'fad lifetime' (dropdown with 'Select'), and 'Uncategorized Features' (dropdown with 'Select'). The second set is titled 'Select the y-axis feature:' and contains three widgets: 'nadh lifetime' (dropdown with 'Select' selected and a dropdown menu open showing options 'Select', 'nt1', 'nt2', 'ntm', and 'normrr'), 'fad lifetime' (dropdown with 'Select'), and 'Uncategorized Features' (dropdown with 'Select').

12.4.1.3 Multivariate Analysis

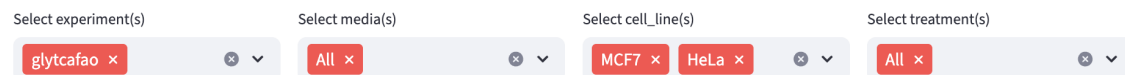
One set of selection widgets, each can select multiple features from a feature group, is rendered. A special value All is introduced so that users can conveniently select all features under the feature group. If users select All, all the other options will be cleared, and vice versa.



The image shows two selection widgets for 'nadh lifetime' and 'fad lifetime'. Each widget has a red 'All x' button, a search icon, and a dropdown arrow. Below these is a section for 'Uncategorized Features' with a 'Choose options' button and a dropdown arrow.

12.4.2 Filter Widgets

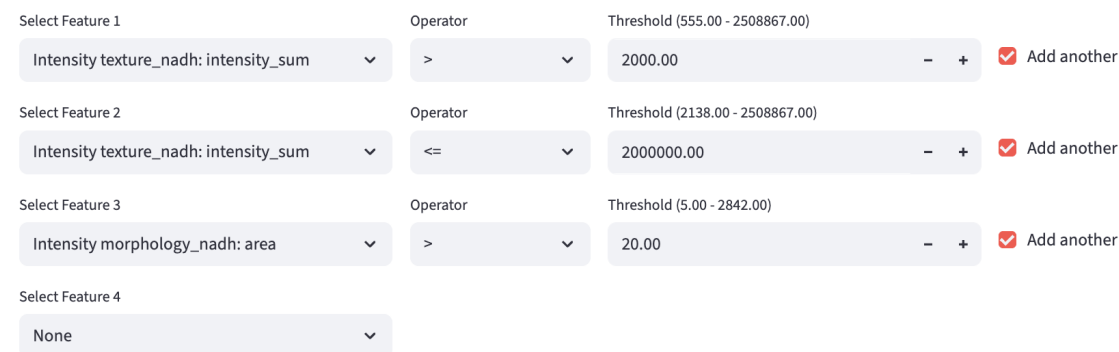
12.4.2.1 Categorical Filters



The image shows four categorical filter widgets: 'Select experiment(s)' with 'glytcafao x', 'Select media(s)' with 'All x', 'Select cell_line(s)' with 'MCF7 x' and 'HeLa x', and 'Select treatment(s)' with 'All x'. Each widget has a red button with an 'x', a search icon, and a dropdown arrow.

For complex datasets that are collected over multiple days, experiments, treatments, etc., it is useful to filter the data to focus on a subset of the data (data of interest). One filter widget is rendered for each categorical feature so that users have the flexibility to filter the data based on combinations of categorical features. All is a special option that include all categories of the selected categorical feature. Once it is selected, all the other options are cleared, and vice versa.

12.4.2.2 Numerical Filters



The image shows four numerical filter widgets. Each widget has a 'Select Feature' dropdown, an 'Operator' dropdown, a 'Threshold' input field with a range, and an 'Add another' button. The first three widgets are for 'Intensity texture_nadh: intensity_sum' with operators '>', '<=', and '>' and thresholds '2000.00', '2000000.00', and '20.00' respectively. The fourth widget is for 'None'.

Users can further filter the data based on numerical features. Each row specifies a condition to filter the data:

- Feature: the numerical feature to filter on

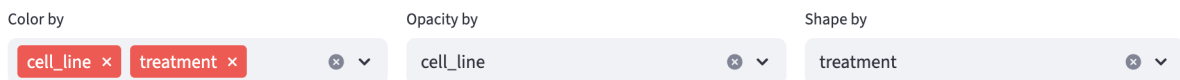
- **Operator:** > or <=
- **Threshold:** the range of values to filter on, dynamically updated based on previous filters, is displayed for reference. The entered value should be within the range.
- **Add Another:** add a new condition in the next row.

After the two sets of filters are applied, the final filtered data is used for visualization and analysis.

! Important

Make sure you clearly state all numerical filters used.

12.4.3 Visual Channels Widgets



Human vision is wired for rapid, parallel pattern and trend detection; good visual encodings (how to map data to visual elements such as color, shape, opacity, etc.) harness this to surface insights that raw numbers or text obscure¹². FLIM Playground provides color, opacity, and shape channels for visualizations that support them:

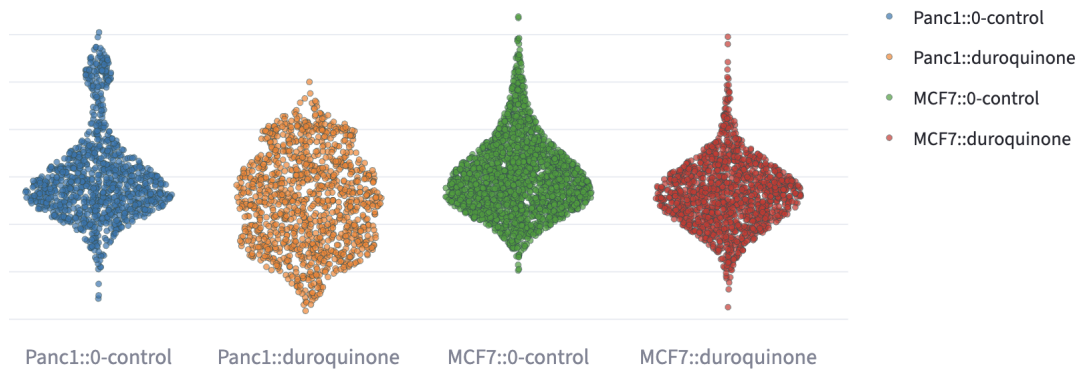
12.4.3.1 Color by

Color is supported in all methods except for [Classification](#). In **Color by**, users can select multiple categorical features, and each unique combination of available categories in selected features is assigned a distinct color. Groups created by **Color by** show up in the x-axis (if multiple features, categories from each feature are delimited by `:`).

i Note 1: How to order the groups

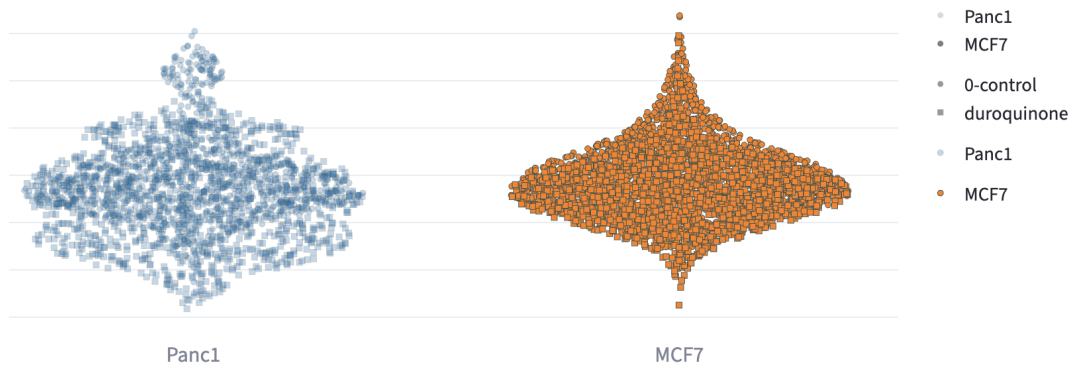
The order of the groups in x-axis and in the legend is determined by:

- the order of the selected features: categories of the first feature appear before those of the second, and so on and so forth. For example, the two treatments of Panc1 are together, because it is sorted first on the cell lines, then on treatments. If you want the treatments to be together, in **Color by** you can select **treatment** then **cell_line**.
- *Numeric-alphabetical sort*: within each feature, the category order is determined by the number inside (e.g. 1 in Panc1) first, then the alphabetical order of the category name. Therefore, although P is after M alphabetically, the 1 is before 7 numerically, making Panc1 appear before MCF7. This may be helpful when you have data from different days or hours. The default string sort will put Day 100 before Day 30, and this is not what we want.



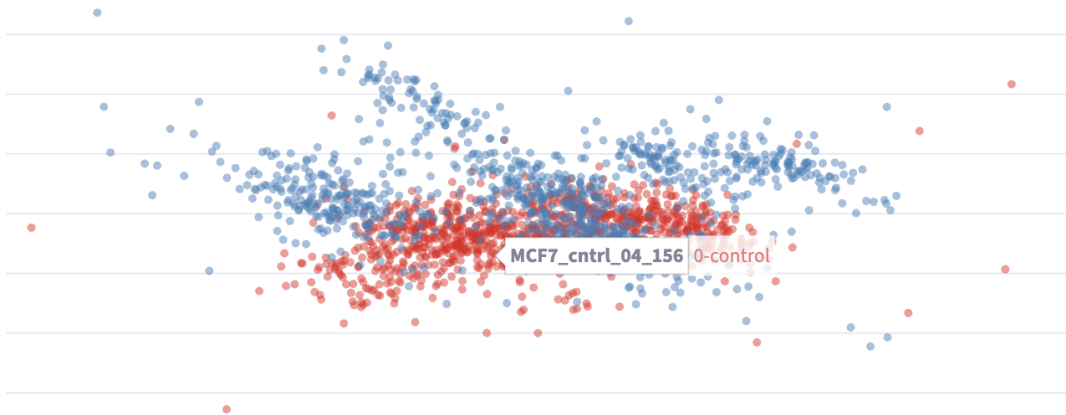
12.4.3.2 Opacity and Shape by

Opacity and shape are supported in all point-based visualizations (e.g. [Feature Comparison](#), [2D Feature Distribution](#), [Phasor Analysis](#), [Dimension Reduction](#)). In **Opacity by** and **Shape by**, users can select one categorical feature, and each unique category is assigned a distinct opacity or shape. The order of the shape and opacity is also sorted [numeric-alphabetically](#).

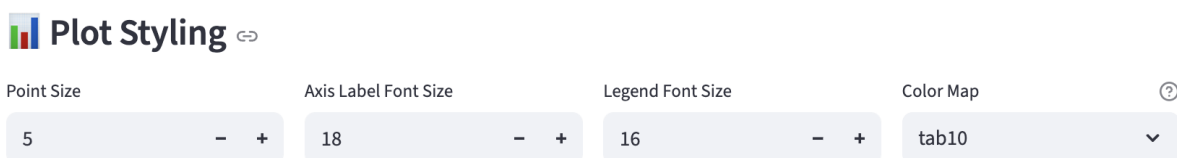


12.4.4 Unique ID Hover

- In all point-based visualizations, when hovering over a point, the unique id of the point will be shown.



12.4.5 Plotting Configuration Widgets



Users can interactively configure the following plot parameters:

- point size (if point-based)
- axis label size
- legend size
- whether to show the per-group count (n) in the legend (when enabled, the count appears on a second line below each group name, e.g. Control with n=42 beneath it)
- the colormap used to color different groups
 - colorblind, tab10, tab20, Set1, Set2, Set3, Pastel1, Pastel2, Accent, viridis, plasma, inferno, magma, cividis

12.4.6 Feature Labels

Numerical features extracted by [Data Extraction](#) are automatically rendered in proper FLIM notation on plot axes, legends, and hover text. For example, the column `Lifetime_fit_nadh: a1` is shown as “ $\text{nadh } \alpha_1$ (%)”, and other extracted features map to symbols such as “ τ_1 (ps)”, “ τ_1 ”, “ $\tau_{1,i}$ ”, “ χ^2_r ”, and “ τ_{mod} ”. The mapping (channel + Greek/scientific symbol + unit) follows the feature naming convention introduced in [Lifetime Features \(Fit\)](#). Columns that are not recognized — identifiers, categorical features, or arbitrary user-provided columns — are displayed unchanged. The same labels are used in the [exported script](#), so its figures match what you see in the app.

12.4.7 Export Workflow as Python Script

At the bottom right of the interface, users can export the entire analysis workflow as a self-contained Python script. This is useful when users want to share their analysis with others, reproduce their analysis later, customize the analysis further, or integrate it into their own workflow. The script will include all the code and analysis configuration (filters, selected numerical features, visual channels, plotting style, module-specific configurations, etc.). After exporting, users need to put the dataset in the same directory as the script and run the script to reproduce the analysis, where the plots are rendered as matplotlib figures and saved as **SVG** files for publication-ready quality.

13 Configuration

FLIM Playground provides an interactive configuration workflow to help users prepare their datasets that are not extracted by [Data Extraction](#) so that they fulfill the [requirements](#).

The following sections use the [iris dataset](#) as an example: the `flower_id` is added to uniquely identify each row (i.e. a single flower). The other columns are: a categorical feature `species`, and a set of numerical features `sepal_length`, `sepal_width`, `petal_length`, and `petal_width`.

Specifically, the steps are:

- On the left side of the screen, uncheck `Use Dataset from Data Extraction`.

☐ Use Dataset from Data Extraction

Use the right panel to configure before loading your data ==>

Then the configuration interface will be displayed.

13.1 Analysis Profiles

✨ **NEW** ✨: Users can now create, save, and switch between multiple analysis profiles—each a ready-to-go configuration tailored for different datasets.

The screenshot displays the 'Analysis Profiles' configuration interface. It consists of three main panels. The left panel, titled 'Profile', contains a dropdown menu with the text 'wine' and a downward arrow. Below the dropdown, the words 'iris' and 'wine' are listed. The middle panel, titled 'New Profile Name', has a text input field containing 'e.g. Profile 2' and a button labeled '+ Create Profile'. The right panel, titled 'Delete Profile', contains a button with a trash icon and the text 'Delete Profile'.

- On the left you can select an existing profile to load.
- In the middle you can create a new profile.
- On the right you can delete an existing profile.

Everything described below pertains to the current profile.

13.2 Identifiers Config

Tell me about ur data ↗

Unique Row ID	FOV Name (if applicable)
flower_id	

- The dataset needs to have a unique identifier column to identify each row. In this case, `flower_id` is the unique identifier column. This dataset does not have a field of view column, so we leave it blank.

13.3 Categorical Feature Config

- Add categorical columns you want FLIM Playground to recognize here. Those columns are converted to strings internally.

Select Categorical Columns

species

Add: species

The default categorical feature list does not include `species`, so we add it manually.

13.4 Numerical Feature Config

Paste numerical features (comma, semicolon, or whitespace separated)

```
sepal_length,sepal_width,petal_length,petal_width
```

- Users can copy the numerical features from Excel or any text editor that opens their CSV files. The features can be separated by `,`, `;`, or whitespaces including newlines.
- After hitting the `ctrl/cmd + enter` key, the numerical features are parsed and added to Available Features. Users can add/delete groups and drag features from the Available Features to the groups they've just created. Any ungrouped features are added to a group called Uncategorized Features.

13.4.1 Example

 **Select Numerical Features**

Choose which features to use for analysis ?

sepal_length × sepal_width × petal_length × petal_width × × ▼

Feature Groups Management

Create New Feature Group

Group Name ?

e.g. lifetime, morphology, texture

Create Group

Delete Feature Group

Select group to delete ?

sepal ▼

 Delete Group

 Available Features

petal_length

 sepal

sepal_length

sepal_width

 petal

petal_width

Save Configuration

Reset Configuration

13.5 Save

- After users finish the configuration and click the **Save Configuration** button, FLIM Playground saves configurations into the current profile and loads it for processing current and future datasets.

13.6 Reset

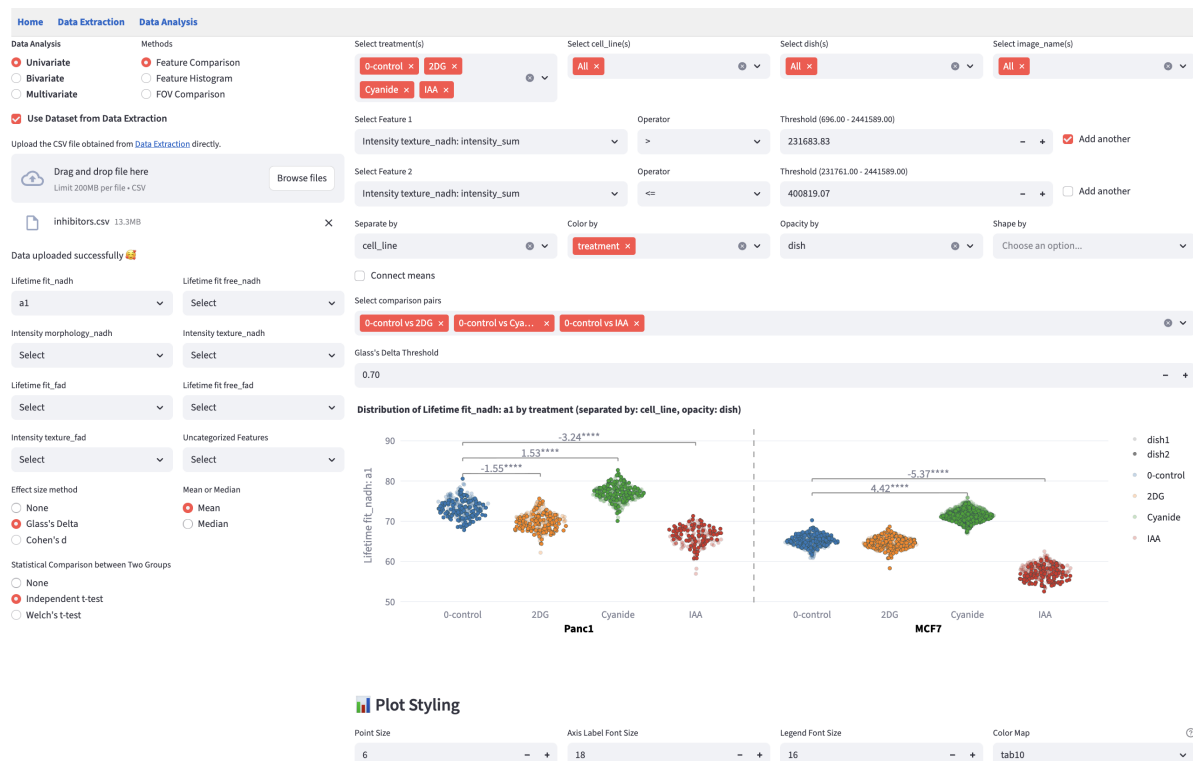
Users can reset the current profile to the default configurations.

Part III

Univariate Analysis

14 Feature Comparison

This method allows users to compare the distributions of a single numerical feature across groups. It can help users spot overarching trends between distributions, identify within-distribution shape-related characteristics such as skewness or bimodality that might signal sub-populations, and diagnose potential outliers.



14.1 Shared Interface Components

- On the left, users can use the [selection widgets](#) to select the numerical feature to be compared. Exactly one feature can be selected from all feature groups.
- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Below the filters, users can apply the [visual channels widgets](#) that allow users to Color by, Opacity by, and Shape by categorical features.

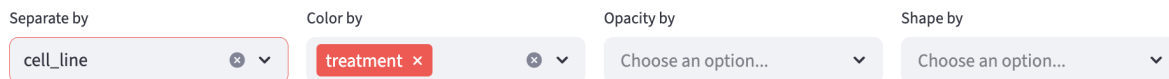
- On the bottom right, users can change the plot style using the [plot styling widgets](#).

14.2 Plot Options

Above the plot, three toggles control how the distributions are displayed:

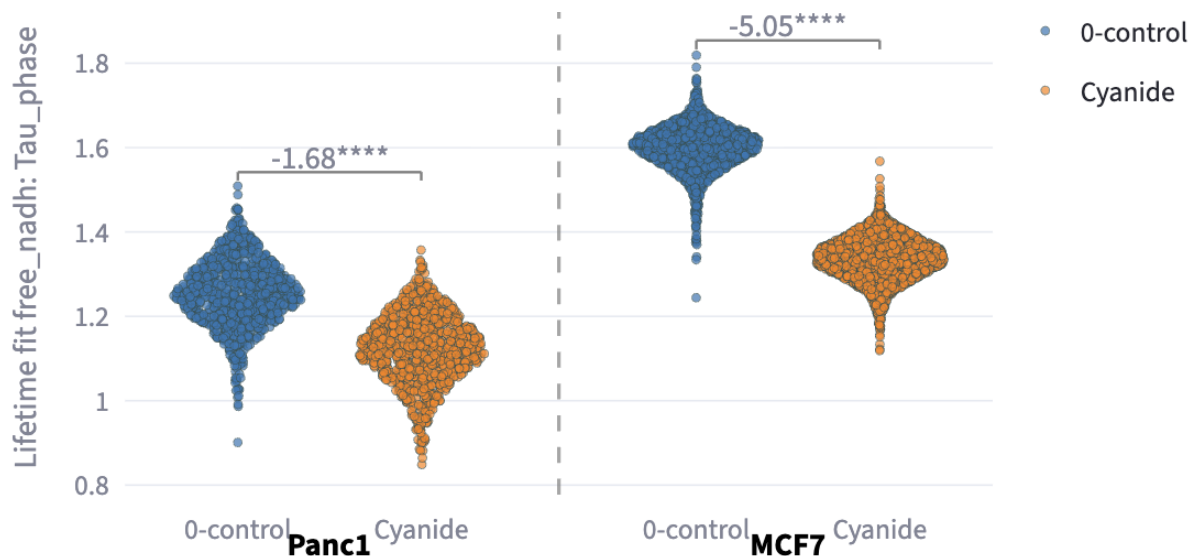
- **Log Y:** plots the selected feature on a base-10 logarithmic y-axis (the axis label becomes $\log(\dots)$). Negative values cannot be log-transformed, so a warning is shown if the selected feature contains any.
- **Add boxplot:** overlays a box-and-whisker summary on each group. The box spans the interquartile range ($Q_1 - Q_3$) with the median as a solid line and the mean as a dashed line; the whiskers follow the [Tukey](#) definition at $Q_1 - 1.5 \times \text{IQR}$ and $Q_3 + 1.5 \times \text{IQR}$, bounded by the data range.
- **Connect means:** draws a line connecting the mean of each group (within each section when [Separate by](#) is used), which can help reveal trends across ordered groups.

14.3 Separate by

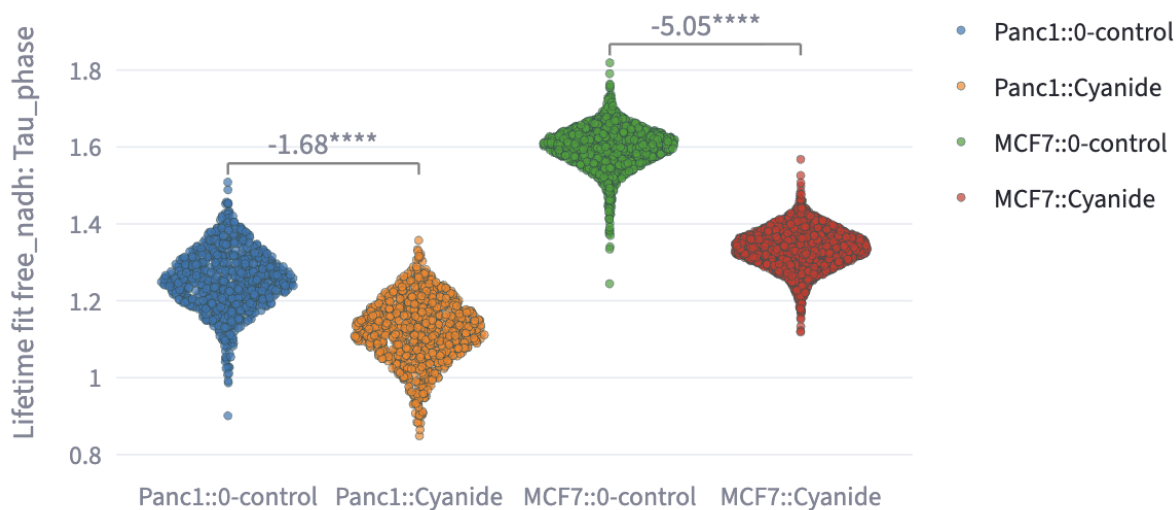


Separate by takes effect before **Color by**. **Separate by** divides the plot into sections separated by a dashed line, with each section labeled by one of the available categories of the selected categorical feature after [filtering](#). The selected feature of **Separate by** is automatically removed from available features in **Color by**. The section order is determined by the order of the categories which are sorted [numeric-alphabetically](#).

Then, visual channels (color, shape, opacity) and effect size annotation are applied within each section. For example, the visualization below was separated by `cell_line`, with each section labeled by the bold text of a cell line. Within each section, the x-axis rendered the **Color by** categories (`treatments`) so colors were consistent across sections. [Effect size](#) calculation and annotation (only effect size > 0.5 is shown) were performed within each section.



If `Separate` by is left blank and both `cell_line` and `treatment` are selected in `Color` by, then colors are assigned to each cell line and treatment combination, and effect sizes are calculated across all combinations of group pairs (only pairs with effect size > 0.5 are shown).

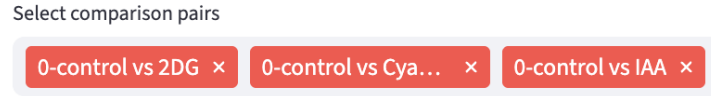


14.4 Comparison Widget

Statistical tests and effect size calculations are performed on the selected comparison pairs. FLIM Playground uses the groups on the x-axis populated by `Color by` to create comparison groups, starting from the leftmost group. For example, if there are 5 groups, then $\binom{5}{2} = 10$ comparisons will be created.

The number of groups can get combinatorially large and many of the comparisons are not meaningful. Therefore, two widgets are provided to filter the comparison groups:

- A selection widget that shows all possible comparisons by default and users can deselect groups they do not need.

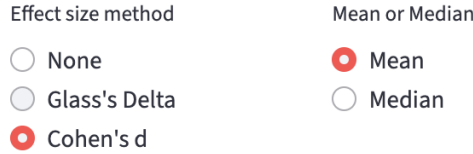


- A threshold by effect size widget that filters out comparisons below the threshold.



14.5 Effect Size

Complementary to statistical tests, which address “is there a difference?”, effect size metrics answer “how large is the difference?”, enabling cross-study comparison. FLIM Playground offers two effect size calculation methods, Glass’s Delta and Absolute Cohen’s d, each can be in mean or median form.



14.5.1 Glass’s Delta

Mean ($\bar{X}$) form:

$$\Delta_G = \frac{\bar{X}_{\text{treatment}} - \bar{X}_{\text{control}}}{s_{\text{control}}}, \quad s_{\text{control}} = \sqrt{\frac{1}{n_c - 1} \sum_{i=1}^{n_c} (X_{\text{control}_i} - \bar{X}_{\text{control}})^2}$$

Median ($\tilde{X}$) form:

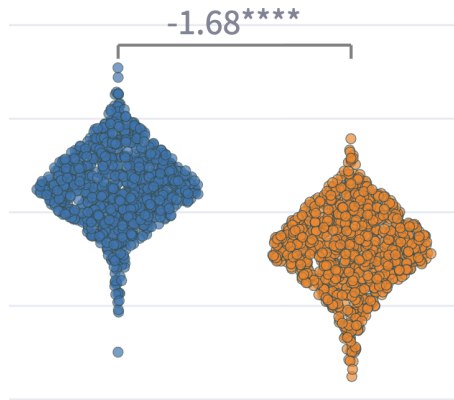
$$\tilde{\Delta}_G = \frac{\tilde{X}_{\text{treatment}} - \tilde{X}_{\text{control}}}{\text{MAD}_{\text{control}}}, \quad \text{MAD}_{\text{control}} = 1.4826 \times \text{median} |X_{\text{control}_i} - \tilde{X}_{\text{control}}|, \quad i = 1, \dots, n_c$$

i Note

MAD stands for [median absolute deviation](#). In order to make MAD asymptotically consistent with the standard deviation under normality, it is scaled by $C = 1/\Phi^{-1}(0.75) \approx 1.4826$. Implementation-wise, it uses `median_abs_deviation` from `scipy.stats`, and `scale` option is set to "normal" to account for the constant multiplier C . THE MEDIAN FORM of both Glass's Delta and Absolute Cohen's d IS NOT REPORTED IN THE LITERATURE.

! Important

Which group is treatment and which is control matters! From the formula of Δ_G , switching the order not only flips the sign of the result, but also changes the denominator, which is s_{control} , the standard deviation of the *control*. Currently, FLIM Playground treats the group on the left as control and the group on the right as treatment (the negative Δ_G below implies this). FLIM Playground now supports user-customizable x-axis order so that users can arrange the groups to make sure the control group is on the left. Or users can use [Absolute Cohen's d](#) that does not assume the treatment-control structure.



i Note

To keep the point-based visualization style that supports [interactive hover](#) while showing the violin-plot-like distribution, [Sina plot](#) is used: it uses `gaussian_kde` from `scipy.stats` to make sure the width of the point distribution is proportional to the kernel density.

14.5.2 Absolute Cohen's d

Mean ($\bar{X}$) form:

$$|d| = \left| \frac{\bar{X}_1 - \bar{X}_2}{s_p} \right|, \quad s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

Median ($\tilde{X}$) form:

$$|\tilde{d}| = \left| \frac{\tilde{X}_1 - \tilde{X}_2}{s_{\tilde{p}}} \right|, \quad s_{\tilde{p}} = \sqrt{\frac{(n_1 - 1) \text{MAD}_1^2 + (n_2 - 1) \text{MAD}_2^2}{n_1 + n_2 - 2}}$$

14.6 Statistical Test

Statistical tests are performed on the selected comparison pairs that exceed the effect size threshold (if users choose to calculate effect sizes). The statistical tests supported are:

- Independent t-test (student's t-test, for equal variances)
- Welch's t-test (for unequal variances)

Statistical Comparison between Two Groups

- ☐ None
☒ Independent t-test
☐ Welch's t-test

FLIM Playground uses `scipy.stats.ttest_ind` to perform the independent t-test and `scipy.stats.ttest_ind` with `equal_var=False` to perform the Welch's t-test.

P-values are summarized with significance asterisks: if $p \leq 0.0001$, four asterisks (****) are shown; otherwise if $p \leq 0.001$, three (**); otherwise if $p \leq 0.01$, two (**); otherwise if $p \leq 0.05$, one (*). Values with $p > 0.05$ are not given a significance asterisk marker in this convention.

14.7 Comparison Annotation

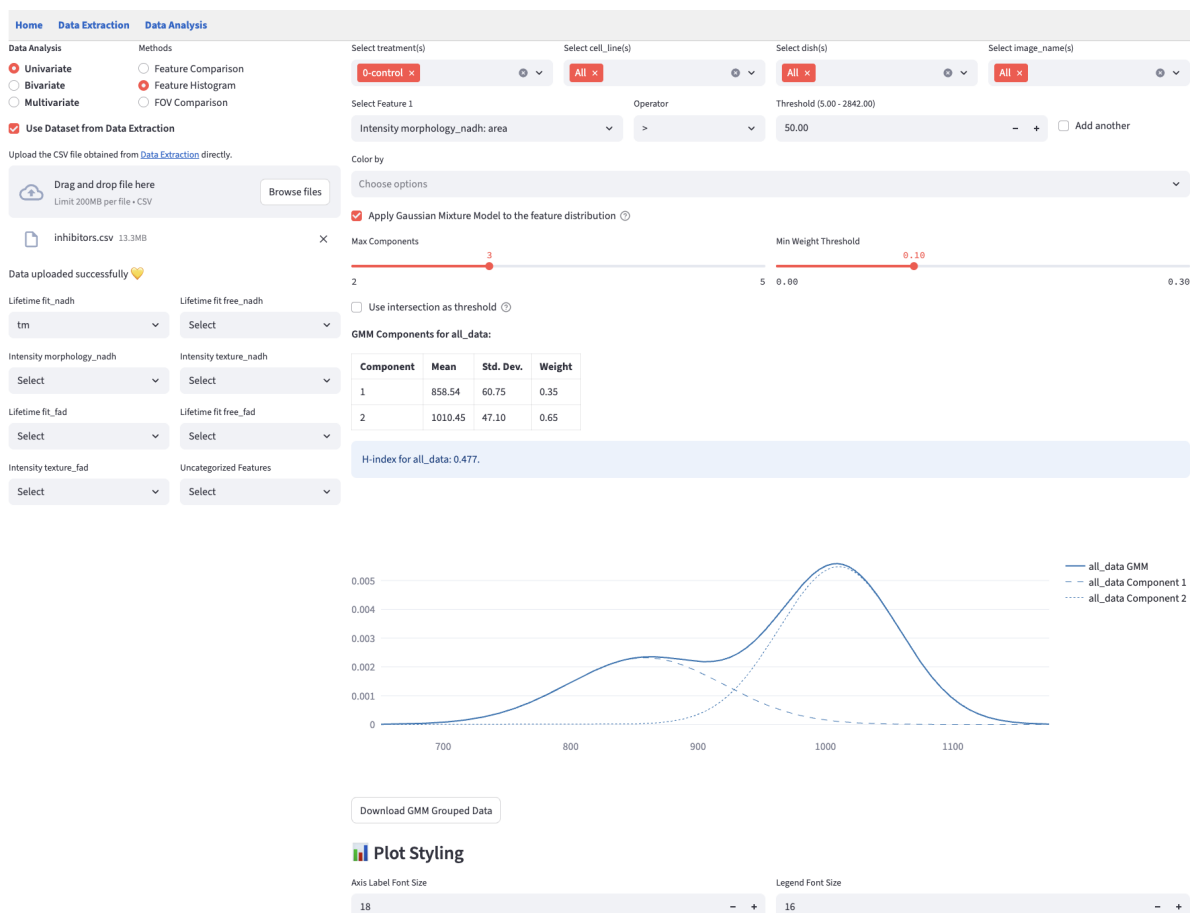
For each selected comparison pair that exceeds the effect size threshold (if any), the statistical significance, effect size, or both are annotated on the plot. It selects the lowest eligible position (over all points in both groups and over existing annotations) to annotate. If both statistical significance and effect size calculation are performed, the annotation is shown in the format of effect size followed by p-value. If only one is performed, then only the performed method's result is shown.

$$\underline{\Delta=1.65}$$

$$\underline{1.65*****}$$

15 Feature Histogram

Feature histograms give an instant readout of each group's center, spread, skew and any multi-peaked shape that raw tables hide. FLIM Playground plots a histogram for each group based on the selected numerical feature. Additionally, the [Gaussian Mixture Model](#) mode can help find subcomponents of each histogram in an unsupervised way. To quantify the distribution heterogeneity, normalized [H-index](#) is calculated for each group.



15.1 Shared Interface Components

- On the left, users can use the [selection widgets](#) to select the numerical feature to see the histogram/GMM distribution. Exactly one feature can be selected from all feature groups.
- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Below the filters, users can apply the [visual channels widgets](#) that allow users to Color by categorical features. Opacity by and Shape by are *not* supported because there are no points in the histogram/GMM plot.
- On the bottom right, users can change the plot style using the [plot styling widgets](#).

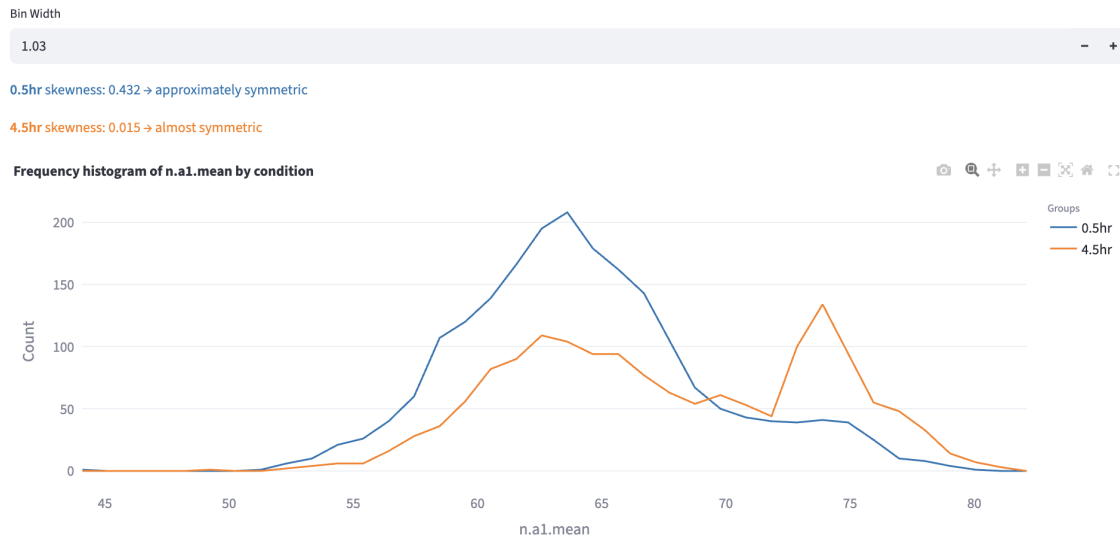
15.2 Histogram Mode

With a single click, users can switch between the [histogram mode](#) and the [Gaussian Mixture Model mode](#). When the checkbox is unchecked, the histogram mode is selected.

☐ Apply Gaussian Mixture Model to the feature distribution [?](#)

A **Log X** toggle (next to the mode switch) replaces the selected feature with its base-10 logarithm *before anything else is computed*, so the transform propagates into both modes: in histogram mode the bins are placed on the log-transformed values, and in GMM mode the model is **fit on the log-transformed feature** — its components, BIC-based model selection, [classification](#) thresholds, and [H-index](#) are therefore all computed in log space. Negative values cannot be log-transformed, so a warning is shown if the selected feature contains any.

Histogram mode plots a histogram of the selected numerical feature for each group. The histogram bin width is user-controllable, with the default bin width determined automatically as the minimum bin width between the ‘sturges’ and ‘fd’ estimators (see [here](#) for more details). The maximum bin width is set to one-third of the range of the data. The step size is set to 1/50 of the range.



Additionally, the skewness of the distribution is computed and displayed¹³.

A rule of thumb categorization of skewness is as follows:

- Strongly left-skewed: < -1
- Moderately left-skewed: -1 to -0.5
- Approximately symmetric: -0.5 to -0.25 and 0.25 to 0.5
- Almost symmetric: -0.25 to 0.25
- Moderately right-skewed: 0.5 to 1
- Strongly right-skewed: > 1

However, the skewness is just one way to quantify the distribution shape. In the example above, the orange distribution was clearly bimodal, but the skewness alone could not capture this. In applications where the distribution is multi-modal, and each mode can represent certain subpopulations (e.g. a cell type, a cell state, a cell cycle phase, etc.), users can use the [Gaussian Mixture Model mode](#) to find subcomponents of each distribution in an unsupervised way.

15.3 Gaussian Mixture Model Mode

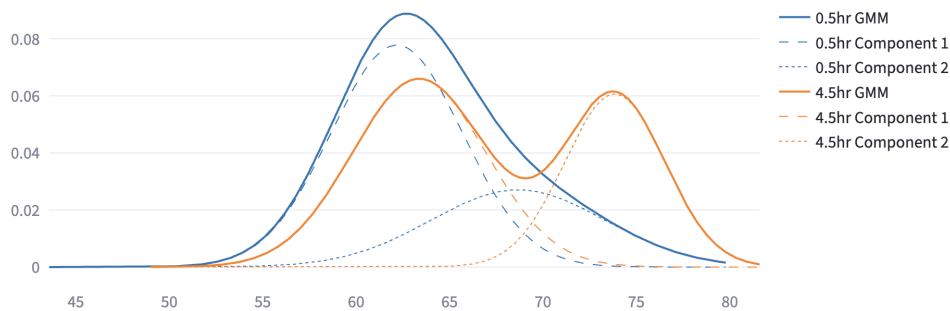
GMM Components for 0.5hr:

Component	Mean	Std. Dev.	Weight
1	62.18	3.51	0.68
2	68.67	4.66	0.32

GMM Components for 4.5hr:

Component	Mean	Std. Dev.	Weight
1	63.37	3.58	0.59
2	73.84	2.68	0.41

H-index for 0.5hr: 0.468. H-index for 4.5hr: 0.485.



A Gaussian Mixture Model (GMM) represents a dataset as a weighted sum of multiple Gaussian (normal) distributions, each defined by its own mean (μ) and (co)variance (σ^2). By estimating the component parameters and their weights (mixing coefficients) from the data—commonly using the Expectation-Maximization (EM) algorithm—GMMs can capture complex, multimodal distributions that a single Gaussian cannot.

15.3.1 Fit Gaussian Mixture Models



Figure 15.1: Gaussian Mixture Model Configuration

- Fits GMMs on the selected numerical feature up to **Max Components** components, each of which has its component weights (w_i), means (μ_i) and variances (σ_i^2).
- Retains only *valid* models in which every component has weight at least **Min Weight Threshold** (the weights of all components add to 1).
- FLIM Playground will choose the valid model with the lowest BIC score that penalizes the model complexity to avoid overfitting.

15.3.2 Heterogeneity index

To quantify the subpopulation structure, based on the GMM fit, FLIM Playground computes an “H-index” for each group as a weighted entropy-distance: each component contributes according to its weight’s uncertainty $(-w_i \log w_i)^{14}$ scaled by how far its mean μ_i sits from the overall mixture mean $\bar{\mu} = \sum_{i=1}^k w_i \mu_i$, normalized by the standard deviation of μ_i s.

$$H = \sum_{i=1}^k (-w_i \log w_i) d_i, \quad d_i = \frac{|\mu_i - \bar{\mu}|}{\sigma}$$

15.3.3 Classification

- Users can choose to perform [intersection thresholding](#) or [hard assignment](#) to determine which GMM component each data point belongs to. It appends a new categorical feature column called GMM_group (recognizable by methods in Data Analysis as a categorical feature) to the filtered dataset, the values of which are {color_group}_group1, {color_group}_group2, etc., numbered in ascending order of the component means. In the example above, the color groups were created by 0.5hr and 4.5hr because the user colored the data by hour, so the resulting values are 0.5hr_group1, 0.5hr_group2, 4.5hr_group1, and 4.5hr_group2. The classification result can be saved by clicking:

Download GMM Grouped Data

15.3.3.1 Intersection thresholding

- Once the GMM is estimated, thresholds can be obtained by locating the intersection points of adjacent component densities:

$$w_1 \mathcal{N}(x; \mu_1, \sigma_1^2) = w_2 \mathcal{N}(x; \mu_2, \sigma_2^2)$$

where w_1, μ_1, σ_1 are the mixture weight, mean, and standard deviation of the first Gaussian component, respectively, and w_2, μ_2, σ_2 those of the second component.

- The above equation can be formed as a quadratic equation and solved for the unique root lying between μ_1 and μ_2 . In practice, we bracket this interval and apply a robust one-dimensional root-finding routine (e.g. Brent’s method) to compute intersection points. Data points are then assigned to the class corresponding to the interval in which they fall.

! Important

In rare cases when there is more than one intersection point, the data point is assigned using hard assignment rather than intersection thresholding.

15.3.3.2 Hard assignment

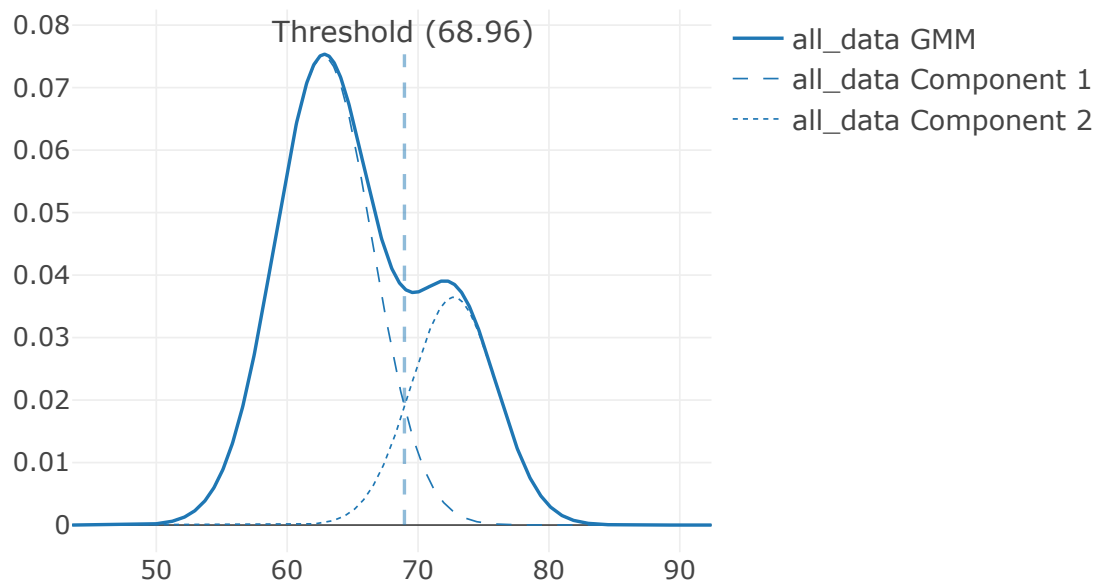
- For each data point, it computes the posterior probability (responsibility) of each component, and assigns the data point to the component with the highest responsibility.

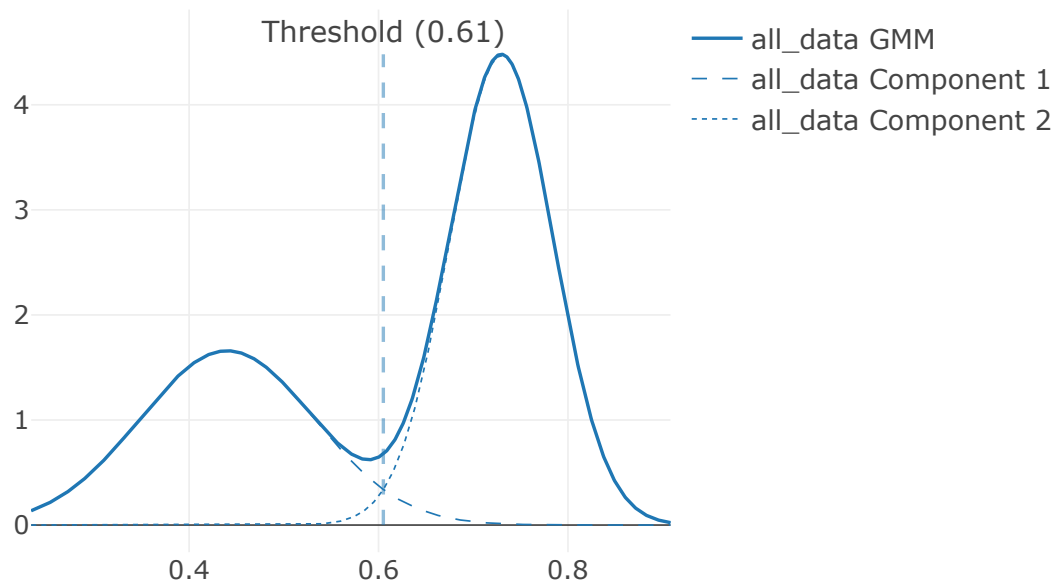
$$\text{label}(x) = \arg \max_i \gamma_i(x)$$

$$\text{where } \gamma_i(x) = \frac{w_i \mathcal{N}(x \mid \mu_i, \sigma_i^2)}{\sum_{j=1}^K w_j \mathcal{N}(x \mid \mu_j, \sigma_j^2)}$$

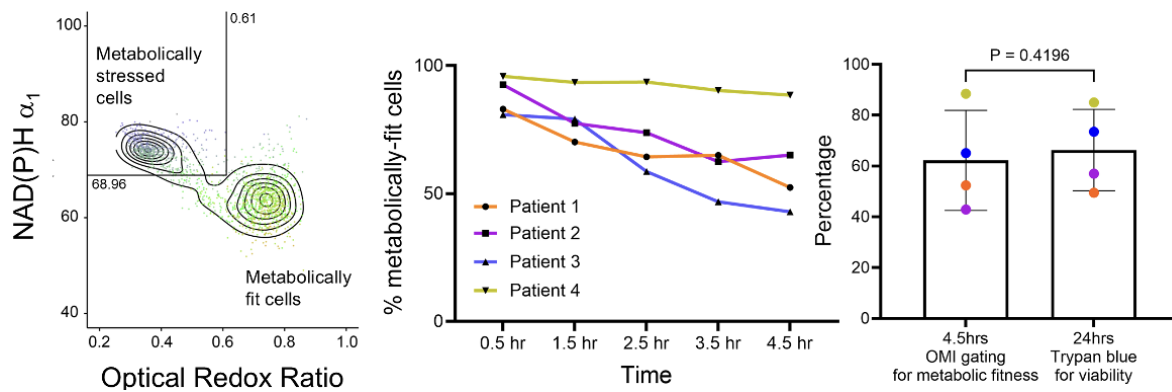
15.3.4 Example

Pham et al.¹⁵ used GMM with intersection thresholding to create gates using NAD(P)H a_1 and optical redox ratio. These gates were used to define metabolically stressed and fit cells in their study of the impact of cryopreservation on T cell metabolism.





They compared the GMM-derived gate with cell viability by Trypan blue staining:



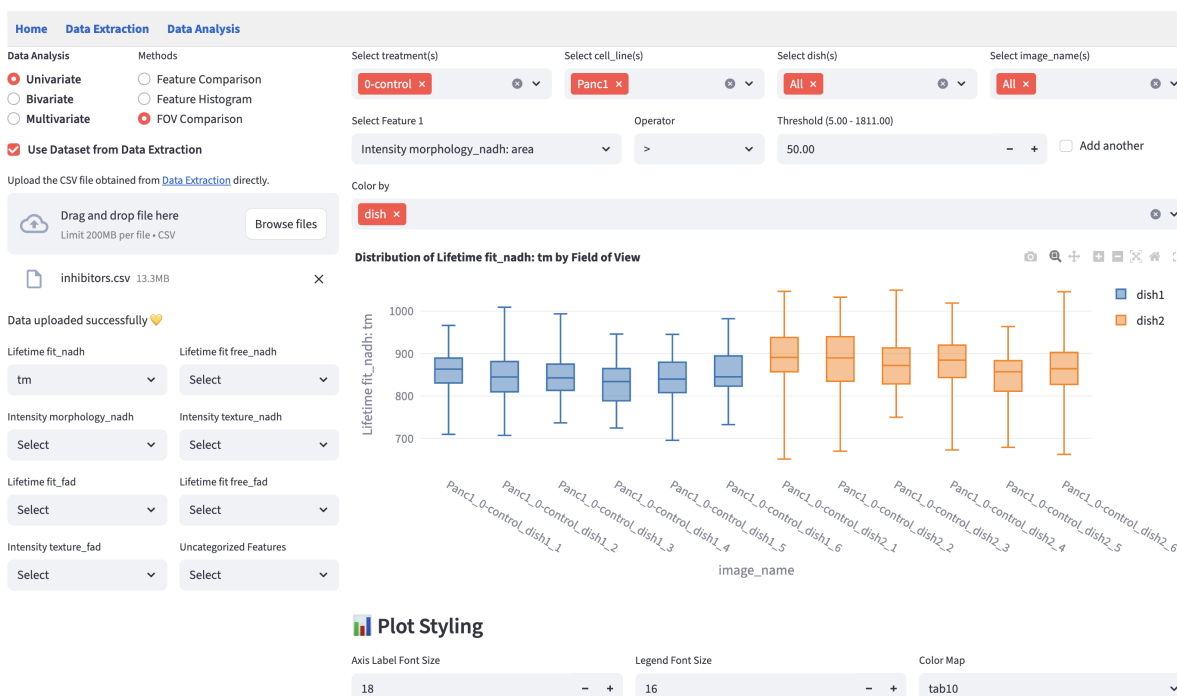
It suggests that optical metabolic imaging (OMI) could serve as an early, non-destructive indicator of T cell viability post-thaw.

16 Field of View Comparison

In imaging datasets like FLIM data, field of view (FOV) is an important categorical feature that documents where a certain set of rows (e.g. cells) originate. Visualizing each FOV side by side on a numerical feature can help identify potential FOV-level outliers.

FLIM Playground finds the FOV identifier column from the dataset using the `FOV column name` field in [data analysis config](#) (dataset not extracted by Data Extraction) or [data extraction config](#) (dataset extracted by [Data Extraction](#)).

If found:



If the column is not found, FLIM Playground does not stop. It falls back to deriving a FOV name for each row from that row's [unique row id](#), by taking everything before the last underscore. A row with the id `day1_image3_17` is therefore assigned to the FOV `day1_image3`. This matches the `{FOV Identifier}_{cell_label}` convention used by [Data Extraction](#), so for datasets extracted by FLIM Playground the derived value is the correct FOV name.

Only rows whose unique row id contains no underscore at all cannot be split. Those are labeled `missing fov name`, and the warning below appears if at least one such row is present:

Warning: We cannot find the fov column from your dataset.

 Warning

Because the fallback is silent when every row id happens to contain an underscore, a plot can be rendered with no warning while grouping by something that is not a real FOV. If your unique row ids do not follow the `{FOV}_{cell}` convention, the boxplots will be grouped by an arbitrary fragment of the id string. Confirm that the `FOV column name` in the [data analysis config](#) actually matches a column in your dataset before reading anything into this plot.

16.1 Shared Interface Components

- On the left, users can use the [selection widgets](#) to select the numerical feature to see the boxplot of each FOV. Exactly one feature can be selected from all feature groups.
- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Below the filters, users can apply the [visual channels widgets](#) that allow users to `Color` by categorical features. `Opacity` by and `Shape` by are *not* supported because there are no points in the boxplot.
- On the bottom right, users can change the plot style using the [plot styling widgets](#).

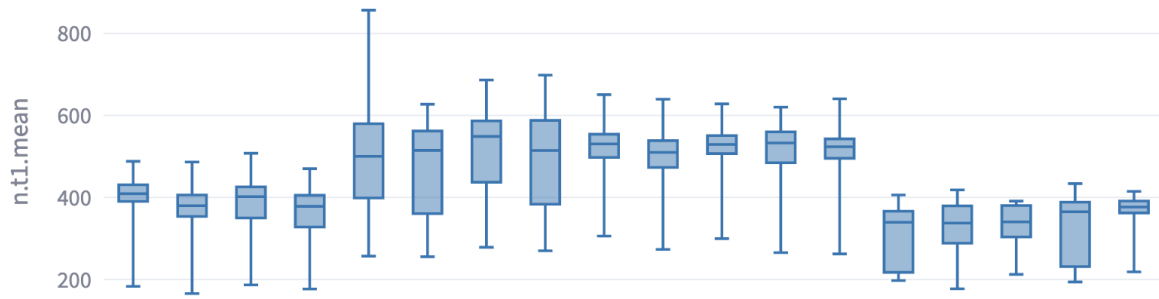
Besides the shared components, this module does not have any method-specific components.

 Note

The box for each FOV follows the [Tukey](#) convention: the box spans the interquartile range ($Q_1 - Q_3$) with the median line, and the whiskers extend to the most extreme points within $Q_1 - 1.5 \times \text{IQR}$ and $Q_3 + 1.5 \times \text{IQR}$.

16.2 Example

Sometimes, the FOV-level boxplots may show consistent divergence across different imaging days, which may suggest that there is experimental or data extraction inconsistency (e.g. shifts are not optimized correctly for one day versus the others). Below is an example that, despite the same conditions, systematically diverged across days, but was consistent within the same day.

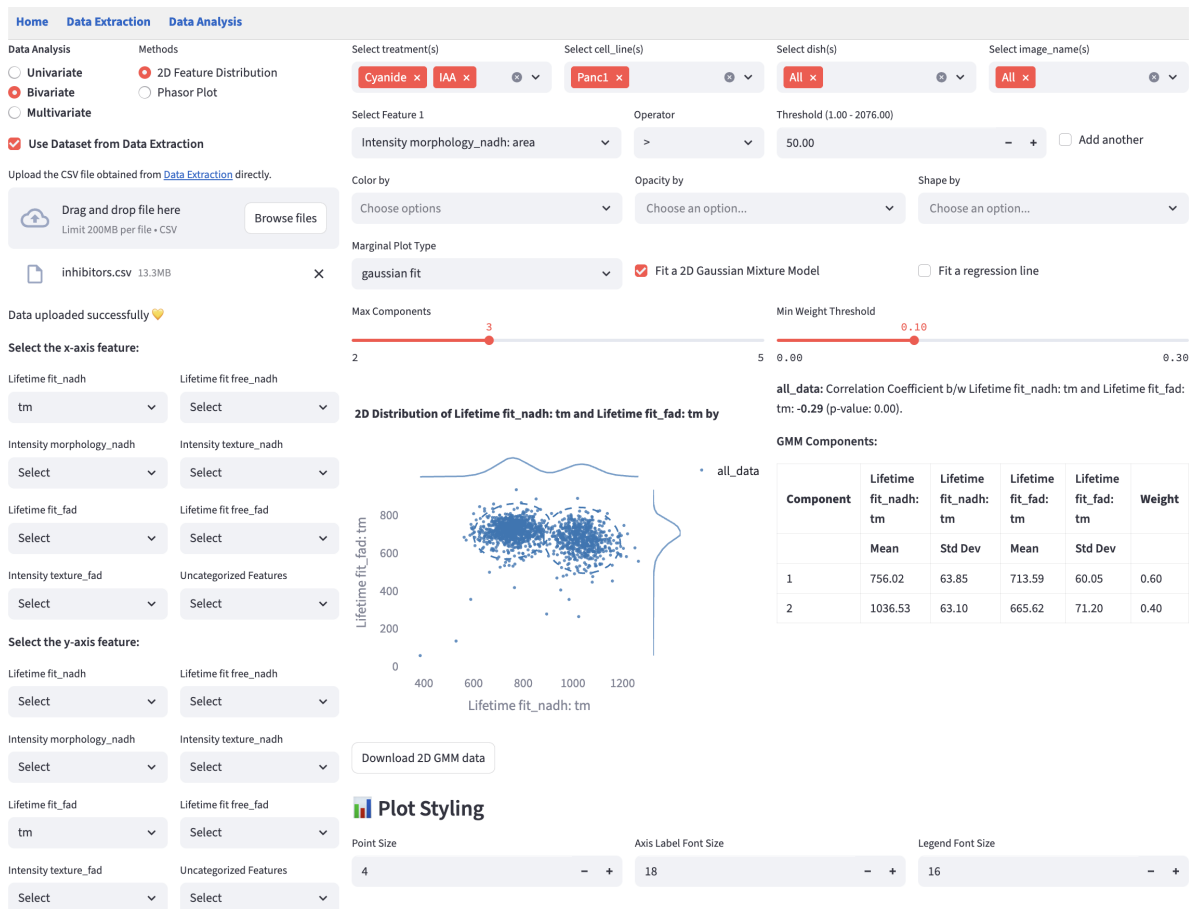


Part IV

Bivariate Analysis

17 Feature Distribution

This method extends the univariate [Feature Histogram](#) method to visualize the distribution of two numerical features on a scatter plot. It helps reveal patterns such as correlations, clusters, trends, or outliers.



17.1 Shared Interface Components

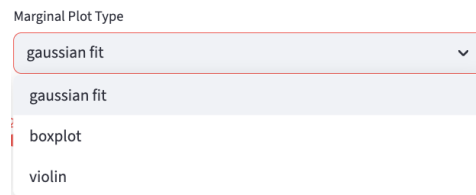
- On the left, users can use the [selection widgets](#) to select the two numerical features on the x-axis and y-axis.
- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Below the filters, users can apply the [visual channels widgets](#) that allow users to Color by, Opacity by, and Shape by categorical features.
- On the bottom right, users can change the plot style using the [plot styling widgets](#).

17.2 Log-scale Axes

Log X and **Log Y** toggles plot the respective axis on a base-10 logarithmic scale, and the axis is relabeled `log ( . . . )`. The base-10 logarithm is applied to the selected feature(s) *before any modeling*, so everything downstream is computed in log space: the **2D GMM** (its component means and covariances, BIC-based model selection, 95% ellipses, and [hard-assignment classification](#)) and the **regression line** (its slope and R^2) are all fit on the log-transformed values. Negative values cannot be log-transformed, so a warning is shown if a selected feature contains any.

17.3 Marginal Distribution

Marginal distributions (the distribution of a single feature) are plotted at the top and the right of the scatter plot. Users can choose the plot type from the dropdown menu, which offers three options.



- **gaussian fit**: provides a kernel-density estimate using Gaussian kernels. It uses `gaussian_kde` from `scipy.stats` and sets all parameters to default values.
- **boxplot**: summarizes each color group with a box and whiskers, which makes the median and spread easier to compare across groups than overlapping density curves.
- **violin**: shows the full distribution shape for each color group, combining the density information of the Gaussian fit with the per-group separation of the boxplot.

17.4 2D Gaussian Mixture Model

Using the same method as fitting a 1D [Gaussian Mixture Model](#), users can fit a 2D GMM by specifying `Max Components` and `Min Weight Threshold` for each color group.

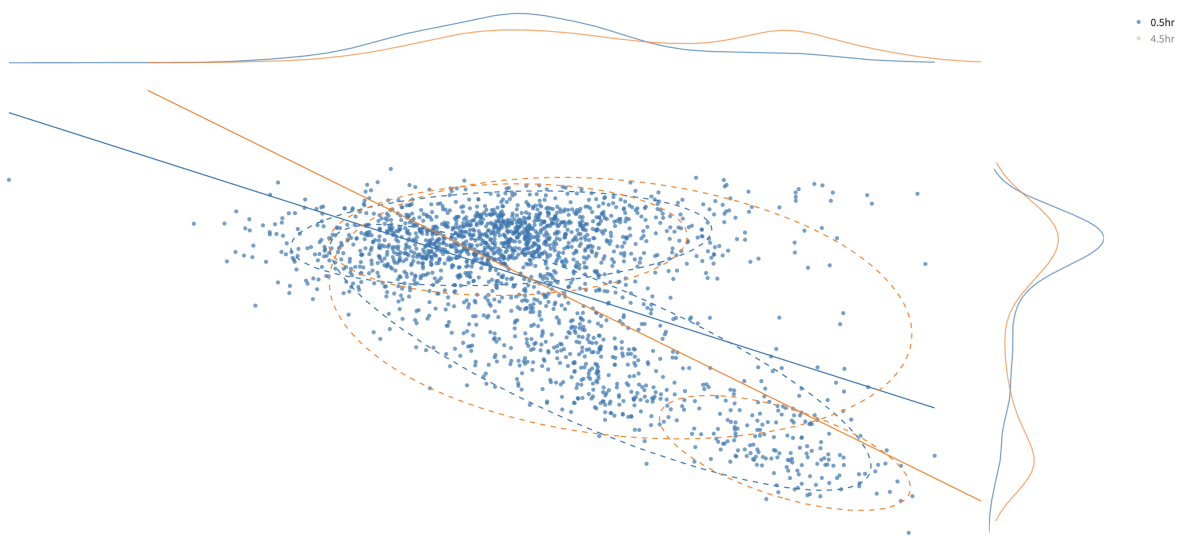
Ellipses that capture approximately 95% of the points in each subcomponent are drawn. The color of the ellipses is determined by the color group that the subcomponent belongs to.

i How ellipses are drawn

Each ellipse represents a subcomponent of the GMM.

- Center of the ellipse is the mean of the subcomponent (μ_x, μ_y) .
- The ellipse is rotated by $\theta = \arctan 2(v_{1y}, v_{1x})$, where $\mathbf{v}_1 = \begin{bmatrix} v_{1x} \\ v_{1y} \end{bmatrix}$ is the first eigenvector of the covariance matrix, so that the major axis of the ellipse is aligned with the eigenvector corresponding to the largest eigenvalue.
- The semi-axis lengths are $r\sqrt{\lambda_1}$ and $r\sqrt{\lambda_2}$, where λ_1 and λ_2 are the covariance eigenvalues (variances along the principal axes) scaled by $r = \sqrt{\chi^2_{df=2, p=0.95}}$ (Mahalanobis distance) that captures approximately 95% of the points in the subcomponent.

All the components below will be rendered for each color group, which is created by user-chosen categorical features in [Color by](#).





Tip

Users can click on a legend group to hide/show the data points in that group. In the above example, to avoid cluttering the plot, the orange group is hidden.

Tables of subcomponents' means, standard deviations, and weights are reported on the side.

0.5hr: Correlation Coefficient b/w n.a1.mean and rr.mean: -0.56 (p-value: 0.00).

GMM Components:

Component	n.a1.mean	n.a1.mean	rr.mean	rr.mean	Weight
	Mean	Std Dev	Mean	Std Dev	
1	66.71	5.29	0.54	0.11	0.36
2	62.83	4.12	0.74	0.04	0.64

4.5hr: Correlation Coefficient b/w n.a1.mean and rr.mean: -0.76 (p-value: 0.00).

GMM Components:

Component	n.a1.mean	n.a1.mean	rr.mean	rr.mean	Weight
	Mean	Std Dev	Mean	Std Dev	
1	73.43	3.12	0.39	0.07	0.39
2	63.87	4.33	0.72	0.07	0.61

The classification result can be saved by clicking:

Download 2D GMM data

The classification is done by [hard assignment](#) to determine which GMM component each data point belongs to. It appends a new categorical feature column called 2D_GMM_group to the filtered dataset, the values of which are {color_group}_group1, {color_group}_group2, etc. The downloaded dataset keeps all the features plus the new categorical feature column, which is recognized by any method in Data Analysis as a categorical feature like others. The rows that pass the [filters](#) are included.

17.5 Regression line

A linear regression line is fitted to the data points in each color group. The key statistics are reported on the side.

Regression R²: 0.311 (Slope: -0.014, Intercept: 1.570)

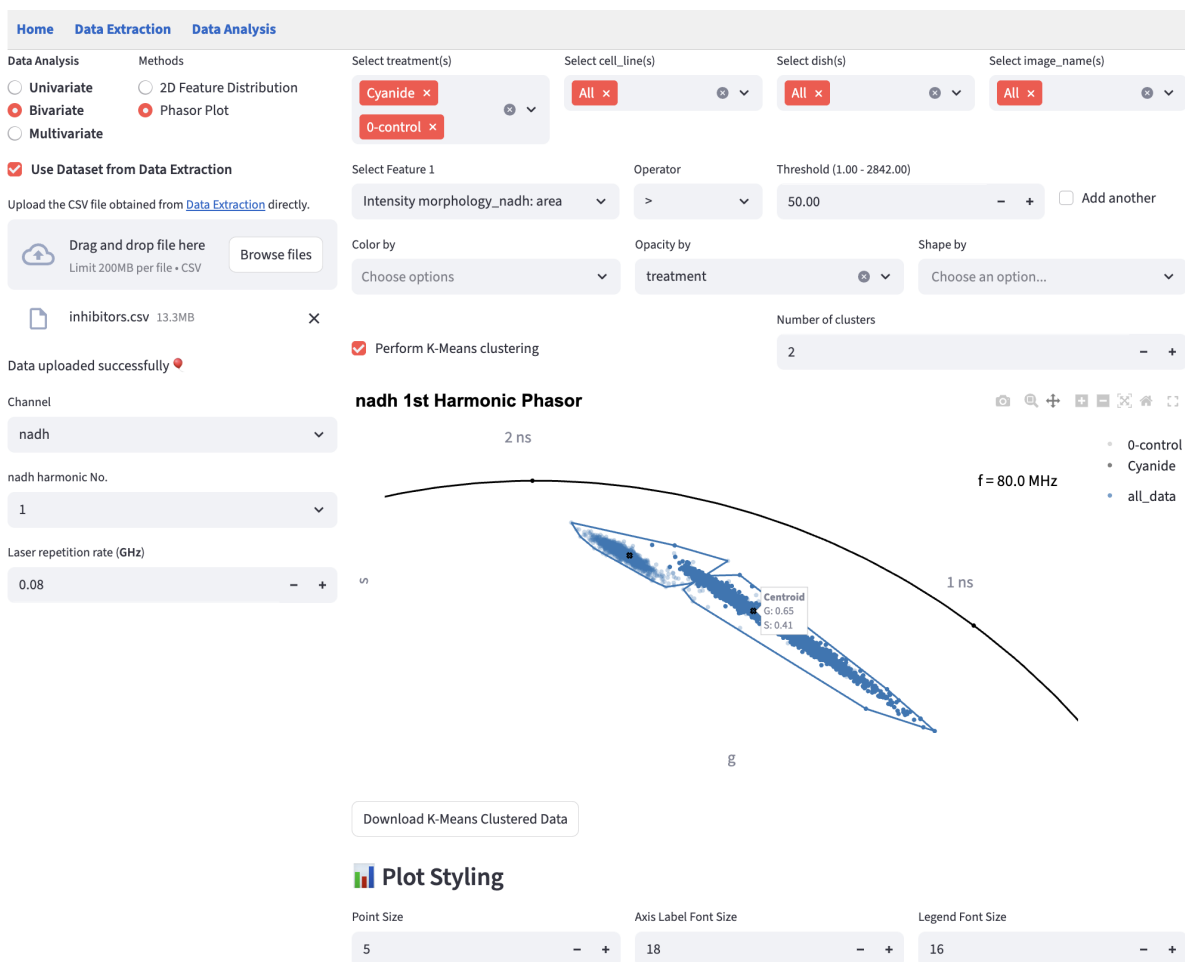
R^2 tells how much of the variability in y is captured by the model, compared to just predicting the $\bar{y}$.

i R^2

- $R^2 = 1 \rightarrow$ perfect prediction.
- $R^2 = 0 \rightarrow$ model predicts no better than the mean.

18 Phasor Analysis

This method focuses on visualizing the distribution of the 1st harmonic or 2nd harmonic phasor coordinates, extracted by the [Lifetime fit free](#) extractor, and performs optional clustering to highlight subpopulations or structural patterns.



18.1 Shared Interface Components

- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Below the filters, users can apply the [visual channels widgets](#) that allow users to Color by, Opacity by, and Shape by categorical features.
- On the bottom right, users can change the plot style using the [plot styling widgets](#).

18.2 Select Phasor Coordinate

Instead of using the [bivariate selection widgets](#) that select two generic numerical features, this method renders a dropdown menu to select the channel and another to select the harmonic.

Channel

nadh|

nadh harmonic No.

1

18.3 K-means Clustering

Phasor coordinates of single-cell ROIs from the selected channel and harmonic are plotted in the phasor space. K-means clustering can be performed for each color group (created by [Color by](#)). If [Perform K-means Clustering](#) is checked, users can specify the number of clusters to be created based on their prior knowledge. In the future, automatic tuning for this parameter, akin to selecting the number of components of [GMM](#), using silhouette analysis will be added.

Number of clusters

☒ Perform K-Means clustering

2 - +

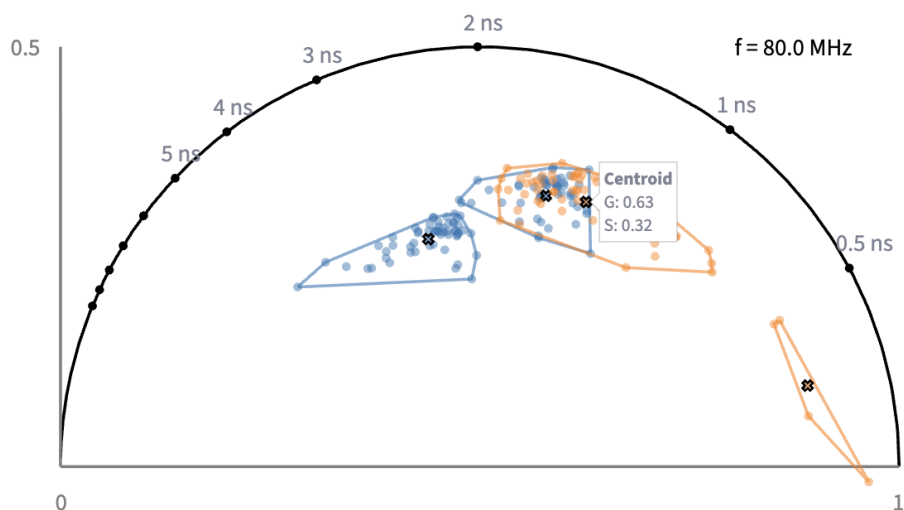
K-means clustering is performed given the number of clusters specified for each color group. Mathematically, it is an iterative algorithm that finds the best cluster assignment with regard to what cluster each point belongs to and what centroid the cluster has. The *best* is defined as the one that minimizes the sum of the squared distances between each point and the center of the cluster it belongs to.

$$\min_{\{C_k\}_{k=1}^K} \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu(C_k)\|_2^2, \quad \mu(C_k) = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i.$$

First, FLIM Playground standardizes the phasor coordinates because g (real part) and s (imaginary) are not necessarily on the same scale.

Then, it performs k-means clustering using the `sklearn.cluster.KMeans` function. All the other hyperparameters control the optimization process (e.g. `init` controls the initialization of the centroids), and they are set to their [default values](#). 42 is used as the random seed to ensure reproducibility.

To visualize the result of clustering, FLIM Playground draws a polygon (convex hull) that contains all the points of each cluster, the color of which is determined by the color group the cluster belongs to. It uses the `scipy.spatial.ConvexHull` function to calculate the convex hull. The center of each cluster is marked as a black bordered cross and users can hover over it to see its coordinates.



18.3.1 Export Clustered Dataset

Users can download the clustered dataset by clicking:

[Download K-Means Clustered Data](#)

The downloaded dataset keeps all the features plus the new categorical feature column (`k_means_cluster`), which is recognized by any method in Data Analysis as a categorical feature like others. The rows that pass the [filters](#) are included.

Part V

Multivariate Analysis

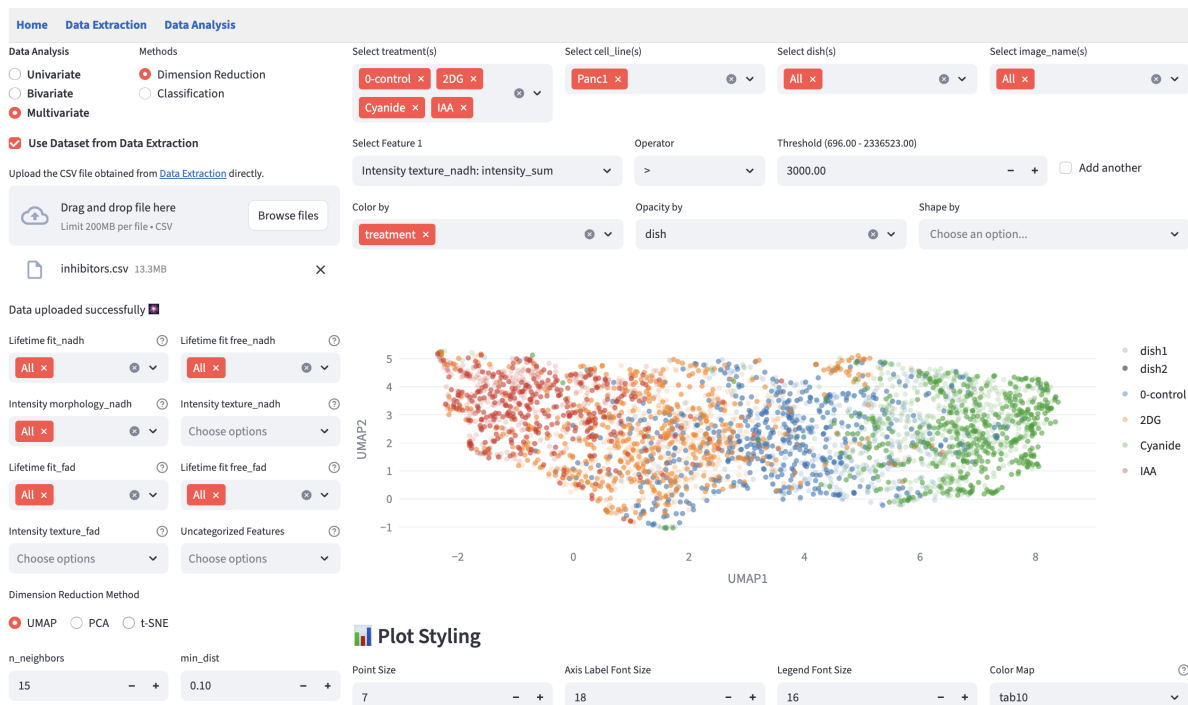
19 Dimension Reduction

To represent and visualize high-dimensional data in a meaningful way and help users interpret the data, dimension reduction methods project the data into a lower-dimensional space. Three popular dimension reduction methods are often used: UMAP, PCA, and t-SNE.

Principal Component Analysis (PCA) is a linear dimension reduction method that projects the data onto the directions of the largest variance. It is fast, deterministic, and easier to interpret (the percentage of variance explained by each principal component is reported on the axis labels), but it is limited by the linear nature of the projection. For a detailed discussion of PCA, please refer to [this post](#).

Uniform Manifold Approximation and Projection (UMAP) and t-Distributed Stochastic Neighbor Embedding (t-SNE) are non-linear dimension reduction methods that can capture more complex patterns in the data. They are more flexible and can handle non-linear relationships between features, but they are more computationally expensive, less interpretable, not deterministic (FLIM Playground uses the random seed 42 to ensure reproducibility), and dependent on the hyperparameters. Here is a [post](#) that explains how UMAP and t-SNE work at a high level and the meaning of some of the hyperparameters.

The numerical features are standardized before the dimension reduction.



19.1 Shared Interface Components

- On the left, users can use the [selection widgets](#) to select multiple numerical features from each feature group.
- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Below the filters, users can apply the [visual channels widgets](#) that allow users to Color by, Opacity by, and Shape by categorical features.
- On the bottom right, users can change the plot style using the [plot styling widgets](#).

19.2 Hyperparameter Widget

Hyperparameters really matter, so we provide a set of interactive widgets to help users change the hyperparameters and see their effects in real time.

19.2.1 UMAP

FLIM Playground uses the `umap-learn` implementation of UMAP. It chooses to expose `n_neighbors` and `min_dist` for users to tweak. Other hyperparameters are set to [default values](#).

19.2.2 t-SNE

FLIM Playground uses the `sklearn.manifold.TSNE` implementation of t-SNE. It chooses to expose `perplexity` and `early_exaggeration` for users to tweak. Other hyperparameters are set to [default values](#).

☒ UMAP ☐ PCA ☐ t-SNE

n_neighbors

15

-

+

min_dist

0.10

-

+

☐ UMAP ☒ PCA ☐ t-SNE

☐ UMAP ☐ PCA ☒ t-SNE

perplexity

15

–

+

early_exaggeration

1

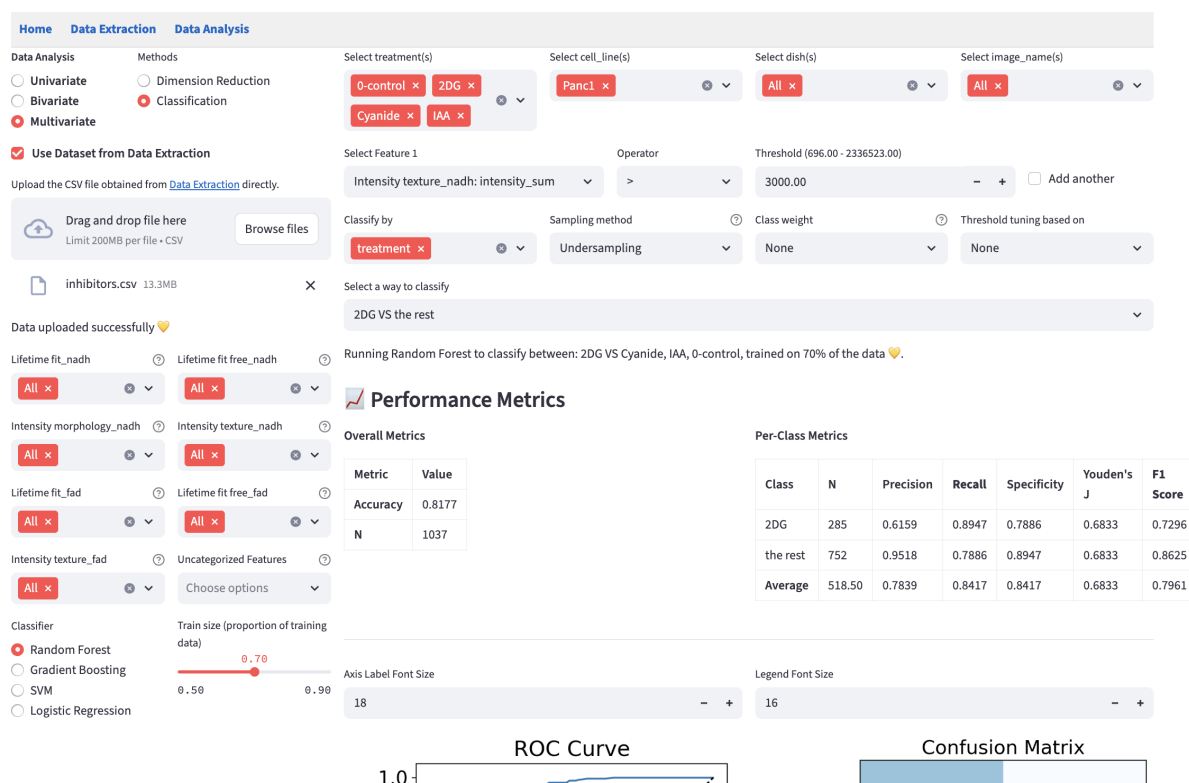
–

+

20 Classification

A machine learning classifier learns patterns from categorized data and then predicts the category of new, unseen data. This module provides a set of machine learning classifiers and interactive widgets to help users perform classification.

All the classifiers are implemented using `scikit-learn`. The confusion matrix and the ROC curve come from `sklearn.metrics`; the remaining [performance metrics](#) are computed directly from that confusion matrix using the standard formulas given below, which avoids re-scanning the predictions once per metric and covers quantities such as specificity and Youden's J that `sklearn.metrics` does not provide. A fixed random seed (42) is used for all the classifiers to ensure reproducibility.



The performance metric plots are cut off in the screenshot. See the [performance metrics section](#) for the complete view.

An overview of the pipeline:

- Choose the numerical features on the left.
 - The z-score normalization is applied to the features before training if the chosen classifier is SVM or Logistic Regression, because they are sensitive to feature magnitudes, while Random Forest and Gradient Boosting are not.
- Choose the classifier and train-test split.
- Filter the dataset.
- Choose categorical features to form classes.
- Choose any of the methods to handle [class imbalance](#).
- Choose the classes to be classified.
- Look at the [performance metrics](#), [confusion matrix](#), and [ROC curve](#).
- Iterate over the previous options to improve the performance with regard to the metrics you care about.

20.1 Shared Interface Components

- On the left, users can use the [selection widgets](#) to select numerical features for the classifier. Multiple features can be selected from each feature group.
- On the top right, users can use the [filters](#) to subset the data to find the groups of interest.
- Above the performance metrics plots, users can change the plot style using the [plot styling widgets](#).

20.2 Method-specific Components

- Choose the classifier and train-test split: the slider controls the proportion of the data used to train the classifier and the rest is used as unseen data to test its generalizability. The following classifiers are available:
 - [Random Forest](#)
 - Gradient Boosting
 - Support Vector Machine (SVM)
 - [Logistic Regression](#)

The [train-test split](#) is used to evaluate the generalizability of the classifier. The Validation set is used in hyperparameter tuning. Since the classifiers above do not have a lot of hyperparameters to tune, it does not partition a separate validation set. However, k-fold cross-validation is used in tuning the threshold.

⚠ The split is by row, not by field of view

Both the train-test split and the cross-validation used in threshold tuning operate on individual rows (i.e. single cells), stratified by class. They do not group rows by field of view, so cells segmented from the same image can end up in both the training set and the test set.

Cells from one image are usually more similar to each other than to cells from a different image, because they share a field of view, an acquisition session, and often a well. When such cells appear on both sides of the split, the reported metrics can look better than the classifier's true performance on a genuinely new image. Keep this in mind when interpreting the [performance metrics](#), particularly when the dataset contains many cells drawn from relatively few fields of view.



- **Classify by:** The **Color by** from the [visual channels widgets](#) is renamed to **Classify by** and it is used to form classes based on (a combination of) selected categorical features. Other visual channels are not supported in this method.

- **Choose the classes to be classified:** classes are formed based on the combinations of categories of the selected categorical features. In general, it supports binary classification and in general n-way classification where n is the number of classes. If one categorical feature is selected and after filtering has 3 categories, then all possible ways to classify are produced including the three-way classification. Additionally, the **class x vs the rest** option is provided for each class.

Select a way to classify

sodium arsenite VS the rest

sodium arsenite VS the rest

2DG VS the rest

0-control VS the rest

2DG VS sodium arsenite

0-control VS sodium arsenite

0-control VS 2DG

0-control VS 2DG VS sodium arsenite

20.2.1 Classifier Hyperparameters

The classifier hyperparameter panel is organized per classifier with three roles:

- **Model-Form/Capacity:** defines the hypothesis family or expressive power.
- **Regularization:** constrains model complexity to reduce overfitting.
- **Optimization:** affects training convergence and runtime behavior.

20.2.1.1 Random Forest

i Parameter table			
Param	Role	Effect	Priority
n_estimators	Model-Form/Capacity	More trees increase ensemble capacity; gains usually show diminishing returns after enough trees.	High
max_depth	Regularization	Caps tree depth to reduce overfitting; also directly limits capacity.	High

<code>min_samples_split</code>	Regularization	Requires more samples before splitting, reducing fragmentation on small partitions.	Medium
<code>min_samples_leaf</code>	Regularization	Enforces minimum leaf size, smoothing predictions.	Medium
<code>max_features</code>	Regularization	Fewer features per split increase tree diversity and reduce variance/overfitting.	Medium-High

20.2.1.2 Gradient Boosting

i Parameter table			
Param	Role	Effect	Priority
<code>n_estimators</code>	Model-Form/Capacity	More stages increase additive model capacity; strongly interacts with <code>learning_rate</code> .	High
<code>learning_rate</code>	Regularization	Shrinks each stage contribution; lower values typically need more trees and often generalize better (also affects optimization dynamics).	High
<code>max_depth</code>	Regularization	Limits base learner complexity; also limits capacity.	High
<code>subsample</code>	Regularization	<code>< 1.0</code> introduces stochastic boosting, reducing variance and overfitting risk.	High

20.2.1.3 Support Vector Machine (SVM)

i Parameter table

Param	Role	Effect	Priority
C	Regularization	Higher C penalizes training errors more, which usually weakens regularization.	High
kernel	Model-Form/Capacity	Chooses hypothesis family (linear vs non-linear kernels).	High
gamma	Model-Form/Capacity	Controls influence radius of samples for rbf/poly/sigmoid; higher values create tighter boundaries.	High
degree	Model-Form/Capacity	Polynomial order for poly kernel; higher degree increases flexibility.	Medium-High
coef0	Model-Form/Capacity	Kernel offset for poly/sigmoid, shifting boundary behavior.	Medium
tol	Optimization	Convergence tolerance; mostly affects runtime/termination behavior.	Low

20.2.1.4 Logistic Regression

i Parameter table

Param	Role	Effect	Priority
regularization	Regularization	Selects regularization type; available options depend on solver (see table below). Maps to l1_ratio internally.	High
C	Regularization	Inverse regularization strength; higher C means weaker regularization.	High
l1_ratio	Regularization	Balances l1 vs l2 when solver='saga' and regularization='elasticnet'.	Medium
fit_intercept	Regularization	Disabling intercept is a mild structural constraint.	Low
solver	Optimization	Optimization algorithm; affects convergence speed and which regularization options are available.	Low-Medium
max_iter	Optimization	Iteration budget; too low can stop before convergence.	Low-Medium
tol	Optimization	Convergence threshold; mainly runtime-sensitive.	Low

Regularization options by solver:

Solver	Available regularization
lbfgs	l2, none
liblinear	l1, l2
newton-cg	l2, none
newton-cholesky	l2, none
sag	l2, none
saga	l1, l2, elasticnet, none

20.2.2 Class Imbalance Handling

We provide three options for handling class imbalance (i.e., one or more classes in a dataset have significantly fewer samples than others, which would otherwise cause a classifier to bias toward the majority class). Note: to prevent data leakage, all the methods below are applied to the training set only.

- **Sampling methods:**

- None: stratified random sampling of the data on classification classes, regardless of the class size. It is implemented using `train_test_split` from `sklearn.model_selection`.
- Undersampling: All classes are downsampled to the size of the smallest class. It is implemented using `RandomUnderSampler` from [imbalanced-learn](#).
- Oversampling: All classes are upsampled to the size of the largest class by randomly duplicating samples from minority classes. It is implemented using `RandomOverSampler` from `imbalanced-learn`.

Sampling method

None

None

Undersampling

Oversampling

- **Class weights:**

- None: no class weights.
- Balanced: the class weights are inversely proportional to the class size (support). Note: Gradient Boosting does not support class weights in the scikit-learn implementation, and thus this option is not available for it.

- **Threshold tuning:** Whereas the previous two options influence the probability estimation part of the classifier, this option adjusts only the decision boundary used to convert predicted probabilities into class labels.

- None: no threshold tuning. The default decision is always the class that has the highest predicted probability (in binary, anything above 0.5; in multi-class, the class with the highest predicted probability).

- **Balanced Accuracy**: the threshold is set to maximize the balanced accuracy score, which is the unweighted average of each class's recall (i.e., the minority class's recall is as important as the majority class's recall).
- **F1 Score**: the threshold is set to maximize the unweighted average of each class's F1 score.

Threshold tuning based on

None|

None

Balanced Accuracy

F1 Score

i How does threshold tuning work?

It treats threshold selection as a separate optimization problem from model training, allowing thresholds to be tuned independently of the underlying classifier's probability estimates.

K-fold cross-validation for the probability model: The procedure begins by establishing a stratified **k-fold cross-validation** scheme (default $k=5$): the dataset is divided into k mutually exclusive subsets, and for each fold, one subset serves as the validation set while the remaining $k-1$ subsets form the training set.

The probability models are trained separately for each cross-validation fold. This process yields k distinct probability models, each trained on a different subset of the data, along with their corresponding validation set probability predictions that collectively cover all samples exactly once.

Threshold optimization: A single threshold optimization procedure is performed once using all cross-validation folds. The threshold optimization process differs for binary and multi-class classification problems. For binary classification, a single threshold parameter $\theta \in [0, 1]$ is optimized, where class 1 is predicted when $P(\text{class 1} \mid x) \geq \theta$, and class 0 otherwise. For multi-class problems, a threshold vector $\theta = [\theta_1, \theta_2, \dots, \theta_k]$ is optimized simultaneously for all k classes, where thresholds are applied through a normalized probability scheme: predictions are made by selecting the class with the highest normalized probability $P(\text{class } i \mid x) / (\theta_i + \epsilon)$, where $\epsilon = 10^{-10}$ prevents division by zero.

The optimization objective function evaluates candidate thresholds by: (1) applying the same threshold(s) on top of the fold-specific probability models to generate prediction classes for the fold-specific validation set, (2) computing the specified evaluation metric (e.g., balanced accuracy, F1-score) between predicted and true labels for each fold, and (3) returning the negative mean cross-validated score across all folds (for minimization using the Nelder-Mead simplex algorithm).

Finally, the optimized threshold(s) are applied on top of the probability model trained on the entire training set to the test set to generate the final predictions.

i Threshold Tuning Example

Before threshold tuning:

Overall Metrics

Metric	Value
Accuracy	0.8854
N	4231

Per-Class Metrics

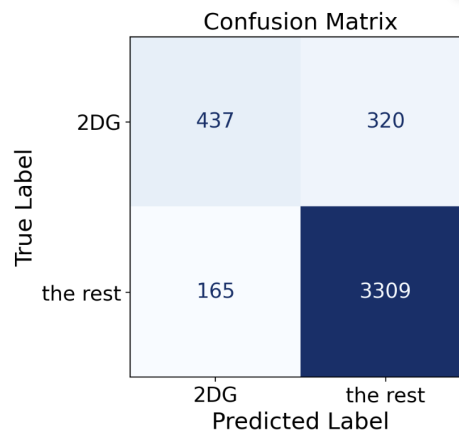
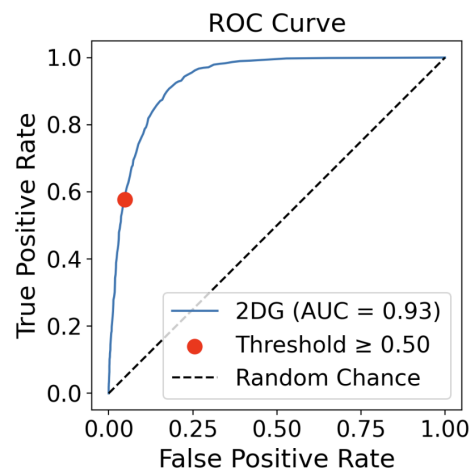
Class	N	Precision	Recall	Specificity	Youden's J	F1 Score
2DG	757	0.7259	0.5773	0.9525	0.5298	0.6431
the rest	3474	0.9118	0.9525	0.5773	0.5298	0.9317
Average	2115.50	0.8189	0.7649	0.7649	0.5298	0.7874

Axis Label Font Size

18

Legend Font Size

16



After threshold tuning based on the balanced accuracy metric:

Performance Metrics

Overall Metrics

Metric	Value
Accuracy	0.8225
N	4231

Per-Class Metrics

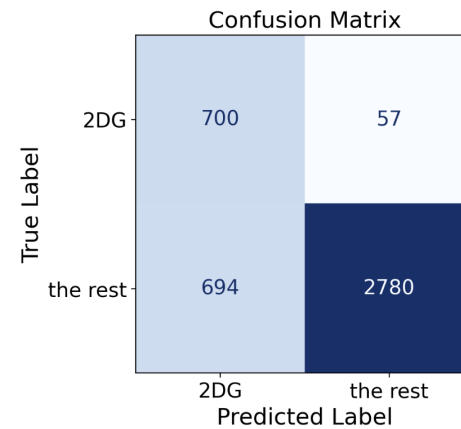
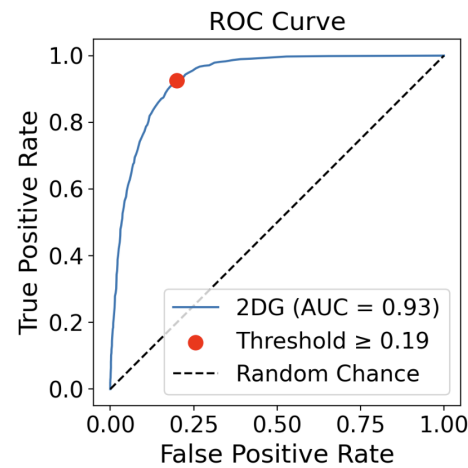
Class	N	Precision	Recall	Specificity	Youden's J	F1 Score
2DG	757	0.5022	0.9247	0.8002	0.7249	0.6509
the rest	3474	0.9799	0.8002	0.9247	0.7249	0.8810
Average	2115.50	0.7410	0.8625	0.8625	0.7249	0.7659

Axis Label Font Size

18

Legend Font Size

16

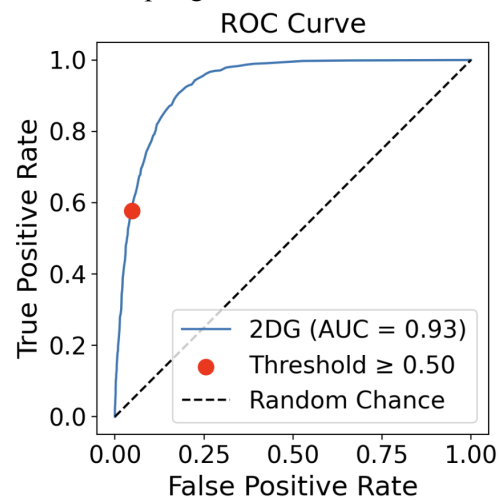


Things to highlight:

- The two ROC curves are the same, because the underlying probability models are the same.
- The annotated operating point in red is “better” *after* the threshold tuning because it achieves higher Recall (TPR) + Specificity (TNR), as can be seen in the metrics table (the optimized metrics are highlighted in red). The threshold before was 0.5, the default value, and after tuning it was optimized to 0.19. The reduction in thresholds means that the classifier is tuned to be more “confident” at predicting the positive class (2DG), which is the minority class. The effect of this threshold optimization is also visible in the confusion matrix.
- However, all of the above come at the cost of other metrics, such as the overall accuracy (0.89 to 0.82) and average precision (0.82 to 0.74). **Select the class imbalance methods/options that maximize the metrics most important to you.**

i Sampling Example

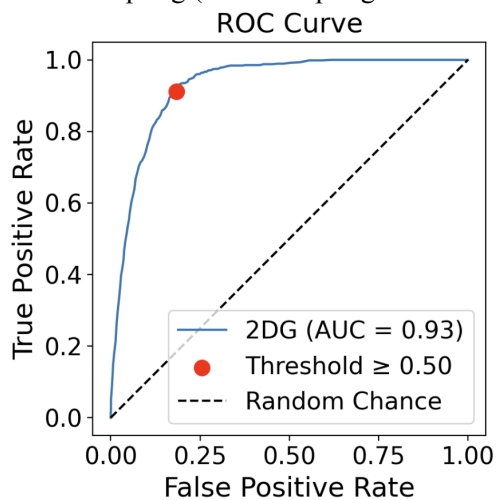
Before sampling:



Confusion Matrix

True Label	Predicted Label	
	2DG	the rest
2DG	437	320
the rest	165	3309

After sampling (undersampling was used in this case):



Confusion Matrix

True Label	Predicted Label	
	2DG	the rest
2DG	690	67
the rest	641	2833

Things to **highlight**:

- The ROC curve is different, because the underlying probability models are different.
- The thresholds before and after are the same (the default value of 0.5), but the operating points are at different locations and their associated confusion matrices are different.

20.2.3 Confusion Matrix

FLIM Playground evaluates the trained classifier on the held-out test set by converting the predicted probabilities into hard labels using the chosen threshold or by selecting the class with the highest predicted probability. The predictions are then summarized using `sklearn.metrics.confusion_matrix`. In the binary case, it is represented as:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP) – correct hits	False Negative (FN) – missed positives
Actual Negative	False Positive (FP) – false alarms	True Negative (TN) – correct rejections

20.2.4 Performance Metrics

The overall accuracy is calculated as the ratio of the sum of the true positives and true negatives to the sum of all the elements in the confusion matrix (the sum of the diagonal over the sum of all the elements).

- Accuracy: $\frac{TP+TN}{TP+FP+TN+FN}$

Other metrics are calculated for each class based on the confusion matrix:

- Precision: $\frac{TP}{TP+FP}$
- Recall (Sensitivity): $\frac{TP}{TP+FN}$
- Specificity: $\frac{TN}{TN+FP}$
- Youden's Index: Sensitivity + Specificity – 1
- F1 Score: $\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Performance Metrics

Overall Metrics

Metric	Value
Accuracy	0.8854
N	4231

Per-Class Metrics

Class	N	Precision	Recall	Specificity	Youden's J	F1 Score
2DG	757	0.7259	0.5773	0.9525	0.5298	0.6431
the rest	3474	0.9118	0.9525	0.5773	0.5298	0.9317
Average	2115.5	0.8189	0.7649	0.7649	0.5298	0.7874

Performance Metrics Example

The reason that we need metrics in addition to accuracy is because *Accuracy is Not Enough*. For a visual explanation of metrics, see [here](#). In short, it is all about trade-off.

20.2.5 ROC Curve

Another way to look at the performance of the classifier is to plot the Receiver Operating Characteristic (ROC) curve and its Area Under the Curve (AUC).

Confusion matrix looks at the performance of the classifier at a single threshold, while ROC curve looks at the performance of the classifier at all thresholds (i.e., from 0 to 1).

At the origin (0, 0), the threshold is 1, so the classifier predicts all samples as negative, making the false positive rate $FPR = \frac{FP}{FP+TN} = 0$ and the true positive rate $TPR = \frac{TP}{TP+FN} = 0$, i.e. the origin. At the point (1, 1), the threshold is 0, so the classifier predicts all samples as positive, making the false positive rate 1 and the true positive rate 1, i.e. the point (1, 1).

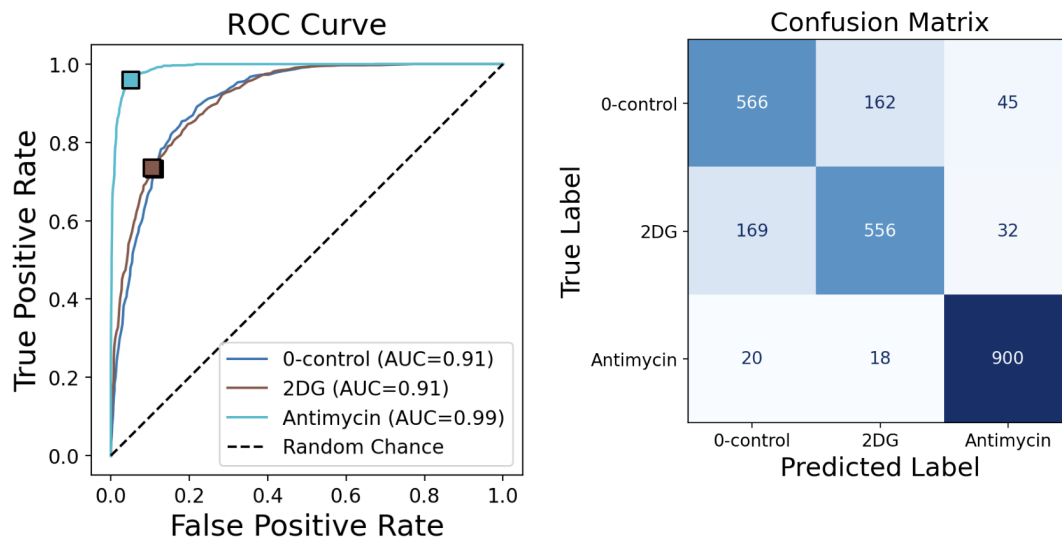
AUC summarizes the ROC curve by integrating it from 0 to 1, so it is threshold-free and has a nice interpretation: it is the probability that the classifier assigns a higher score to a randomly chosen positive sample X^+ than to a randomly chosen negative sample X^- .

The ROC curve displays the operating point (FPR, TPR) associated with the confusion matrix on the right.

For a visual explanation, see [here](#).

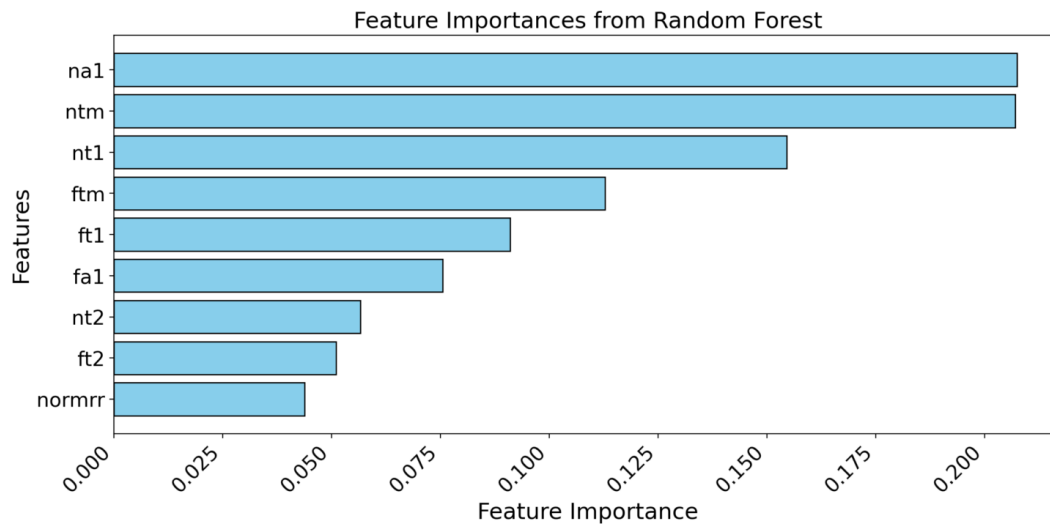
20.2.5.1 Multi-class ROC Curve

In multi-class classification, each class's ROC curve is computed as a one-vs-rest binary classification problem. The operating point (FPR, TPR) associated with the confusion matrix is displayed for each class.



20.2.6 Feature Importance

Random Forest and Gradient Boosting classifiers provide feature importance scores.



21 Common Q & A

This chapter addresses frequently asked questions about FLIM Playground.

Q: Why can't I find a certain **numerical feature** under numerical selection/filter widgets?

A: If you cannot find a certain **numerical feature**, try the following:

- look inside the Uncategorized Features feature group
- inspect the dataset (e.g., using Excel filters) and look for non-numeric values in that column. Stray non-numeric values (e.g., “-”) are tolerated as long as they make up **1% or less** of the column's non-empty values; FLIM Playground converts the column anyway and turns just those cells into NaN, telling you how many it converted. If more than 1% of the column is non-numeric, the column stays as text and will not be recognized. In that case, clear those cells (Excel cells, not real cells), save the file, and upload it again.

Q: Why can't I find a certain **categorical feature** under categorical filter/visual channel widgets?

A:

- If the uploaded dataset was extracted by [Data Extraction](#), FLIM Playground will only match categorical features specified in the [Data Extraction configuration](#). Otherwise, FLIM Playground will only match categorical features specified in the [configuration](#).
 - If the categorical feature was in the corresponding configuration, and it is still not visible:
 - * If you just added it, FLIM Playground may not have picked it up yet. Try refreshing the page.
 - * If you have already added it, then there is only one unique value in that categorical feature column. FLIM Playground will not show it.
- It is ok if your dataset does not have all of the categorical features specified in either of the configurations.

Q: How do I change the FLIM Playground's display mode (i.e., dark mode or light mode)?

A: FLIM Playground uses the browser's current display mode. You can change it by going to the browser's settings and changing the display mode.

1. Datta, R., Heaster, T. M., Sharick, J. T., Gillette, A. A. & Skala, M. C. [Fluorescence lifetime imaging microscopy: fundamentals and advances in instrumentation, analysis, and applications](#). *Journal of Biomedical Optics* **25**, 071203 (2020).

2. Stringer, C., Wang, T., Michaelos, M. & Pachitariu, M. Cellpose: A generalist algorithm for cellular segmentation. *Nature methods* **18**, 100–106 (2021).
3. Gohlke, C., Pannunzio, B., Schüty, B. & Blanco, R. Phasorpy/phasorpy: v0.7. Zenodo <https://doi.org/10.5281/zenodo.16923774> (2025).
4. Gottlieb, D., Asadipour, B., Kostina, P., Ung, T. P. L. & Stringari, C. [FLUTE: A python GUI for interactive phasor analysis of FLIM data](#). *Biological Imaging* **3**, e21 (2023).
5. Kapsiani, S. *et al.* FLIMPA: A versatile software for fluorescence lifetime imaging microscopy phasor analysis. *Analytical Chemistry* (2025).
6. Becker, W. *The Bh TCSPC Handbook*. (Becker & Hickl GmbH, 2021).
7. Stirling, D. R. *et al.* [CellProfiler 4: Improvements in speed, utility and usability](#). *BMC Bioinformatics* **22**, 433 (2021).
8. Stirling, D. R., Carpenter, A. E. & Cimini, B. A. [CellProfiler analyst 3.0: Accessible data exploration and machine learning for image analysis](#). *Bioinformatics* **37**, 3992–3994 (2021).
9. Samimi, K. *et al.* Segmentation-guided photon pooling enables robust single cell analysis and fast fluorescence lifetime imaging microscopy. *bioRxiv* <https://doi.org/10.1101/2025.09.30.679660> (2025) doi:[10.1101/2025.09.30.679660](https://doi.org/10.1101/2025.09.30.679660).
10. Samimi, K. *et al.* [Autofluorescence lifetime flow cytometry with time-correlated single photon counting](#). *Cytometry Part A* **105**, 607–620 (2024).
11. Warren, S. C. *et al.* [Rapid global fitting of large fluorescence lifetime imaging microscopy datasets](#). *PLoS ONE* **8**, e70687 (2013).
12. Cleveland, W. S. & McGill, R. [Graphical perception: Theory, experimentation, and application to the development of graphical methods](#). *Journal of the American Statistical Association* **79**, 531–554 (1984).
13. Virtanen, P. *et al.* [SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python](#). *Nature Methods* **17**, 261–272 (2020).
14. Shannon, C. E. [A mathematical theory of communication](#). *The Bell System Technical Journal* **27**, 379–423 (1948).
15. Pham, D. L. *et al.* [Autofluorescence imaging reveals the impact of cryopreservation on t cell metabolism and activation response](#). *Molecular Therapy Advances* **34**, 201704 (2026).